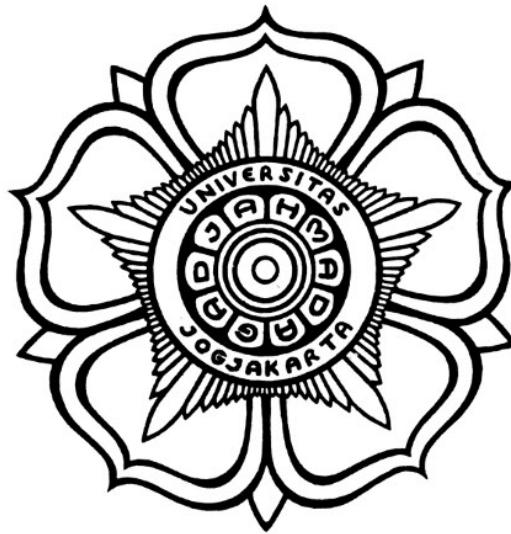


**RANCANG BANGUN SISTEM *BLOCKCHAIN* APLIKASI
MEETCOIN MENGGUNAKAN KERANGKA KERJA
HYPERLEDGER FABRIC**

SKRIPSI



Disusun oleh:

LUTHFI IZZUDDIN HANIF

20/463605/TK/51597

**PROGRAM STUDI TEKNOLOGI INFORMASI
DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA
2024**

HALAMAN PENGESAHAN
RANCANG BANGUN SISTEM *BLOCKCHAIN* APLIKASI MEETCOIN
MENGGUNAKAN KERANGKA KERJA HYPERLEDGER FABRIC
SKRIPSI

Diajukan sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi Fakultas Teknik
Universitas Gadjah Mada



Disusun oleh :
Luthfi Izzuddin Hanif
20/463605/TK/51597

Telah disetujui dan disahkan
pada tanggal, 04 November 2024

Dosen Pembimbing I

Teguh Bharata Adji, S.T., M.T., M.Eng., Ph.D.
NIP. 196909201995121001

Dosen Pembimbing II

Ir. Noor Akhmad Setiawan, S.T., M.T., Ph.D., IPM.
NIP. 197506071999031002



SKRIPSI

**RANCANG BANGUN SISTEM *BLOCKCHAIN* APLIKASI MEETCOIN
MENGUNAKAN KERANGKA KERJA HYPERLEDGER FABRIC**

Dipersiapkan dan disusun oleh

Luthfi Izzuddin Hanif
20/463605/TK/51597

Telah dipertahankan di depan dewan penguji
pada tanggal : **04 November 2024**

Susunan Dewan Penguji

Pembimbing Utama



Teguh Bharata Adji, S.T., M.T., M.Eng., Ph.D.

Pembimbing Pendamping



**Ir. Noor Akhmad Setiawan, S.T., M.T., Ph.D.,
IPM.**

Anggota Dewan Penguji Lain



Dr. Ir. Guntur Dharma Putra, S.T., M.Sc.



Prof. Dr. Ir. Risanuri Hidayat, M.Sc., IPM.

Skripsi ini telah diterima sebagai salah satu persyaratan
untuk memperoleh gelar Sarjana Teknik

Tanggal: 13 Januari 2025

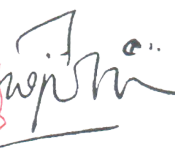
Pengelola Program Studi: Sarjana Teknologi Informasi



Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM.

NIP 198001012015041002

Mengetahui,
Ketua Departemen Teknik Elektro dan Teknologi Informasi



Prof. Ir. Hanung Adi Nugroho, S.T., M.Eng., Ph.D., IPM., SMIEEE.

NIP 197802242002121001



PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

Nama : Luthfi Izzuddin Hanif
NIM : 20/463605/TK/51597
Tahun terdaftar : 2020
Program Studi : S1 Teknologi Informasi
Fakultas : Teknik Universitas Gadjah Mada

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, 19 Oktober 2024



Luthfi Izzuddin Hanif
20/463605/TK/51597

HALAMAN PERSEMBAHAN

Bismillaahirrahmaanirrahiim.

Dengan nama Allah Yang Maha Pengasih, Maha Penyayang.

Tugas akhir ini kupersembahkan kepada kedua orang tuaku. Kupersembahkan pula kepada keluarga dan teman-teman, serta untuk bangsa, negara, dan agamaku.

KATA PENGANTAR

Puja dan puji syukur ke hadirat Allah *Subhaanahu wa ta'aala* atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga tugas akhir berupa penyusunan skripsi dengan judul "Rancang Bangun Sistem *Blockchain* Aplikasi MeetCoin Menggunakan Kerangka Kerja Hyperledger Fabric" ini telah terselesaikan dengan baik. Dalam hal penyusunan tugas akhir ini, penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu, pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Bapak Prof. Ir. Hanung Adi Nugroho, Ph.D., IPM, SMIEEE, MACM selaku Ketua Departemen Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada.
2. Bapak Ir. Lesnanto Multa Putranto, S.T., M.Eng., Ph.D., IPM., selaku Sekretaris Departemen Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada.
3. Bapak Teguh Bharata Adji, S.T., M.T., M.Eng., Ph.D., selaku dosen pembimbing pertama yang senantiasa memberikan bimbingan dan masukan yang terbaik dalam pelaksanaan serta penulisan laporan tugas akhir.
4. Bapak Ir. Noor Akhmad Setiawan, S.T., M.T., Ph.D., IPM., selaku dosen pembimbing kedua yang telah memberikan arahan dan masukan teknis yang terbaik dalam pelaksanaan tugas akhir.
5. Seluruh dosen dan tenaga pendidik Departemen Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada yang telah memberikan bekal ilmu kepada penulis selama menjalani masa perkuliahan.
6. Bapak Gatot Sadewo dan Ibu Titik Indrayati, selaku kedua orang tua penulis yang senantiasa memanjatkan doanya serta memberikan dukungan, semangat, dan motivasi kepada penulis dalam pengerjaan tugas akhir.
7. Khalda Shahiba Chairunnisa, sahabat masa kecil penulis yang selalu memberikan dukungan dan semangatnya kepada penulis selama pengerjaan tugas akhir.
8. Teman-teman Program Studi Teknologi Informasi Departemen Teknik Elektro dan Teknologi Informasi Universitas Gadjah Mada Angkatan 2020 yang telah mene mani dan membantu penulis selama menjalani masa perkuliahan.

Tiada gading yang tak retak. Penulis menyadari bahwa laporan skripsi ini masih jauh dari kata sempurna. Oleh karena itu, penulis sangat mengharapkan segala bentuk masukan, saran, dan kritik yang membangun. Akhir kata, penulis berharap semoga tulisan ini dapat memberikan manfaat dan wawasan tambahan bagi kita semua, *aamiin*.

Yogyakarta, Oktober 2024

Luthfi Izzuddin Hanif

DAFTAR ISI

HALAMAN PENGESAHAN	ii
PERNYATAAN BEBAS PLAGIASI	iii
HALAMAN PERSEMBAHAN	iv
KATA PENGANTAR	v
DAFTAR ISI	vi
DAFTAR TABEL	ix
DAFTAR GAMBAR	x
DAFTAR SINGKATAN.....	xi
INTISARI.....	xii
ABSTRACT	xiii
BAB I Pendahuluan	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	4
1.3 Tujuan Penelitian	4
1.4 Batasan Penelitian	5
1.5 Manfaat Penelitian	5
1.6 Sistematika Penulisan.....	5
BAB II Tinjauan Pustaka dan Dasar Teori	7
2.1 Tinjauan Pustaka	7
2.1.1 Pengembangan Sistem <i>Blockchain</i> dari Aplikasi MeetCoin Sebe- lumnya	7
2.1.2 Pengembangan Sistem Berbasis <i>Blockchain</i> Menggunakan Ke- rangka Kerja Hyperledger Fabric.....	8
2.2 Dasar Teori	13
2.2.1 <i>Blockchain</i>	13
2.2.1.1 Struktur <i>Blockchain</i>	14
2.2.1.2 Algoritme Konsensus	15
2.2.1.3 Tipe <i>Blockchain</i>	18
2.2.2 Jaringan <i>Peer-to-Peer</i>	19
2.2.3 Hyperledger Fabric.....	20
2.2.3.1 Arsitektur Hyperledger Fabric	21
2.2.3.2 Hyperledger Fabric <i>Ledger</i>	23
2.2.3.3 Alur Transaksi Hyperledger Fabric	24
2.2.4 Hyperledger Caliper	27
2.2.5 <i>Application Programming Interface</i>	27
2.2.6 Bahasa Pemrograman Go	28

2.3	Analisis Perbandingan Metode	29
2.3.1	Perbandingan Hyperledger Fabric dengan Kerangka Kerja <i>Blockchain</i> Lain	29
2.3.2	Pemilihan Algoritme Konsensus Hyperledger Fabric	31
2.3.3	Pemilihan <i>State Database</i> Hyperledger Fabric	34
2.3.4	Perbandingan Hyperledger Caliper dengan Alat Pengujian Lain ...	36
BAB III Metode Penelitian.....		38
3.1	Alat dan Bahan Tugas Akhir	38
3.1.1	Alat Tugas Akhir	38
3.1.2	Bahan Tugas Akhir.....	38
3.2	Metode yang Digunakan.....	39
3.2.1	Metode Perancangan	39
3.2.1.1	Analisis Kebutuhan Sistem.....	39
3.2.1.2	Perancangan <i>Activity Diagram</i>	40
3.2.1.3	Perancangan Arsitektur Sistem <i>Blockchain</i>	46
3.2.2	Metode Implementasi	47
3.2.2.1	<i>Software Development Life Cycle</i>	48
3.2.2.2	Kerangka Kerja <i>Blockchain</i>	48
3.2.2.3	Mekanisme Konsensus	49
3.2.2.4	<i>State Database</i>	49
3.2.2.5	Bahasa Pemrograman.....	49
3.2.3	Metode Pengujian	50
3.3	Alur Tugas Akhir	52
3.3.1	Mempelajari Proses Bisnis & Sistem <i>Blockchain</i> dari Aplikasi MeetCoin	52
3.3.2	Melakukan Studi Literatur.....	53
3.3.3	Menentukan Metode Penelitian	53
3.3.4	Pengembangan Sistem <i>Blockchain</i> MeetCoin	53
3.3.4.1	Instalasi Kerangka Kerja Hyperledger Fabric	53
3.3.4.2	Konfigurasi Jaringan <i>Blockchain</i>	54
3.3.4.3	<i>Deploy Chaincode</i>	54
3.3.4.4	Pengembangan Fabric Gateway	60
3.3.4.5	Pengembangan <i>Backend</i>	61
3.3.5	Pengujian Sistem <i>Blockchain</i> MeetCoin	62
3.3.5.1	Konfigurasi Hyperledger Caliper	63
3.3.5.2	Menjalankan Pengujian Hyperledger Caliper	65
BAB IV Hasil dan Pembahasan.....		69
4.1	Hasil Pengembangan Sistem <i>Blockchain</i> MeetCoin	69
4.1.1	Jaringan Fabric	69

4.1.2	Fabric Gateway	71
4.1.3	<i>Backend</i>	71
4.2	Hasil Evaluasi Kinerja Sistem <i>Blockchain</i> MeetCoin	72
4.2.1	Pengujian Pengaruh Jumlah <i>Worker</i> terhadap Kinerja Sistem	72
4.2.2	Pengujian Pengaruh Jumlah Transaksi terhadap Kinerja Sistem ...	74
4.2.3	Pengujian Pengaruh Laju Transaksi terhadap Kinerja Sistem	75
4.3	Perbandingan Hasil Penelitian dengan Hasil Terdahulu	77
BAB V	Kesimpulan dan Saran	80
5.1	Kesimpulan	80
5.2	Saran	81
DAFTAR PUSTAKA		82
LAMPIRAN		L-1
L.1	<i>Source Code</i> Konfigurasi <i>Node</i> Jaringan Fabric	L-1
L.2	<i>Source Code</i> Konfigurasi Certificate Authority (CA) Jaringan Fabric	L-6
L.3	<i>Source Code</i> Konfigurasi <i>Channel</i> dan Mekanisme Konsensus Jaringan Fabric	L-8
L.4	<i>Source Code</i> Fungsi <i>Mint</i> pada <i>Chaincode</i>	L-12
L.5	<i>Source Code</i> Fungsi <i>Burn</i> pada <i>Chaincode</i>	L-15
L.6	<i>Source Code</i> Fungsi <i>Transfer</i> pada <i>Chaincode</i>	L-18
L.7	<i>Source Code</i> Konfigurasi <i>Network</i> pada Caliper	L-21
L.8	<i>Source Code</i> Konfigurasi <i>Benchmark</i> pada Caliper	L-22
L.9	<i>Source Code</i> Modul <i>Workload Mint</i> pada Caliper	L-24
L.10	<i>Source Code</i> Modul <i>Workload Balance</i> pada Caliper	L-25
L.11	<i>Source Code</i> Modul <i>Workload Transfer</i> pada Caliper	L-26

DAFTAR TABEL

Tabel 2.1	Perbandingan Penelitian Terdahulu dengan Tugas Akhir	11
Tabel 2.2	Perbandingan Hyperledger Fabric, Hyperledger Besu, dan R3 Corda [1–4]	29
Tabel 2.3	Perbandingan Mekanisme Konsensus Kafka, Raft, dan PBFT [5–7] .	32
Tabel 2.4	Perbandingan <i>State Database</i> LevelDB dan CouchDB [8–10]	35
Tabel 2.5	Perbandingan Hyperledger Caliper dan BlockBench [11, 12]	37
Tabel 4.1	Komponen Arsitektur Sistem <i>Blockchain</i> MeetCoin	69
Tabel 4.2	Hasil Pengujian Pengaruh Jumlah <i>Worker</i> pada Pengisian Saldo	73
Tabel 4.3	Hasil Pengujian Pengaruh Jumlah <i>Worker</i> pada Pengecekan Saldo ..	73
Tabel 4.4	Hasil Pengujian Pengaruh Jumlah <i>Worker</i> pada Transfer Saldo	73
Tabel 4.5	Hasil Pengujian Pengaruh Jumlah Transaksi pada Pengisian Saldo ..	75
Tabel 4.6	Hasil Pengujian Pengaruh Jumlah Transaksi pada Pengecekan Saldo	75
Tabel 4.7	Hasil Pengujian Pengaruh Jumlah Transaksi pada Transfer Saldo	75
Tabel 4.8	Hasil Pengujian Pengaruh Laju Transaksi pada Pengisian Saldo	76
Tabel 4.9	Hasil Pengujian Pengaruh Laju Transaksi pada Pengecekan Saldo ...	77
Tabel 4.10	Hasil Pengujian Pengaruh Laju Transaksi pada Transfer Saldo	77
Tabel 4.11	Perbandingan Sistem <i>Blockchain</i> MeetCoin yang Dikembangkan dengan Terdahulu	77

DAFTAR GAMBAR

Gambar 2.1	Arsitektur Sistem <i>Blockchain</i> Aplikasi MeetCoin Sebelumnya ...	7
Gambar 2.2	Struktur Blok pada <i>Blockchain</i>	15
Gambar 2.3	Merkle <i>Tree</i>	15
Gambar 2.4	Alur Tahapan PBFT [6].....	17
Gambar 2.5	Jaringan <i>Peer-to-Peer</i>	20
Gambar 2.6	Arsitektur Hyperledger Fabric [13]	21
Gambar 2.7	Komponen Buku Besar Hyperledger Fabric [14].....	23
Gambar 2.8	Struktur <i>World State</i> [14]	24
Gambar 2.9	Struktur <i>Blockchain</i> [14].....	25
Gambar 2.10	Alur Transaksi Hyperledger Fabric [15]	25
Gambar 2.11	Cara Kerja API.....	28
Gambar 3.1	<i>Activity Diagram</i> Proses Pendaftaran Akun	41
Gambar 3.2	<i>Activity Diagram</i> Proses Autentikasi Akun.....	42
Gambar 3.3	<i>Activity Diagram</i> Proses Melihat Profil Akun	43
Gambar 3.4	<i>Activity Diagram</i> Proses Pengisian Saldo.....	43
Gambar 3.5	<i>Activity Diagram</i> Proses Pengecekan Saldo	44
Gambar 3.6	<i>Activity Diagram</i> Proses Transfer Saldo	44
Gambar 3.7	<i>Activity Diagram</i> Proses Penarikan Saldo	45
Gambar 3.8	<i>Activity Diagram</i> Proses Melihat Riwayat Transaksi	45
Gambar 3.9	Arsitektur Sistem <i>Blockchain</i> MeetCoin.....	46
Gambar 3.10	SDLC dengan Metode <i>Agile</i> [16]	48
Gambar 3.11	Alur Tugas Akhir	52
Gambar 3.12	Konfigurasi <i>Node</i>	55
Gambar 3.13	Konfigurasi Mekanisme Konsensus Raft	56
Gambar 3.14	Fungsi <i>Mint</i>	57
Gambar 3.15	Fungsi <i>Burn</i>	58
Gambar 3.16	Fungsi <i>ClientAccountBalance</i>	59
Gambar 3.17	Fungsi <i>Transfer</i>	59
Gambar 3.18	Konfigurasi Fabric Gateway	60
Gambar 3.19	<i>Script File Register</i>	61
Gambar 3.20	<i>Struct</i> Pengguna.....	61
Gambar 3.21	Fungsi Enkripsi <i>Password</i>	62
Gambar 3.22	Fungsi <i>Generate Token JWT</i>	62
Gambar 3.23	<i>Struct</i> Transaksi	63
Gambar 3.24	Cuplikan Kode untuk Membuat Data Transaksi	63
Gambar 3.25	Konfigurasi <i>Network</i> pada Hyperledger Caliper.....	64
Gambar 3.26	Konfigurasi <i>Benchmark</i> pada Hyperledger Caliper	65
Gambar 3.27	Cuplikan Kode Modul <i>Workload Mint</i>	66
Gambar 3.28	Cuplikan Kode Modul <i>Workload Balance</i>	67
Gambar 3.29	Cuplikan Kode Modul <i>Workload Transfer</i>	68

DAFTAR SINGKATAN

API	=	Application Programming Interface
BFT	=	Byzantine Fault Tolerance
CA	=	Certificate Authority
CFT	=	Crash Fault Tolerance
CLI	=	Command Line Interface
CPU	=	Central Processing Unit
DBMS	=	Database Management System
DLT	=	Distributed Ledger Technology
JSON	=	JavaScript Object Notation
JWT	=	JSON Web Token
LTS	=	Long-Term Support
MSP	=	Membership Service Provider
Nonce	=	Number Only Used Once
OSN	=	Ordering Service Node
PBFT	=	Practical Byzantine Fault Tolerance
P2P	=	Peer-To-Peer
PoS	=	Proof-of-Stake
PoW	=	Proof-of-Work
SDLC	=	Software Development Life Cycle
TPS	=	Transactions Per Second

INTISARI

Blockchain merupakan teknologi buku besar terdistribusi yang menggabungkan jaringan *peer-to-peer* dan fungsi *hash* kriptografi untuk mencatat data transaksi dan menyimpannya dalam blok-blok yang saling terhubung. Teknologi *blockchain* mengalami perkembangan yang sangat cepat sejak diperkenalkan oleh Satoshi Nakamoto pada tahun 2009. Teknologi *blockchain* yang awalnya ditujukan untuk sektor keuangan, saat ini telah diadopsi oleh banyak sektor seperti perbankan, kesehatan, dan pendidikan. Adopsi teknologi *blockchain* ke banyak sektor juga didukung dengan munculnya berbagai macam kerangka kerja atau platform *blockchain* yang dapat disesuaikan untuk berbagai kebutuhan bisnis. Salah satu aplikasi yang mengadopsi teknologi *blockchain* yaitu MeetCoin. Namun, sistem *blockchain* dari aplikasi MeetCoin awal memiliki kesalahan pada rancangan sistem dengan hanya adanya satu *peer node* yang menjaga buku besar serta teknologi yang digunakan sudah memasuki fase *deprecated* sehingga sistem memiliki kerentanan dari sisi keandalan dan keamanan serta kinerja yang menurun akibat tidak adanya optimasi lebih lanjut. Oleh karena itu, pada tugas akhir ini akan dirancang dan dibangun ulang sistem *blockchain* MeetCoin dengan mempertimbangkan aspek keandalan, keamanan, dan kinerja sistem. Selain itu, sistem *blockchain* yang dikembangkan akan dievaluasi kinerjanya untuk mengidentifikasi kemampuan sistem dalam menangani beban kerja yang diberikan. Pengembangan sistem dilakukan menggunakan kerangka kerja Hyperledger Fabric. Hyperledger Fabric dipilih karena memiliki arsitektur modular yang memberikan fleksibilitas dalam kebutuhan pengembangan, kontrol konkurensi dalam mengeksekusi transaksi sehingga meningkatkan kinerja, dan mendukung penggunaan *smart contract* untuk mengatur logika bisnis dan tata kelola jaringan. Pengujian kinerja sistem dilakukan menggunakan alat Hyperledger Caliper. Pengujian yang dilakukan bertujuan untuk melihat efek perubahan jumlah *worker*, jumlah transaksi, dan laju transaksi terhadap *success rate*, *throughput*, *latency*, dan *resource consumption*. Hasil pengujian menunjukkan bahwa sistem dapat menangani transaksi pada semua skenario dengan tingkat keberhasilan mencapai 100%. Hasil pada skenario perubahan jumlah *worker* dan transaksi tidak berdampak pada peningkatan *throughput* secara signifikan, namun berdampak pada peningkatan penggunaan CPU dan memori. Pada skenario perubahan laju transaksi, *throughput* dan penggunaan CPU meningkat secara signifikan seiring bertambahnya laju transaksi.

Kata kunci : *blockchain, e-wallet, hyperledger fabric, hyperledger caliper, performance evaluation*

ABSTRACT

Blockchain is a distributed ledger technology that combines peer-to-peer networks and cryptographic hash functions to record transaction data and store it in interconnected blocks. Blockchain technology has experienced rapid development since it was introduced by Satoshi Nakamoto in 2009. Initially intended for the financial sector, many sectors have adopted blockchain technology, such as banking, healthcare, and education. The adoption of blockchain technology into many sectors is also supported by the emergence of various blockchain frameworks or platforms that can be customized for various business needs. One application that adopts blockchain technology is MeetCoin. However, the blockchain system of the initial MeetCoin application had an error in its design. Only one peer node maintained the ledger, and the technology used had entered the deprecated phase. Hence, the system had vulnerabilities regarding reliability, security, and decreased performance due to the lack of further optimization. Therefore, in this final project, the blockchain system of the MeetCoin application will be designed and rebuilt by considering reliability, security, and system performance. In addition, the developed blockchain system will be evaluated for its performance to identify its ability to handle the given workload. System development is conducted using the Hyperledger Fabric framework. Hyperledger Fabric was chosen for its modular architecture. It provides flexibility in development needs, concurrency control for executing transactions to enhance performance, and support for smart contracts to manage business logic and network governance. System performance testing is done using the Hyperledger Caliper tool. The test aims to see the effect of changes in the number of workers, transactions, and transaction rate on success rate, throughput, latency, and resource consumption. The test results show that the system can handle transactions in all scenarios with a success rate of 100%. The results in the scenario of changing the number of workers and transactions did not significantly impact increasing throughput. However, they had an impact on increasing CPU and memory usage. In the scenario of changing the transaction rate, throughput, and CPU usage increase significantly as the transaction rate increases.

Keywords : *blockchain, e-wallet, hyperledger fabric, hyperledger caliper, performance evaluation*

BAB I

PENDAHULUAN

1.1 Latar Belakang

Blockchain diperkenalkan oleh Satoshi Nakamoto pada tahun 2009 sebagai komponen inti dalam teknologi Bitcoin [17]. *Blockchain* pada mulanya dikenal sebagai buku besar terdistribusi (*distributed ledger*) yang mencatat semua transaksi yang terjadi di dalam Bitcoin dengan menggabungkan teknologi komunikasi *peer-to-peer* dengan kriptografi [18]. Transaksi tersebut akan dikelompokkan dalam suatu blok yang selanjutnya akan diberi fungsi *hash* kriptografi. Setiap blok di dalam *blockchain* berisikan *hash* dari blok sebelumnya, *timestamp*, *nonce* (*number only used once*), dan data transaksi yang telah di-*hash* [17]. Rantai berupa blok yang saling terhubung secara kriptografis kemudian didistribusikan kepada setiap pihak yang terhubung secara *peer-to-peer* di dalam jaringan Bitcoin. *Blockchain* pada Bitcoin bersifat publik sehingga siapa saja yang terhubung dengan internet dapat bergabung dalam jaringan dan memiliki level yang setara dengan yang lain. Identitas setiap pihak yang tergabung dalam Bitcoin bersifat *pseudonymous* yang ditandai dengan penggunaan alamat publik random sebagai identitasnya, meskipun secara teknis identitas asli pihak tersebut dapat ditelusuri dan diketahui [19].

Blockchain memiliki tiga karakteristik utama, yaitu terdesentralisasi, terpercaya, dan transparan [20]. Desentralisasi merupakan sistem yang tidak bergantung dengan suatu otoritas sentral atau terpusat baik itu berupa individu, organisasi, atau kelompok. Desentralisasi pada *blockchain* terbentuk dari setiap pihak yang terhubung secara *peer-to-peer* di dalam jaringan. Meskipun tidak adanya otoritas sentral, kepercayaan dan integritas di dalam jaringan *blockchain* dapat diperoleh dan dijamin melalui adanya algoritme konsensus [20]. Desentralisasi juga membuat *blockchain* mengeliminasi risiko *single point of failure* yang banyak terjadi pada sistem terpusat [21]. Setiap blok berisi transaksi dibagikan kepada setiap pihak yang terlibat dan terlihat sehingga masing-masing pihak dapat mengetahui semua transaksi yang terjadi di dalam *blockchain*. Selain itu, dengan adanya kombinasi *hash* pada setiap blok yang saling terhubung secara kriptografis membuat data transaksi sangat sulit untuk diubah sehingga menjadikan *blockchain* sangat aman. Jika terjadi perubahan pada suatu transaksi di dalam blok, maka *hash* blok tersebut dan blok-blok setelahnya akan berubah dan menjadi tidak valid.

Dalam beberapa tahun terakhir, teknologi *blockchain* mengalami banyak perkembangan. Salah satunya ditandai dengan munculnya proyek Hyperledger yang diprakarsai oleh Linux Foundation yang bertujuan untuk mengembangkan berbagai macam platform *blockchain* yang dapat diintegrasikan dengan bisnis [22]. Hyperledger menawarkan solusi *blockchain* skala perusahaan yang mendukung keamanan privasi dan skalabilitas serta

memberikan fleksibilitas dan efisiensi dalam penerapan teknologi *blockchain*. Perkembangan tersebut memberikan banyak kesempatan untuk menerapkan teknologi *blockchain* di berbagai sektor seperti sektor keuangan, perbankan, kesehatan, pendidikan, dan teknologi.

Adapun di sektor keuangan dan perbankan, banyak riset mengenai potensi teknologi *blockchain* yang dilakukan oleh industri keuangan dan perbankan. Salah satu contohnya yaitu Bank of America yang telah menyusun 38 paten tentang teknologi *blockchain* [23]. Motivasi di balik banyaknya industri keuangan dan perbankan melakukan riset dan tertarik untuk menerapkan teknologi *blockchain* yaitu untuk mengurangi biaya operasional dengan adanya desentralisasi dan otomatisasi, dapat mengontrol risiko dengan lebih efektif dengan memanfaatkan komunikasi *peer-to-peer*, dan mencari profit melalui cara yang inovatif [24]. Teknologi *blockchain* juga mengubah industri keuangan dan perbankan konvensional dengan mengoptimalkan berbagai layanan mulai dari penyelesaian pembayaran hingga cara kerja dari perbankan itu sendiri seperti pengawasan pinjaman [24].

Salah satu aplikasi keuangan yang mengimplementasikan *blockchain* yaitu MeetCoin. MeetCoin merupakan aplikasi *mobile* dompet digital berbasis *blockchain* yang dikembangkan oleh Prastyo (2022) [25]. Tujuan awal pengembangan aplikasi MeetCoin yaitu untuk memberikan opsi pembayaran dari aplikasi MeetingYuk karena mahalnya biaya administrasi untuk menggunakan sistem pembayaran pihak ketiga [25]. Aplikasi MeetingYuk merupakan aplikasi *e-commerce* yang menyediakan layanan penyewaan tempat untuk rapat atau pertemuan. Pengguna dapat memesan atau menyewa tempat yang disediakan oleh mitra MeetingYuk. MeetCoin memiliki proses bisnis dasar yang terdapat pada aplikasi dompet digital lain seperti autentikasi akun, pengisian saldo, melakukan transfer antar pengguna, melakukan pembayaran, dan melihat riwayat transaksi [25].

Pada aplikasi MeetCoin, *blockchain* dimanfaatkan untuk menjaga integritas data transaksi dalam buku besar sehingga data yang disimpan dalam buku besar tetap akurat, lengkap, dan konsisten. Sistem *blockchain* dari aplikasi MeetCoin dikembangkan dengan menggunakan kerangka kerja Hyperledger Fabric dan alat Hyperledger Composer yang mempermudah serta mempercepat pengembangan model bisnis. Hyperledger Fabric merupakan salah satu kerangka kerja *blockchain* privat berizin yang dikembangkan oleh proyek Hyperledger yang memungkinkan untuk mengembangkan *blockchain* skala perusahaan yang memiliki manajemen identitas dan kontrol akses. *Blockchain* berizin membuat setiap partisipan di dalam jaringan dapat diidentifikasi sesuai dengan identitas dan tugasnya masing-masing.

Dalam pengembangan awal, terdapat kesalahan rancangan pada sistem *blockchain* MeetCoin. Rancangan sistem *blockchain* MeetCoin awal hanya terdiri dari satu *peer node* yang menjaga buku besar. Hal tersebut membuat sistem memiliki kerentanan terha-

dap *single point of failure* yang dapat menyebabkan kegagalan sistem secara keseluruhan jika *peer node* tersebut mengalami masalah. Tidak adanya *peer node* lain juga menunjukkan bahwa hubungan *peer-to-peer* di dalam *blockchain* tidak terbentuk sehingga kontrol atas buku besar menjadi terpusat dan hanya dimiliki oleh satu *peer node* tersebut. Selain itu, risiko keamanan yang muncul akibat hanya terdapat satu *peer node* yang menjaga buku besar yaitu *node* tersebut lebih rentan terhadap serangan siber, karena penyerang hanya perlu menargetkan satu *node* untuk mendapatkan akses ke data sensitif atau memanipulasi data. Berbeda dengan adanya hubungan *peer-to-peer* dalam *blockchain* yang membuat penyerang harus menargetkan banyak *node* untuk dapat merusak data. Permasalahan lain akibat hanya terdapat satu *peer node* di dalam *blockchain* yaitu rentan terjadinya *bottleneck* yang akan memperlambat kinerja sistem karena keterbatasan *peer node* dalam memproses transaksi.

Dari penggunaan teknologi, sistem *blockchain* MeetCoin dikembangkan menggunakan kerangka kerja Hyperledger Fabric dengan versi 1.2.1 dan alat Hyperledger Composer. Namun, kedua teknologi yang digunakan tersebut sudah *deprecated* atau tidak digunakan lagi sehingga tidak lagi mendapat pembaruan serta dukungan baik dari sisi *bug fixes* hingga keamanan [26]. Teknologi tersebut juga tidak direkomendasikan untuk digunakan karena terdapat teknologi lain yang lebih baik dari sisi kinerja hingga keamanannya. Selain itu, seluruh kerangka kerja Hyperledger Fabric versi 1 akan dihapus dalam beberapa pembaruan mendatang [27]. Penggunaan teknologi yang sudah *deprecated* juga membuat sistem memiliki berbagai kerentanan dan kelemahan, seperti risiko celah keamanan yang dapat dieksploitasi karena tidak adanya pembaruan dari sisi keamanan, kinerja yang menurun akibat tidak adanya optimasi lebih lanjut, serta keterbatasan kompatibilitas dengan teknologi baru yang menghambat pengembangan lebih lanjut. [28].

Berdasarkan uraian di atas, implementasi *blockchain* pada aplikasi MeetCoin memiliki kelemahan dan kekurangan dari sisi rancangan sistem dan teknologi yang digunakan sehingga perlu dilakukan perbaikan agar sistem memiliki keandalan yang baik dan untuk mengurangi risiko keamanan. Oleh sebab itu, pada tugas akhir ini akan dirancang dan dikembangkan ulang sistem *blockchain* MeetCoin. Perancangan sistem dan pemilihan teknologi *blockchain* juga akan berpengaruh terhadap keandalan, keamanan, dan kinerja dari sistem yang dikembangkan. Sistem *blockchain* yang dikembangkan pada tugas akhir ini akan dievaluasi melalui pengujian kinerja sistem. Hasil evaluasi kinerja dapat menjadi acuan untuk mengidentifikasi kemampuan sistem dalam menangani beban kerja yang diberikan serta menjadi dasar untuk perbaikan dan optimasi lebih lanjut.

Pada tugas akhir ini, sistem *blockchain* MeetCoin akan dikembangkan menggunakan kerangka kerja Hyperledger Fabric. Arsitektur modular yang dapat dikonfigurasi pada Hyperledger Fabric memberikan fleksibilitas untuk dapat disesuaikan dalam memenuhi berbagai kebutuhan dan persyaratan bisnis [29]. Hal tersebut membuat Hyper-

ledger Fabric memiliki tingkat kerahasiaan, keandalan, resiliensi, dan skalabilitas yang tinggi [30]. Hyperledger Fabric memiliki kontrol konkurensi yang dapat mengeksekusi transaksi secara paralel sehingga dapat meningkatkan kinerja sistem [15]. Selain itu, Hyperledger Fabric juga mendukung penggunaan *smart contract* untuk mendefinisikan logika bisnis dan aturan tata kelola di dalam jaringan. Oleh sebab itu, Hyperledger Fabric menjadi pilihan yang cocok dan relevan dalam mengembangkan ulang sistem *blockchain* MeetCoin.

1.2 Rumusan Masalah

Permasalahan yang dirumuskan pada tugas akhir ini adalah sebagai berikut.

1. Terdapat kesalahan rancangan sistem *blockchain* MeetCoin awal dengan hanya adanya satu *peer node* yang mengakibatkan sistem rentan terhadap *single point of failure* yang menyebabkan sistem dapat mengalami kegagalan secara keseluruhan jika *peer node* mengalami masalah.
2. Penggunaan teknologi yang sudah *deprecated* atau tidak digunakan lagi pada sistem *blockchain* MeetCoin awal dapat menyebabkan kerentanan terhadap keamanan berupa risiko eksploitasi sistem karena tidak adanya pembaruan dari sisi keamanan dan kinerja yang menurun akibat tidak adanya optimasi lebih lanjut.
3. Pengembangan sistem *blockchain* MeetCoin awal tidak mampu menunjukkan kinerja dari sistem yang dikembangkan sehingga informasi mengenai kinerja sistem masih sangat terbatas sehingga dapat menyulitkan optimasi sistem di masa depan.

1.3 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah dipaparkan, tujuan dari tugas akhir ini adalah sebagai berikut.

1. Merancang dan mengembangkan sistem *blockchain* MeetCoin yang memiliki tingkat keandalan dan keamanan serta kinerja yang lebih baik menggunakan kerangka kerja Hyperledger Fabric yang mendukung arsitektur modular, kontrol konkurensi untuk eksekusi transaksi paralel, serta *smart contract* untuk mengatur logika bisnis dan tata kelola jaringan.
2. Mengevaluasi sistem *blockchain* yang dikembangkan melalui pengujian kinerja sistem untuk mengidentifikasi kemampuan sistem dalam menangani beban kerja yang diberikan dan memberikan rekomendasi serta referensi untuk pengembangan lebih lanjut di masa depan.

1.4 Batasan Penelitian

Batasan-batasan penelitian yang ditetapkan pada tugas akhir ini adalah sebagai berikut.

1. Tugas akhir hanya berfokus pada perancangan dan pengembangan sistem *blockchain* MeetCoin sehingga tidak mempertimbangkan aspek lain seperti pengalaman pengguna dan *front-end* aplikasi.
2. Evaluasi sistem *blockchain* terbatas hanya pada pengujian kinerja dari jaringan *blockchain* yang dikembangkan.
3. Sistem *blockchain* dikembangkan dan dievaluasi pada jaringan privat dan *server* lokal sehingga tidak benar-benar merepresentasikan sistem *production* secara utuh.
4. Sistem *blockchain* yang dikembangkan hanya memiliki dan digunakan oleh satu peran yaitu pengguna.
5. Sistem *blockchain* yang dikembangkan tidak terintegrasi dengan aplikasi MeetingYuk.
6. Proses bisnis yang dikembangkan merupakan proses bisnis yang terdapat pada aplikasi MeetCoin.

1.5 Manfaat Penelitian

Adapun manfaat yang dapat diperoleh dari tugas akhir ini adalah sebagai berikut.

1. Mengetahui cara kerja dan proses bisnis dari sistem *blockchain* MeetCoin.
2. Memberikan evaluasi kinerja sistem dan rekomendasi untuk pengembangan aplikasi MeetCoin lebih lanjut di masa depan.
3. Memberikan referensi untuk mengembangkan sistem *blockchain* di bidang lain.

1.6 Sistematika Penulisan

Sistematika penulisan pada laporan yang didasarkan pada pembahasan setiap Bab adalah sebagai berikut.

Bab I membahas tentang pendahuluan yang berisikan latar belakang penelitian, perumusan masalah, tujuan penelitian, batasan penelitian, manfaat penelitian, dan sistematika penulisan laporan.

Bab II berisi tentang pembahasan penelitian-penelitian sebelumnya yang terkait dengan pengembangan sistem *blockchain* MeetCoin, pengembangan sistem *blockchain* menggunakan kerangka kerja Hyperledger Fabric, dasar teori penelitian, dan analisis perbandingan metode.

Bab III membahas tentang metode penelitian yang meliputi alat dan bahan penelitian, metode yang digunakan, dan langkah-langkah yang dilakukan dalam penelitian.

Bab IV membahas tentang hasil pengembangan sistem *blockchain* MeetCoin dan hasil evaluasi kinerjanya serta membandingkan sistem yang dikembangkan dengan sistem terdahulu.

Bab V berisi tentang kesimpulan dari tugas akhir serta saran yang dapat dipertimbangkan untuk ditambahkan atau dikembangkan pada penelitian atau tugas akhir selanjutnya.

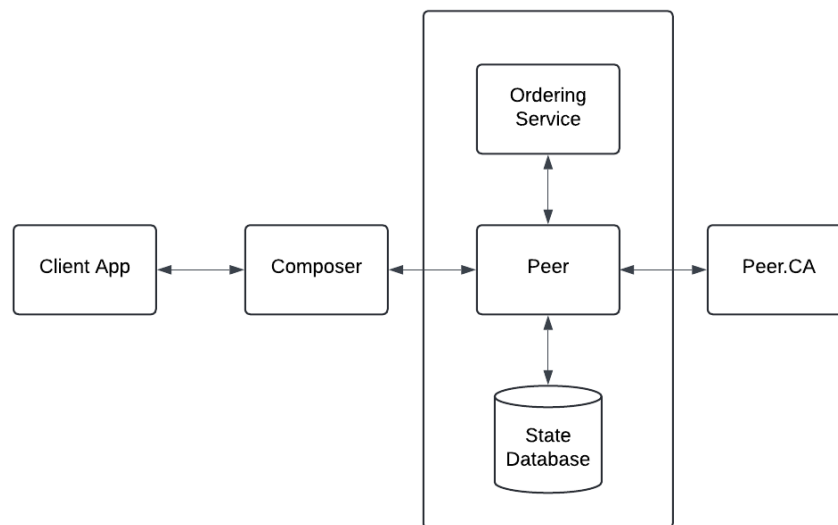
BAB II

TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Pustaka

2.1.1 Pengembangan Sistem *Blockchain* dari Aplikasi MeetCoin Sebelumnya

Pada awalnya, Prastyo (2022) [25] mengembangkan aplikasi MeetCoin untuk memberikan opsi pembayaran dari aplikasi MeetingYuk yang memiliki masalah dalam sistem pembayarannya, yaitu mahal biaya administrasi untuk menggunakan sistem pembayaran pihak ketiga [25]. Aplikasi MeetCoin yang dikembangkan merupakan aplikasi *android* dompet digital berbasis *blockchain*. Lebih spesifik, sistem yang dikembangkan meliputi proses bisnis dasar yang terdapat pada aplikasi dompet digital lain, yaitu autentikasi akun, pengisian saldo, transfer antar pengguna, pembayaran, dan riwayat transaksi pengguna. Teknologi *blockchain* dimanfaatkan untuk dapat menjaga integritas riwayat pencatatan transaksi dengan lebih baik dibandingkan dengan teknologi konvensional.



Gambar 2.1. Arsitektur Sistem *Blockchain* Aplikasi MeetCoin Sebelumnya

Pada penelitian tersebut, penulis mengembangkan aplikasi MeetCoin menggunakan kerangka kerja Hyperledger Fabric versi 1.2.1 dan alat Hyperledger Composer untuk bagian *blockchain*, bahasa pemrograman Javascript (Node.js) untuk bagian API, dan bahasa pemrograman Kotlin untuk aplikasi *mobile* [25]. Jaringan Hyperledger Fabric yang dikembangkan terdiri dari satu *peer node* dan *ordering service*. *State database* yang digunakan untuk menyimpan data aset *blockchain* adalah CouchDB. Mekanisme konsensus yang digunakan pada *ordering service* pada penelitian tersebut tidak disebutkan. Gambar 2.1 menunjukkan arsitektur sistem *blockchain* dari aplikasi MeetCoin

sebelumnya yang terdiri dari dua bagian utama, yaitu jaringan Hyperledger Fabric dan Composer. Aplikasi klien berkomunikasi dengan jaringan Hyperledger Fabric melalui API yang disediakan oleh Hyperledger Composer. Fungsi atau logika bisnis dari aplikasi MeetCoin disimpan dalam *smart contract* atau *chaincode* di dalam jaringan Hyperledger Fabric yang akan dieksekusi setelah menerima permintaan transaksi dari aplikasi klien. *Chaincode* pada penelitian tersebut dikembangkan menggunakan bahasa pemrograman Javascript (Node.js) yang didukung oleh Hyperledger Composer.

Pengujian yang dilakukan oleh penulis adalah *unit testing* API menggunakan alat pengujian Postman yang bertujuan untuk menguji keberhasilan proses bisnis atau fungsi dari aplikasi yang dikembangkan. Selain itu, penulis juga melakukan *android testing* menggunakan kerangka kerja Espresso untuk menguji antarmuka pengguna dari aplikasi *android*. Lebih spesifik dalam pengujian fungsionalitas, hasil pengujian menunjukkan bahwa mayoritas fungsi yang dikembangkan telah berjalan sesuai dengan fungsinya. Namun, pengujian untuk sistem *blockchain* hanya terbatas pada pengujian fungsionalitasnya saja, tidak terdapat pengujian lain seperti pengujian kinerja sehingga informasi mengenai kinerja dari sistem yang dikembangkan pada penelitian tersebut tidak diketahui.

Penelitian tersebut memiliki beberapa kekurangan dan keterbatasan seperti yang disebutkan oleh penulis. Arsitektur jaringan dari aplikasi MeetCoin masih terpusat dengan satu *peer node* yang menjaga buku besar sehingga sangat rentan terhadap serangan keamanan dan keandalan sistem. Aplikasi MeetCoin yang dikembangkan masih belum dapat menyelesaikan permasalahan *double spending* karena hanya memiliki satu *peer node* yang menjaga buku besar [25]. Mekanisme konsensus yang tidak disebutkan juga menjadi pertanyaan mengenai bagaimana konsensus dapat dicapai yang berpengaruh terhadap integritas data di dalam jaringan. Selain itu, terdapat penerapan algoritme yang terlalu kompleks sehingga menyebabkan terjadinya *infinite loop request* pada fungsi riwayat transaksi [25]. Versi kerangka kerja Hyperledger Fabric yang digunakan masih menggunakan versi 1.2.1 yang dirilis pada tahun 2018. Versi tersebut saat ini sudah memasuki fase *deprecated* sehingga tidak digunakan lagi dan akan dihapus dalam beberapa versi pembaruan Hyperledger Fabric mendatang [27]. Penggunaan teknologi yang sudah *deprecated* berisiko terhadap keamanan, kinerja, dan skalabilitas aplikasi.

2.1.2 Pengembangan Sistem Berbasis *Blockchain* Menggunakan Kerangka Kerja Hyperledger Fabric

Penelitian yang dilakukan oleh Srivastava et al. (2021) [31] mengembangkan sistem rantai pasokan makanan berbasis *blockchain* untuk mengatasi permasalahan yang sering terjadi pada rantai pasokan, yaitu transparansi dan keterlacakan barang, karena rantai pasokan bergantung kepada setiap proses yang saling berhubungan satu sama lain. Sistem *blockchain* yang dikembangkan pada penelitian tersebut menggunakan ke-

rangka kerja Hyperledger Fabric versi 2.2. Terdapat enam *peer node* organisasi yang tergabung dalam satu jaringan *blockchain* yang terdiri dari penyedia bahan baku, manufaktur/produksi, pemroses, distributor, ritel, dan konsumsi. Pada penelitian tersebut, Solo digunakan sebagai mekanisme konsensus pada *ordering service* dan CouchDB sebagai *state database* untuk menyimpan data aset. Bahasa pemrograman yang digunakan untuk mengembangkan *chaincode* pada penelitian tersebut tidak disebutkan. Penggunaan konsensus Solo menjadi kekurangan pada penelitian tersebut karena Solo merupakan konsensus yang sudah *deprecated* mulai pada Hyperledger Fabric versi 2 dan seterusnya. Pengujian sistem dilakukan menggunakan alat Hyperledger Caliper pada tipe transaksi atau fungsi *Open* dan *Query*. Metrik pengujian yang digunakan adalah *throughput* atau jumlah transaksi yang dapat proses dalam satu detik (TPS), *latency*, dan skalabilitas jaringan terhadap jumlah *peer node*. Untuk mengestimasi *throughput*, parameter yang digunakan yaitu jumlah transaksi yang bervariasi dari 50 hingga 1000000 transaksi. Pengujian *throughput* menunjukkan hasil *throughput* bervariasi dari 707 TPS hingga 955 TPS dengan jumlah transaksi mulai dari 50 hingga 1000000 transaksi dan *latency* rata-rata untuk jumlah transaksi tertinggi kurang dari 150 milidetik. Hasil tersebut menunjukkan bahwa semakin banyak transaksi yang dikirim, *throughput* akan meningkat, tetapi *latency* akan bertambah. Pada pengujian skalabilitas, parameter *peer node* ditetapkan sebanyak 2, 4, dan 6 *peer node* dengan mempertahankan jumlah transaksi yang bervariasi dari 50 hingga 1000000 transaksi. Hasil pengujian skalabilitas menunjukkan sistem dengan 2 *peer node* memiliki *throughput* tertinggi sebesar 907 TPS dan rata-rata *latency* terendah 67,3 milidetik, dibandingkan dengan sistem dengan 4 (*throughput* 873 TPS, rata-rata *latency* 114,3 milidetik) dan 6 (*throughput* 858 TPS, rata-rata *latency* 124,5 milidetik) *peer node*.

Vidwans et al. (2022) [32] melakukan penelitian yang mengembangkan sistem pemungutan suara berbasis *blockchain* untuk mengatasi berbagai keterbatasan yang terkait dengan metode pemungutan suara yang umum digunakan, seperti pemungutan suara ganda, manipulasi suara, identifikasi dan autentikasi pemilih, peretasan EVM (*electronic voting machines*), dan hilangnya suara karena kerusakan EVM. Kerangka kerja *blockchain* yang digunakan untuk mengembangkan sistem pemungutan suara adalah Hyperledger Fabric versi 2.2.3. Pada penelitian tersebut, didefinisikan dua *peer node* organisasi dan *ordering service*. Mekanisme konsensus yang digunakan pada *ordering service* tidak disebutkan. *State database* yang digunakan untuk menyimpan data aset juga tidak disebutkan. *Chaincode* pada penelitian tersebut dikembangkan menggunakan bahasa pemrograman Javascript (Node.js). Sistem tersebut selanjutnya diuji menggunakan alat Hyperledger Caliper dengan metrik pengujian penggunaan sumber daya dan *latency*. Parameter yang digunakan pada pengujian: jumlah transaksi yang dikirim sebanyak 20, 60, dan 100 transaksi per detik; durasi pengujian dibagi menjadi 10, 30, 60, 120, dan 240 detik. Pengujian juga menguji dua algoritme *chaincode* yang berbeda, yaitu versi *basic* yang tidak

mengenkripsi data transaksi dan versi *encryption* yang mengenkripsi data transaksi untuk meningkatkan privasi dan kerahasiaan data. Hasil pengujian tersebut menunjukkan bahwa penggunaan CPU meningkat seiring dengan meningkatnya jumlah transaksi yang dikirimkan. Namun, *latency* rata-rata berkurang ketika jumlah transaksi yang dikirimkan meningkat. *Chaincode* versi *encryption* juga menunjukkan penggunaan sumber daya CPU yang lebih tinggi dibandingkan dengan *chaincode* versi *basic*. Pada pengujian tersebut, metrik *throughput* dan tingkat keberhasilan transaksi tidak digunakan sehingga kinerja sistem tidak diketahui.

Al-Sumaidae et al. (2023) [33] mengembangkan sistem pencatatan rekam medis berbasis *blockchain* menggunakan kerangka kerja Hyperledger Fabric yang bertujuan untuk meningkatkan interoperabilitas dari beberapa sistem pencatatan rekam medis agar dapat berkomunikasi dan terintegrasi satu sama lain. Versi kerangka kerja Hyperledger Fabric yang digunakan adalah versi terbaru yang rilis pada tahun 2023 ketika penelitian tersebut sedang dilakukan. Pada penelitian tersebut, jaringan *blockchain* terdiri dari dua *peer node* organisasi yang merepresentasikan institusi medis. Mekanisme konsensus yang digunakan pada *ordering service* adalah Raft. *State database* yang digunakan untuk menyimpan data aset adalah LevelDB. Bahasa pemrograman yang digunakan untuk mengembangkan *chaincode* pada penelitian tersebut adalah Javascript (Node.js). Sistem tersebut selanjutnya diuji menggunakan alat Hyperledger Caliper pada fungsi *CreateRecord* dan *ReadRecord*. Indikator atau metrik yang digunakan dalam pengujian tersebut yaitu *throughput*, *latency*, dan tingkat keberhasilan transaksi. Parameter pengujian yang digunakan: jumlah *worker*/klien sebanyak 5, 25, dan 50 *worker*; transaksi yang dikirimkan berjumlah 1000, 5000, dan 15000 transaksi; laju transaksi yang dikirim berjumlah 75, 150, dan 250 transaksi per detik. Pengujian dilakukan menggunakan dua *controller* Hyperledger Caliper, yaitu *fixed-rate* dan *linear-rate controller*. Hasil pengujian menggunakan *fixed-rate controller* menunjukkan bahwa semakin banyak *worker* meningkatkan kemungkinan *blockchain* akan kelebihan beban dan transaksi cenderung gagal ketika laju transaksi meningkat yang mengakibatkan *throughput* menurun. Pada *linear-rate controller*, hasil pengujian menunjukkan bahwa jumlah *worker* memiliki dampak positif pada *throughput* dan peningkatan laju transaksi berdampak positif pada *latency*, *throughput*, dan tingkat keberhasilan transaksi.

Berdasarkan penelitian-penelitian terdahulu yang telah dijelaskan di atas, banyak bidang atau sektor yang telah memanfaatkan teknologi *blockchain* dengan menggunakan kerangka kerja Hyperledger Fabric. Hal tersebut juga menunjukkan bahwa Hyperledger Fabric memiliki fleksibilitas dan skalabilitas yang tinggi sehingga dapat diaplikasikan dalam berbagai kasus penggunaan. Tabel 2.1 merangkum penjelasan penelitian-penelitian terdahulu mengenai pengembangan sistem berbasis *blockchain* menggunakan kerangka kerja Hyperledger Fabric serta menyertakan rencana pengembangan yang akan dilakuk-

an pada tugas akhir ini. Secara khusus, pada pengembangan sistem *blockchain* MeetCoin sebelumnya, kerangka kerja Hyperledger Fabric dan alat Hyperledger Composer yang digunakan sudah memasuki fase *deprecated* (tidak digunakan lagi) dan akan dihapus pada beberapa pembaruan Hyperledger Fabric mendatang. Selain itu, rancangan serta implementasi sistem *blockchain* MeetCoin sebelumnya belum mempertimbangkan aspek keandalan, keamanan, dan kinerja. Oleh sebab itu, tugas akhir ini bertujuan untuk merancang dan mengembangkan ulang sistem *blockchain* MeetCoin menggunakan kerangka kerja Hyperledger Fabric dengan mempertimbangkan aspek keandalan, keamanan, dan kinerja serta mengevaluasi kinerja sistem menggunakan Hyperledger Caliper agar dapat dianalisis untuk mengidentifikasi kemampuan sistem dan dapat dijadikan sebagai evaluasi untuk pengembangan sistem *blockchain* MeetCoin di masa depan.

Tabel 2.1. Perbandingan Penelitian Terdahulu dengan Tugas Akhir

Penulis	Kontribusi Penelitian	Implementasi Sistem	Evaluasi Sistem
Prastyo (2022)	Mengembangkan aplikasi dompet digital MeetCoin berbasis <i>blockchain</i>	• Menggunakan kerangka kerja Hyperledger Fabric versi 1.2.1 dan Hyperledger Composer dengan konfigurasi <i>peer node</i> tunggal • Mekanisme konsensus tidak disebutkan • <i>State database</i> CouchDB • Bahasa pemrograman Javascript (Node.js)	Pengujian fungsional API menggunakan Postman

Srivastava et al. (2021)	Mengembangkan sistem rantai pasokan makanan berbasis <i>blockchain</i>	• Menggunakan kerangka kerja Hyperledger Fabric versi 2.2 dengan konfigurasi enam <i>peer node</i> • Mekanisme konsensus Solo • <i>State database</i> CouchDB • Bahasa pemrograman tidak disebutkan	Pengujian kinerja dan skalabilitas menggunakan Hyperledger Caliper
Vidwans et al. (2022)	Mengembangkan sistem pemungutan suara berbasis <i>blockchain</i>	• Menggunakan kerangka kerja Hyperledger Fabric versi 2.2.3 dengan konfigurasi dua <i>peer node</i> • Mekanisme konsensus tidak disebutkan • <i>State database</i> tidak disebutkan • Bahasa pemrograman Javascript (Node.js)	Pengujian sumber daya dan <i>latency</i> menggunakan Hyperledger Caliper

Al-Sumaidae et al. (2023)	Mengembangkan sistem pencatatan rekam medis berbasis <i>blockchain</i>	• Menggunakan kerangka kerja Hyperledger Fabric versi terbaru yang rilis pada tahun 2023 dengan konfigurasi dua <i>peer node</i> • Mekanisme konsensus Raft • <i>State database</i> LevelDB • Bahasa pemrograman Javascript (Node.js)	Pengujian kinerja menggunakan Hyperledger Caliper
Tugas Akhir	Merancang dan mengembangkan ulang sistem <i>blockchain</i> dari aplikasi MeetCoin sebelumnya dengan mempertimbangkan aspek keandalan, keamanan, dan kinerja	• Menggunakan kerangka kerja Hyperledger Fabric versi 2.5.9 dengan konfigurasi dua <i>peer node</i> • Mekanisme konsensus Raft • <i>State database</i> LevelDB • Bahasa pemrograman Go	Pengujian kinerja menggunakan Hyperledger Caliper

2.2 Dasar Teori

2.2.1 *Blockchain*

Blockchain adalah teknologi buku besar terdistribusi (*distributed ledger technology* atau DLT) yang mencatat dan menyimpan data di dalam blok yang saling terhubung membentuk rantai dengan memanfaatkan jaringan *peer-to-peer* dan teknik kriptografi.

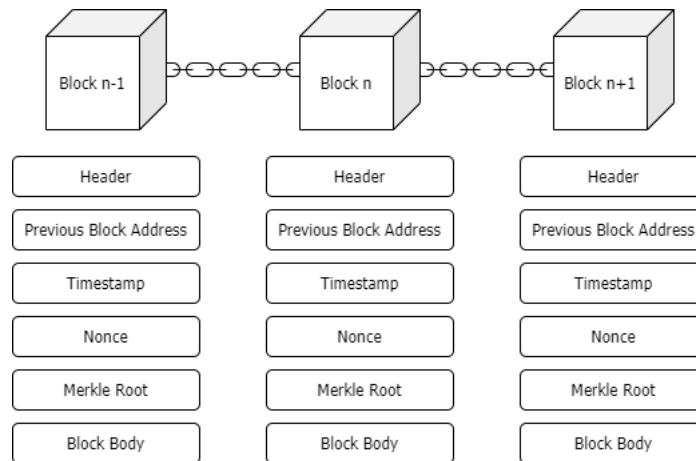
grafi [18, 34]. Dikatakan terdistribusi karena buku besar, atau dapat disebut dengan basis data, *blockchain* dibagikan kepada setiap pihak, atau *node*, yang terhubung secara *peer-to-peer* di dalam jaringan *blockchain*. Hal tersebut menjadikan setiap pihak memiliki salinan dari buku besar tersebut dan mengeliminasi permasalahan *single point of failure* yang dapat terjadi pada jaringan atau sistem dengan arsitektur *client-server* [35]. Selain itu, setiap salinan buku besar saling tersinkronisasi sehingga setiap pihak memiliki salinan yang valid dan terbaru.

Blockchain memiliki empat komponen inti, yaitu *hash*, *digital signature*, jaringan *peer-to-peer*, dan mekanisme konsensus [35]. Keempat komponen inti yang saling terkait membentuk *blockchain* menjadi teknologi yang memiliki tingkat keandalan dan keamanan yang tinggi. Fungsi *hash* dimanfaatkan untuk menjaga integritas data pada setiap blok yang ada di dalam *blockchain*. Perubahan yang terjadi pada data transaksi akan mengubah nilai *hash* sehingga perubahan data yang tidak diinginkan dapat terdeteksi dengan mudah. Selain itu, setiap blok juga memiliki *hash* dari blok sebelumnya untuk memastikan bahwa *hash* tersebut tidak dapat diubah tanpa mengubah blok sebelumnya. Jaringan *peer-to-peer* yang digunakan dalam *blockchain* membentuk sifat desentralisasi sehingga tidak terdapat satu otoritas pusat yang mengatur jaringan. Meskipun tidak adanya otoritas pusat, kepercayaan dan integritas di dalam jaringan *blockchain* dapat diperoleh dan dijamin melalui adanya algoritme konsensus [20].

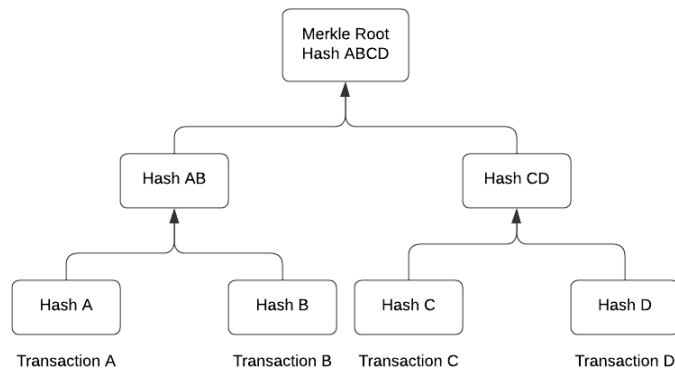
2.2.1.1 Struktur *Blockchain*

Blok pada *blockchain* merupakan tempat untuk menampung data berupa data transaksi, *timestamp*, dan fungsi kriptografi [34]. Sebuah blok terdiri dari *header* blok yang berisikan *metadata*, informasi dari sebuah data, yang meliputi *hash* blok sebelumnya, *timestamp* yang mengacu pada waktu pembentukan blok, *nonce*, dan Merkle *root* [17]. Selain itu, sebuah blok juga terdiri dari daftar transaksi yang terjadi di dalam *blockchain*. Daftar transaksi mengandung detail tentang transaksi yang meliputi alamat penerima, alamat pengirim, jumlah aset yang ditransfer, *timestamp*, dan tanda tangan digital. Gambar 2.2 menunjukkan struktur sederhana dari suatu blok.

Setiap transaksi yang ada di dalam blok akan diberi fungsi *hash*. *Hash* value dari setiap transaksi selanjutnya akan dikumpulkan secara kolektif membentuk Merkle *root* yang disimpan di dalam *header* blok [36]. Ilustrasi dari Merkle *tree* yang menghasilkan Merkle *root* ditunjukkan pada Gambar 2.3. *Nonce* (*number only used once*) adalah angka acak yang akan dicari oleh *miner node* untuk dapat menemukan *hash* yang sesuai dengan tingkat kesulitan untuk menambahkan blok pada suatu jaringan *blockchain* [36]. *Miner* yang berhasil menemukan *nonce* dan *hash* dari blok yang baru akan mendapatkan hadiah berupa *cryptocurrency*. Blok baru yang berhasil dipecahkan oleh *miner* akan ditambahkan ke dalam *blockchain*.



Gambar 2.2. Struktur Blok pada *Blockchain*



Gambar 2.3. Merkle *Tree*

2.2.1.2 Algoritme Konsensus

Dalam jaringan *blockchain* yang bersifat terdesentralisasi, diperlukan seperangkat aturan untuk menentukan cara bagi pengguna untuk mencapai kesepakatan pada suatu keadaan tertentu. Seperangkat aturan tersebut didefinisikan dalam sebuah algoritme konsensus. Dengan adanya algoritme konsensus, integritas jaringan *blockchain* dapat terjaga dari manipulasi buku besar atau serangan terhadap jaringan *blockchain* itu sendiri. Algoritme konsensus memiliki banyak jenis berdasarkan pada kebutuhan jaringan *blockchain*. Berikut adalah beberapa jenis algoritme konsensus yang umum digunakan.

1. *Proof-of-Work*

Proof-of-Work (PoW) merupakan algoritme konsensus yang digunakan di dalam jaringan Bitcoin. PoW memerlukan daya komputasi untuk berkompetisi dalam menambang blok. Orang atau komputer yang menambang blok disebut sebagai *miner*. Blok tersebut ditambang dengan cara memecahkan teka-teki matematika yang tingkat kesulitannya akan bertambah secara dinamis seiring banyaknya jum-

lah partisipan atau daya komputasi yang tergabung dalam jaringan *blockchain* [37]. Penambang yang berhasil memecahkan teka-teki matematika akan mendapatkan hadiah berupa *cryptocurrency* dan blok yang ditambang akan ditambahkan ke dalam *blockchain*. Pada PoW, untuk mencapai konsensus dibutuhkan kesepakatan oleh 51% *node* dari total *node* yang tergabung dalam jaringan yang membuat jaringan menjadi lebih aman dan efektif [20]. Namun, PoW memiliki kekurangan, yaitu membutuhkan konsumsi energi listrik yang tinggi yang menyebabkan pemborosan daya [38].

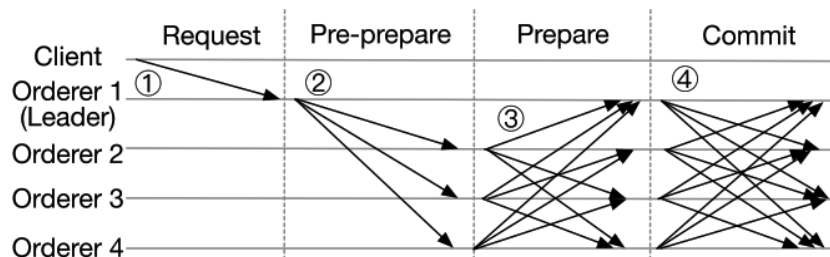
2. *Proof-of-Stake*

Proof-of-Stake (PoS) tidak memerlukan daya komputasi yang besar seperti PoW [20]. PoS merupakan algoritme konsensus yang digunakan di dalam jaringan Ethereum. Berbeda dengan PoW yang mengharuskan setiap *miner* berlomba untuk menambang blok, PoS akan memilih *node* secara acak untuk memvalidasi setiap transaksi dan menambahkannya ke dalam blok. *Node* yang bertugas untuk memvalidasi transaksi disebut sebagai *validator*. *Node* akan mengumpulkan sejumlah koin yang miliki ke dalam *stake* untuk dapat menjadi *validator*. Semakin besar koin yang dikumpulkan dalam *stake*, *node* tersebut memiliki peluang yang lebih besar untuk menjadi *validator* [20]. Setelah selesai memvalidasi transaksi dan menambahkannya ke dalam blok, blok tersebut akan divalidasi terlebih dahulu oleh *validator* lain. Jika tidak ada penolakan dari *validator* lain, blok akan dianggap valid dan ditambahkan ke dalam *blockchain*. Setelah berhasil, *validator* akan mendapatkan hadiah berupa biaya transaksi dari blok tersebut.

3. *Practical Byzantine Fault Tolerance*

Practical Byzantine Fault Tolerance (PBFT) adalah algoritme konsensus yang dirancang untuk mengatasi permasalahan yang terjadi di dalam sistem terdistribusi yang memungkinkan beberapa *node* mengalami kesalahan atau berperilaku jahat seperti memalsukan informasi [39]. PBFT umumnya digunakan di dalam jaringan *blockchain* berizin seperti Hyperledger Fabric dan Hyperledger Sawtooth. PBFT dapat menoleransi kesalahan hingga $\frac{1}{3}$ dari jumlah *node* dan hanya membutuhkan $\frac{2}{3}$ kesepakatan dari jumlah *node* untuk mencapai konsensus [37]. Struktur PBFT terdiri dari *leader* dan *follower node*. Pemilihan *leader* dan *follower* dirotasi menggunakan algoritme Round-robin setiap *leader* sebelumnya dicurigai melakukan kesalahan oleh *follower*. PBFT memiliki empat tahapan, yaitu *request*, *pre-prepare*, *prepare*, dan *commit* [6]. Alur dari keempat tahapan PBFT ditunjukkan pada Gambar 2.4 dengan *node* 0 sebagai *leader* dan lainnya sebagai *follower*. Tahapan *request* yaitu ketika *leader* menerima permintaan transaksi dari klien. *Leader* akan mengumpulkan transaksi-transaksi tersebut dan membuat blok baru dari transaksi yang diterima. Selanjutnya pada tahapan *pre-prepare*, *leader* akan mem-

bagikan blok tersebut kepada seluruh *follower* untuk diverifikasi, mulai dari isi transaksi hingga urutan terjadinya transaksi. Pada tahapan *prepare*, setelah setiap *follower* memvalidasi blok, blok tersebut akan diberi tanda atau pesan bahwa blok tersebut valid dan akan membagikannya ke *node* yang lain. *Node* yang menerima $2n$ (n adalah jumlah *node* dalam jaringan) pesan *prepare* yang valid akan mengirimkan pesan *commit* ke *node* yang lain. Pada tahapan *commit*, setiap *node* yang menerima pesan *commit* sejumlah $2n + 1$ (pesan *node* lain dan miliknya sendiri) akan mengirimkan pesan *commit* ke klien yang menunjukkan bahwa blok tersebut disetujui untuk ditambahkan ke dalam *blockchain*.



Gambar 2.4. Alur Tahapan PBFT [6]

4. Raft

Raft merupakan algoritme konsensus berbasis *crash fault-tolerant* (CFT) yang digunakan di dalam jaringan Hyperledger Fabric. Mirip seperti PBFT, Raft dapat menoleransi kesalahan yang terjadi pada beberapa *node*, tetapi tidak didesain untuk menangani *node* yang mungkin dapat berperilaku jahat. Rentang kesalahan yang dapat ditoleransi oleh Raft adalah $(n - 1)/2$, dengan n adalah jumlah *node*, yang berarti Raft dapat mencapai konsensus ketika lebih dari setengah *node* berjalan normal [6]. Sebagai contoh, ketika terdapat 5 *node*, Raft dapat menoleransi hingga 2 *node* yang salah. Raft juga memiliki struktur *leader* dan *follower*, namun dengan tambahan *candidate*. Pada Raft, hanya *leader* yang dapat berkomunikasi dengan klien. Apabila klien mengirim permintaan transaksi pada *follower*, *follower* akan mengirimkan permintaan tersebut ke *leader*. Hanya *leader* juga yang dapat membuat blok baru dari transaksi-transaksi yang masuk. Setelah blok baru dibuat, *leader* akan mengirimkan salinan dari blok ke seluruh *follower*. *Follower* akan mengirimkan *acknowledgement* kepada *leader* bahwa telah menerima salinan blok. Jika *leader* belum menerima semua *acknowledgement* dari *follower*, *leader* akan mengirimkan ulang blok tersebut hingga semua *follower* mengirimkan *acknowledgement*. Ketika semua *node* memiliki salinan dari blok, semua *node* (*leader* dan *follower*) akan mengirimkan blok tersebut ke *validator node*.

Pada Raft, ketika *leader* mengalami kegagalan, permintaan transaksi oleh klien tidak dapat diterima dan pembuatan blok akan terhenti sehingga Raft akan mencari

leader baru. *Leader* akan mengirimkan sinyal berulang kepada *follower*. Ketika sinyal tersebut berhenti, *leader* akan dianggap gagal dan pemilihan *leader* baru dilakukan. Kemudian, secara acak, *follower* akan berubah menjadi *candidate* dan mengirimkan sinyal ke *node* lain. *Node* lain akan membalas sinyal tersebut. *Candidate* yang memiliki jumlah respon sinyal terbanyak akan terpilih menjadi *leader* baru. Ketika *leader* baru terpilih, permintaan transaksi dan pembuatan blok akan dilanjutkan.

2.2.1.3 Tipe *Blockchain*

Blockchain memiliki beberapa tipe yang dapat dikategorikan berdasarkan struktur dan pengaturannya, seperti publik, privat, atau campuran keduanya. Beberapa tipe lain dari *blockchain* yaitu konsorsium, tanpa izin, dan berizin. Berikut adalah penjelasan mengenai masing-masing tipe *blockchain*.

1. *Blockchain* Publik

Blockchain publik adalah jaringan *blockchain* yang terbuka untuk semua orang tanpa memerlukan izin. Siapapun yang terhubung dengan internet dapat bergabung ke dalam jaringan *blockchain* publik untuk berpartisipasi menjadi *node*. Setiap *node* di dalam *blockchain* publik memiliki hak yang sama untuk melakukan validasi dan menambang blok. Setiap *node* juga dapat melihat riwayat transaksi yang pernah terjadi termasuk siapa yang melakukan transaksi sehingga memiliki tingkat transparansi yang tinggi. Contoh dari *blockchain* publik adalah Bitcoin dan Ethereum.

2. *Blockchain* Privat

Blockchain privat adalah jaringan *blockchain* yang membutuhkan izin khusus untuk dapat bergabung di dalamnya. Partisipan di dalam *blockchain* privat memiliki identitas yang diketahui oleh partisipan lain. *Blockchain* privat umumnya dikendalikan oleh satu entitas tertentu dan memiliki skala yang lebih kecil dibandingkan dengan *blockchain* publik. Berbeda dengan *blockchain* publik yang memiliki transparansi terhadap data di dalamnya, *blockchain* privat memiliki privasi yang memungkinkan data hanya dapat diakses oleh pihak tertentu yang memiliki izin. Data dan transaksi yang sepenuhnya dapat dikontrol menjadikan *blockchain* privat cocok digunakan untuk kasus penggunaan perusahaan atau industri. Contoh dari *blockchain* privat adalah Hyperledger Fabric dan Quorum.

3. *Blockchain* Hibrida

Blockchain hibrida merupakan kombinasi atau gabungan dari *blockchain* publik dan *blockchain* privat. *Blockchain* hibrida dikelola oleh sebuah entitas pusat, dengan penerapan *blockchain* privat berizin di samping *blockchain* publik [40]. *Blockchain* hibrida memiliki fleksibilitas untuk mengontrol siapa saja yang dapat mengakses data tertentu yang tersimpan di dalam *blockchain* dan data apa saja yang di-

buka untuk publik. Contoh dari *blockchain* hibrida adalah Hyperledger Besu yang mengombinasikan jaringan privat berizin dengan jaringan Ethereum.

4. *Blockchain* Konsorsium

Blockchain konsorsium adalah *blockchain* yang diatur dan dikelola oleh lebih dari satu entitas. Beberapa entitas atau organisasi yang tergabung saling berkolaborasi untuk mengelola jaringan *blockchain*. Pada *blockchain* konsorsium, setiap organisasi memiliki akses terbatas sesuai dengan perannya masing-masing di dalam jaringan. Mekanisme konsensus di dalam *blockchain* konsorsium dikendalikan oleh *node* yang sudah ditentukan sebelumnya yang disebut sebagai *validator node* untuk menginisiasi, menerima, dan memproses transaksi [40]. Contoh dari *blockchain* konsorsium adalah R3 Corda dan Hyperledger Fabric.

5. *Blockchain* Tanpa Izin

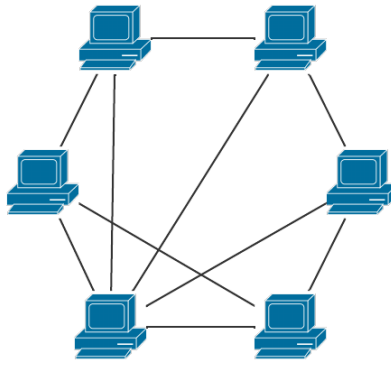
Blockchain tanpa izin umumnya dihubungkan dengan *blockchain* publik karena siapa saja dapat bergabung dan berpartisipasi di dalamnya. Semua *node* pada *blockchain* tanpa izin memiliki level yang setara. Jaringan *blockchain* tanpa izin juga sepenuhnya terdesentralisasi yang diatur melalui algoritme konsensus. Semua data dan transaksi dalam *blockchain* tanpa izin transparan untuk semua partisipan dan identitas setiap partisipan bersifat anonim.

6. *Blockchain* Berizin

Blockchain berizin memiliki batasan jumlah pengguna yang dapat berpartisipasi dalam transaksi dan memiliki tugas serta tanggung jawab yang terbatas untuk masing-masing pengguna. Hanya pengguna atau *node* yang telah diberi otoritas yang dapat bergabung ke dalam jaringan *blockchain* berizin. Umumnya, *blockchain* berizin terdapat pada *blockchain* privat. Namun, terdapat juga *blockchain* berizin yang memperbolehkan siapa saja untuk bergabung dan dapat melihat isi buku besar, tetapi hanya *node* tertentu yang diperbolehkan untuk melakukan perubahan pada buku besar.

2.2.2 Jaringan *Peer-to-Peer*

Jaringan *Peer-to-Peer* (P2P) adalah suatu topologi jaringan komputer yang terbentuk dari masing-masing *peer* atau komputer yang saling terhubung secara langsung satu sama lain sehingga memungkinkan komunikasi langsung antara tiap *peer* [41]. Ilustrasi dari jaringan P2P ditunjukkan pada Gambar 2.5. Dalam jaringan P2P, *peer* dapat diidentifikasi sebagai *node*. *Node* yang terhubung dalam jaringan P2P memiliki tugas dan tanggung jawab yang sama untuk berbagi sumber daya serta mempertahankan jaringan. Semua *node* terhubung satu sama lain sehingga tidak memerlukan otoritas atau *server* pusat dan memiliki sifat terdesentralisasi [42].



Gambar 2.5. Jaringan *Peer-to-Peer*

Jaringan P2P memiliki kelebihan dalam hal keandalan. Jika satu *node* mengalami masalah atau terputus dari jaringan, *node* lain tetap dapat beroperasi dan melanjutkan fungsi jaringan. Jaringan P2P juga memiliki kelebihan dalam hal skalabilitas. *Node* baru dapat bergabung dengan jaringan tanpa mengubah infrastruktur yang sudah ada. Selain itu, bertambahnya *node* yang bergabung dalam jaringan dapat meningkatkan sumber daya dan kemampuan jaringan. Sifat desentralisasi dalam jaringan P2P juga memungkinkan pemrosesan dan penyimpanan terdistribusi sehingga jaringan dapat menangani proses dengan beban yang lebih tinggi tanpa bergantung pada *server* terpusat yang dapat menyebabkan *bottleneck*.

2.2.3 Hyperledger Fabric

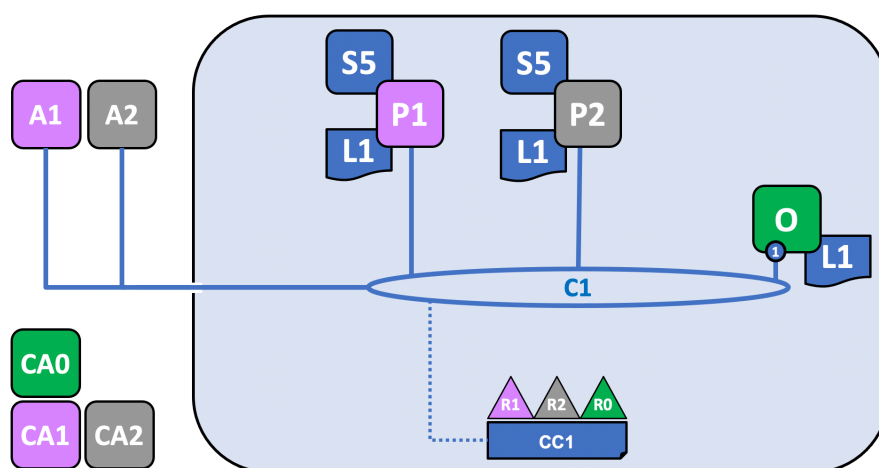
Hyperledger Fabric, atau Fabric, adalah platform *blockchain* atau *Distributed Ledger Technology* (DLT) *open-source* yang dikembangkan oleh proyek Hyperledger di bawah naungan Linux Foundation. Fabric memperkenalkan arsitektur *blockchain* yang berfokus pada ketahanan, fleksibilitas, skalabilitas, dan kerahasiaan [15]. Fabric dikembangkan untuk penggunaan *blockchain* berskala perusahaan yang memiliki arsitektur modular dan dapat dikonfigurasi sesuai dengan berbagai kebutuhan bisnis atau industri. Fabric merupakan platform *blockchain* privat berizin sehingga setiap partisipan di dalam jaringan memiliki identitas yang dapat diidentifikasi dengan sah serta memiliki tugas dan tanggung jawab sesuai dengan peranannya. Fabric juga mendukung pengembangan *blockchain* konsorsium yang memungkinkan beberapa organisasi dapat saling tergabung dan berkolaborasi dalam satu jaringan.

Fabric merupakan platform DLT pertama yang mendukung penulisan *smart contract*, atau *chaincode* dalam jaringan Fabric, menggunakan bahasa pemrograman *general-purpose* seperti Java, Go, dan Node.js [43]. Dengan adanya dukungan tersebut, pengembang tidak perlu mempelajari bahasa baru yang dikhususkan untuk menulis *smart contract* seperti pada Ethereum yang menggunakan bahasa Solidity. Fabric juga mendukung

berbagai penggunaan mekanisme konsensus, seperti konsensus berbasis CFT dan BFT, yang dapat dibongkar pasang sehingga dapat disesuaikan dengan berbagai kasus penggunaan dan model kepercayaan. Selain itu, Fabric tidak memerlukan mata uang kripto untuk menjalankan jaringan sehingga dapat menghemat biaya operasional.

2.2.3.1 Arsitektur Hyperledger Fabric

Hyperledger Fabric memperkenalkan arsitektur baru dalam memproses transaksi yang disebut dengan *execute-order-validate* [15]. Pada sistem *blockchain* lainnya seperti Bitcoin dan Ethereum, pemrosesan transaksi dilakukan dengan mengikuti arsitektur *order-execute* yang mengurutkan transaksi terlebih dahulu menggunakan mekanisme konsensus, kemudian mengeksekusi transaksi pada semua *peer* secara berurutan. Arsitektur *execute-order-validate* memungkinkan Hyperledger Fabric dapat mengeksekusi transaksi secara konkuren yang dapat meningkatkan kinerja sistem [15]. Berbeda dengan arsitektur pemrosesan transaksi pada sistem *blockchain* lain, Fabric memerlukan arsitektur sistem baru yang ditunjukkan pada Gambar 2.6. Pada Gambar 2.6, jaringan Fabric terdiri dari *peer* (P) organisasi (R) dan *ordering service* (O) yang tergabung dalam *channel* (C) yang diatur oleh konfigurasi *channel* (CC) berdasarkan kesepakatan antara organisasi yang membentuk jaringan. Semua *node* dalam jaringan Fabric memiliki salinan buku besar (L), tetapi *ordering service* hanya memiliki salinan *blockchain* tanpa *world state*. Setiap *peer* juga memiliki *chaincode* (S) yang terpasang untuk dapat memiliki akses ke buku besar (L). Klien atau organisasi dapat berinteraksi dengan *chaincode* (S) melalui aplikasi (A) yang terhubung dengan *channel* (C). Setiap organisasi juga memiliki *Certificate Authority* (CA) yang mengeluarkan sertifikat digital sebagai identitas *node* dalam jaringan.



Gambar 2.6. Arsitektur Hyperledger Fabric [13]

Secara sederhana, aplikasi terdistribusi Fabric terdiri dari dua bagian, yaitu *chaincode* dan kebijakan pengesahan transaksi (*endorsement policy*). *Chaincode* adalah kode

program yang mendefinisikan logika bisnis dan transaksi dari aplikasi. *Chaincode* akan dijalankan ketika klien mengirim permintaan transaksi ke aplikasi Fabric. *Chaincode* juga memiliki akses langsung ke buku besar sehingga dapat melakukan perubahan dan pencatatan pada buku besar. *Chaincode* akan dipasang pada setiap *peer* setelah *chaincode* tersebut disetujui dan di-*deploy* ke jaringan Fabric. Kebijakan pengesahan transaksi merupakan kebijakan yang dirancang oleh administrator jaringan untuk menentukan siapa saja dan berapa *peer* yang dibutuhkan untuk mengesahkan transaksi atau yang disebut dengan *endorsing peer* (*endorser*). Transaksi dari klien perlu diproses oleh *chaincode* dan disahkan oleh *peer* sesuai dengan kebijakan pengesahan untuk dapat ditambahkan ke dalam buku besar.

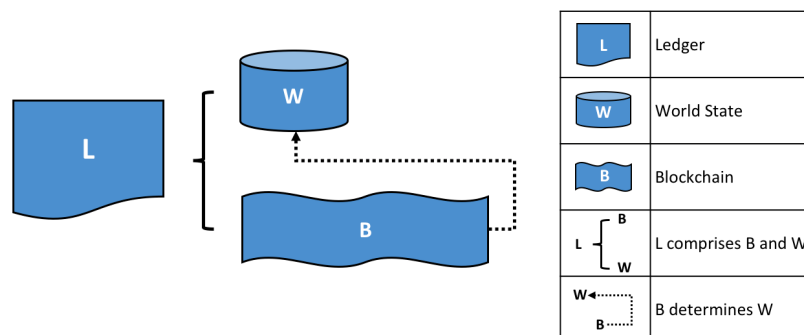
Dalam membentuk jaringan *blockchain*, Fabric terdiri dari sekumpulan *node* yang saling terhubung secara *peer-to-peer* dan tergabung pada suatu *channel*. *Channel* adalah subset privat dari jaringan utama yang terdiri dari dua atau lebih anggota jaringan tertentu untuk meningkatkan privasi dan kerahasiaan transaksi. Karena Fabric merupakan *blockchain* berizin, setiap *node* yang tergabung dalam jaringan memiliki identitas yang ditetapkan oleh *Membership Service Provider* (MSP). Identitas tersebut dibuat dan dikeluarkan dalam bentuk sertifikat digital dengan format X.509 oleh suatu otoritas bernama *Certificate Authority* (CA). Identitas yang dimiliki oleh setiap *node* harus dapat dibuktikan dan diverifikasi bahwa identitas tersebut dikeluarkan oleh CA yang ada di dalam jaringan. *Node* dalam Fabric memiliki tiga identitas atau peran yang masing-masing memiliki tugas dan tanggung jawab yang berbeda:

- *Client* bertanggung jawab untuk mengirimkan proposal transaksi kepada *peer* yang ditugaskan untuk mengesahkan transaksi dan mengirimkan proposal transaksi yang telah disahkan ke *orderer*.
- *Peer* menerima proposal transaksi dari *client*, mensimulasikan dan mengesahkan proposal transaksi, dan mengirimkan kembali proposal yang telah disahkan dan ditandatangani kepada *client*. *Peer* juga memvalidasi transaksi yang telah diurutkan oleh *orderer* sebelum melakukan *commit* transaksi ke buku besar. Namun, tidak semua *peer* bertugas untuk memproses proposal transaksi yang dikirimkan oleh *client*, hanya *peer* yang sudah ditentukan dalam kebijakan pengesahan yang dapat memproses proposal transaksi. Setiap *peer*, baik *endorser* maupun bukan, memiliki dan menjaga salinan buku besar.
- *Orderer* atau *Ordering Service Node* (OSN) merupakan sekumpulan *node* yang bertugas untuk mengurutkan semua transaksi yang sudah disahkan oleh *endorser*. Urutan transaksi yang telah diurutkan *orderer* bersifat final sehingga dapat menjaga konsistensi data di dalam jaringan. *Orderer* memiliki

tugas yang terpisah dari *endorser*. Mekanisme pengurutan transaksi pada Fabric menggunakan protokol konsensus yang bersifat modular dan dapat diganti dengan protokol konsensus lain. Protokol konsensus dalam Fabric dapat diimplementasikan dengan berbagai cara, mulai dari *orderer* tunggal Solo (saat ini sudah *deprecated*) hingga konsensus berdasarkan toleransi kesalahan CFT atau BFT.

2.2.3.2 Hyperledger Fabric Ledger

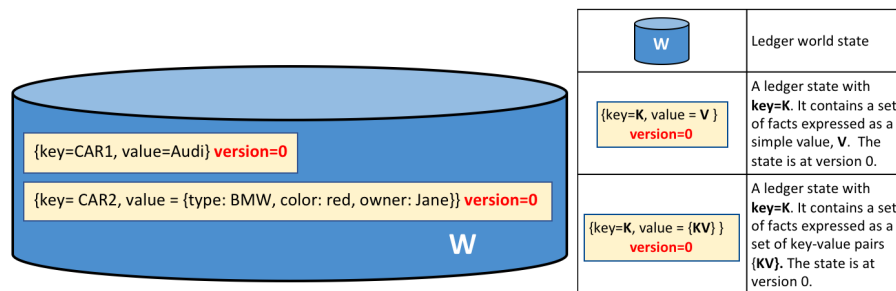
Ledger atau buku besar merupakan tempat penyimpanan data di dalam jaringan *blockchain*. Buku besar menyimpan fakta dari suatu objek bisnis pada kondisi saat ini berdasarkan riwayat transaksi yang menyebabkan objek bisnis tersebut ada pada kondisi saat ini. Meskipun fakta tentang objek bisnis saat ini dapat berubah, riwayat mengenai fakta yang berhubungan dengan objek bisnis saat ini tidak dapat diubah dan memiliki sifat *immutable*. Teknologi DLT atau *blockchain* sering dikaitkan dengan buku besar yang secara logis hanya tunggal, tetapi sebenarnya memiliki banyak salinan yang konsisten yang didistribusikan di seluruh jaringan. Pada Fabric, buku besar memiliki dua komponen utama yang saling berhubungan, yaitu *world state* dan *transaction log* atau *blockchain*. Komponen buku besar dari Fabric ditunjukkan pada Gambar 2.7.



Gambar 2.7. Komponen Buku Besar Hyperledger Fabric [14]

World state adalah buku besar yang menyimpan nilai suatu objek bisnis atau *state* aset pada saat ini. Adanya *world state* akan mempermudah aplikasi untuk mengakses *state* aset pada saat ini, dibandingkan aplikasi harus menelusuri seluruh riwayat transaksi untuk mendapatkan *state* aset pada saat ini. *State* aset pada *world state* dapat berubah karena aset dapat dibuat, ditambahkan, diperbarui, dan dihapus. *World state* menyimpan aset dalam bentuk pasangan *key-value*. Pada Gambar 2.8, setiap aset memiliki nomor versi yang dimulai dari versi 0. Nomor versi akan selalu bertambah setiap kali *state* dari aset berubah. Versi juga akan dicek sebelum perubahan pada *world state* untuk memastikan bahwa perubahan yang terjadi sesuai dengan yang diinginkan. *World state*

diimplementasikan dalam bentuk basis data sehingga memungkinkan untuk melakukan berbagai macam *query* pada *world state*.

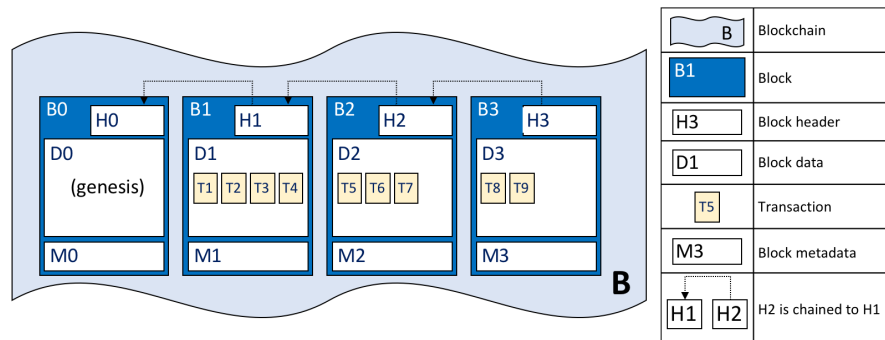


Gambar 2.8. Struktur *World State* [14]

Berbeda dengan *world state* yang menyimpan *state* aset yang dapat berubah, *blockchain* menyimpan semua riwayat transaksi yang terjadi pada seluruh aset yang ada pada *world state* hingga *state* aset pada saat ini. *Blockchain* mencatat setiap versi sebelum dan sesudah perubahan yang terjadi pada *state* aset. *Blockchain* memiliki struktur data yang berbeda dengan *world state*, di mana data di *world state* masih dapat dilakukan perubahan. Setelah data transaksi ditulis di *blockchain*, data transaksi tidak dapat dimodifikasi sehingga menjadikannya *immutable* yang membuat *blockchain* dapat menjaga integritas dan konsistensi data. *Blockchain* memiliki struktur yang terdiri dari blok-blok yang saling terikat dan di blok tersebut terdapat urutan transaksi. Setiap transaksi merepresentasikan *query* atau perubahan yang terjadi pada *world state*. Struktur *blockchain* pada Fabric ditunjukkan pada Gambar 2.9. Setiap blok memiliki *header* yang berisi *hash* transaksi pada blok tersebut dan *hash* dari *header* blok sebelumnya. Seluruh transaksi di dalam *blockchain* dihubungkan secara kriptografis yang membuat data pada buku besar sangat aman karena untuk merusak data pada satu *node* memerlukan komputasi yang sangat besar untuk memecahkan *hash* kriptografis pada seluruh transaksi dan juga perlu meyakinkan semua *node* lain bahwa data yang dirusak merupakan data yang valid. Blok pertama dalam *blockchain* disebut dengan blok genesis yang berisi konfigurasi transaksi yang mengandung *state* awal jaringan. *Blockchain* diimplementasikan dalam suatu file yang dapat dikonversi menjadi format JavaScript Object Notation (JSON) karena struktur data pada *blockchain* sangat bias terhadap operasi yang sangat sederhana serta jarang dilakukannya *query* pada *blockchain* sehingga operasi penambahan blok di akhir *blockchain* masih menjadi operasi utama [14].

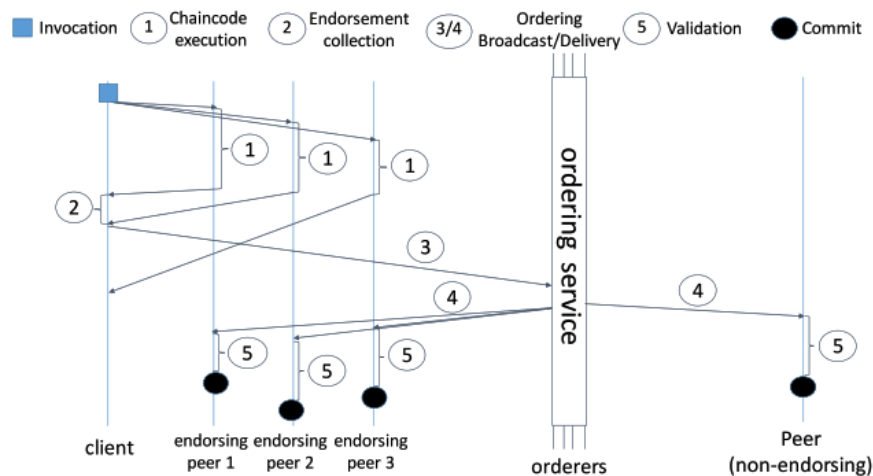
2.2.3.3 Alur Transaksi Hyperledger Fabric

Dalam memproses transaksi, Fabric menggunakan arsitektur *execute-order-validate* yang terdiri dari tiga tahapan, yaitu *execution*, *ordering*, dan *validation*. Alur transaksi



Gambar 2.9. Struktur *Blockchain* [14]

Fabric ditunjukkan pada Gambar 2.10. Berikut adalah penjelasan mengenai setiap tahapan pemrosesan transaksi yang dilakukan oleh Fabric.



Gambar 2.10. Alur Transaksi Hyperledger Fabric [15]

- Tahapan *execution*

Pada tahapan *execution*, klien akan mengirimkan proposal transaksi yang telah ditandatangani ke *endorsing peer* yang telah ditentukan pada kebijakan pengesahan untuk disimulasikan atau dieksekusi. Proposal transaksi yang dikirimkan berisi identitas klien yang sesuai dengan MSP, *payload* transaksi berupa operasi pada *chaincode* yang akan dieksekusi, parameter, identitas *chaincode*, *nonce*, dan identitas transaksi berupa kombinasi dari identitas klien dan *nonce* [15]. Proposal transaksi akan dieksekusi oleh masing-masing *endorsing peer* secara lokal sesuai dengan operasi *chaincode* yang terdapat pada *payload* proposal transaksi. Hasil eksekusi oleh *endorsing peer* tidak langsung di-commit ke buku besar. *Endorsing peer* akan membuat *read-write set*, yaitu keadaan *state* aset sebelum (*read*) dan sesudah (*write*) eksekusi pro-

posal transaksi.

Setelah mengeksekusi proposal transaksi dan membuat *read-write set*, *endorsing peer* akan menandatangani sebuah pesan yang disebut dengan pengesahan (*endorsement*). Pesan tersebut berisi *read-write set* dan *metadata* seperti identitas transaksi, identitas *endorser*, dan tanda tangan *endorser* [15]. Setelah pengesahan, *endorsing peer* akan mengirimkan kembali pesan pengesahan proposal kepada klien. Klien akan mengumpulkan pesan pengesahan sesuai dengan kebijakan pengesahan yang telah ditentukan. Semua pesan pengesahan yang diterima harus memiliki isi berupa *read-write set* yang identik dari semua *endorsing peer*. Setelah semua pesan pengesahan diterima, klien akan membuat transaksi dan mengirimkannya ke *ordering service*.

- Tahapan *ordering*

Ketika klien telah mengumpulkan pesan pengesahan yang dibutuhkan, klien akan membuat transaksi yang dikirimkan ke *ordering service*. Transaksi tersebut berisi operasi pada *chaincode* yang akan dijalankan, parameter, *metadata* transaksi, dan satu set pesan pengesahan [15]. Setelah transaksi diterima, *ordering service* akan membuat urutan dari transaksi, kemudian menyiarkan atau membagikan transaksi tersebut ke *ordering service* lain untuk menetapkan konsensus. Urutan transaksi yang dihasilkan dari konsensus bersifat final untuk menjaga konsistensi data. Selanjutnya, *ordering service* akan memasukkan urutan transaksi ke dalam blok dan menandatangani blok tersebut.

- Tahapan *validation*

Blok yang sudah ditandatangani oleh *ordering service* akan dikirimkan ke setiap *peer*, baik *endorser* maupun bukan, untuk divalidasi. *Peer* akan memeriksa dua hal sebelum blok dinyatakan valid dan di-*commit* ke buku besar. Pertama, semua transaksi di dalam blok akan dievaluasi secara paralel berdasarkan kebijakan pengesahan yang ditentukan. Apabila tidak memenuhi kebijakan pengesahan, transaksi dianggap tidak valid dan perubahan atas *state* aset diabaikan. Kedua, setiap transaksi akan dicek versi pada *world state* dan versi pada *read set*. Jika versinya sama, maka transaksi dianggap valid. Sebaliknya, jika versinya berbeda, transaksi dianggap tidak valid dan perubahan atas *state* aset diabaikan. Setelah kedua pemeriksaan tersebut selesai dilakukan, setiap *peer* akan melakukan *commit* ke buku besar yang disimpan di lokal *peer*. Transaksi yang valid akan mengubah *state* aset pada buku besar berdasarkan *write set*, sedangkan transaksi yang tidak valid efek perubahannya terhadap buku besar diabaikan.

2.2.4 Hyperledger Caliper

Hyperledger Caliper adalah alat yang digunakan melakukan *benchmarking* atau pengujian *blockchain*. Caliper dirancang untuk mengevaluasi kinerja dari sistem *blockchain*, khususnya *blockchain* yang dikembangkan menggunakan platform Hyperledger. Pada versi terbaru dari Caliper, yaitu versi 0.6.0, platform *blockchain* yang didukung dan kompatibel dengan Caliper yaitu Hyperledger Fabric, Hyperledger Besu, dan Ethereum. Caliper mendukung beberapa metrik pengujian, yaitu *success rate*, *transaction & read latency*, dan *transaction & read throughput*. Caliper juga dapat melakukan pengukuran penggunaan sumber daya, seperti daya CPU dan memori, ketika *blockchain* sedang menjalankan suatu proses atau siklus.

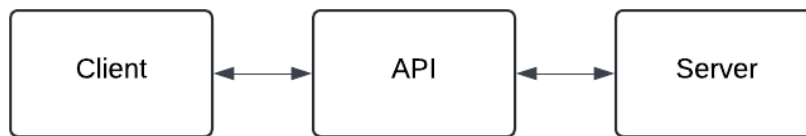
Caliper memiliki fleksibilitas tinggi yang dapat menguji berbagai konfigurasi jaringan dan parameter sistem. Pengembang dapat melakukan simulasi yang mencerminkan kondisi pada dunia nyata, seperti jumlah pengguna, jenis transaksi yang dilakukan, dan juga dapat merancang skenario beban kerja. Hasil pengujian dapat digunakan sebagai evaluasi untuk mengetahui bagaimana perubahan pada arsitektur atau pengaturan sistem dapat memengaruhi kinerja secara keseluruhan. Sebagai contoh, dalam pengujian yang dilakukan pada jaringan Hyperledger Fabric, Caliper dapat menunjukkan perbedaan signifikan pada *throughput* dan *latency* berdasarkan konfigurasi jumlah transaksi yang disimpan dalam satu blok. Selain itu, Caliper juga dapat mengidentifikasi *bottleneck* dalam sistem serta dapat digunakan untuk mengoptimalkan kinerja berdasarkan hasil pengujian.

2.2.5 Application Programming Interface

Application Programming Interface (API) adalah antarmuka aplikasi yang memungkinkan berbagai aplikasi dapat berkomunikasi satu sama lain [44]. API menyediakan berbagai aturan dan protokol untuk komunikasi antar aplikasi dengan cara yang terstandarisasi. API memungkinkan aplikasi dapat saling bertukar data dan fungsionalitas satu sama lain. API juga dapat digunakan sebagai sarana integrasi dari beberapa aplikasi yang memiliki fungsi berbeda seperti dalam arsitektur *microservices*.

API memiliki cara kerja yang melibatkan klien dengan *server* melalui komunikasi *request-response*. Cara kerja dari API ditunjukkan pada Gambar 2.11. Klien adalah aplikasi yang berinteraksi dengan pengguna. *Server* adalah aplikasi yang mengelola logika bisnis dan basis data. Klien akan mengirim *request* ke *server* melalui internet yang akan diterima oleh API. Selanjutnya API akan memproses *request* seperti fungsi serta data yang diminta oleh klien. *Server* akan menjalankan fungsi tersebut dan mengirimkan keluaran fungsi sebagai *response* yang dikirimkan balik ke klien melalui API.

Terdapat beberapa jenis API yang dirancang berdasarkan ketersediaannya, yaitu API privat, API partner, dan API publik. API privat umumnya dikembangkan secara



Gambar 2.11. Cara Kerja API

internal dalam suatu organisasi untuk mendukung solusi dan layanan organisasi serta sebagai sarana integrasi sistem atau aplikasi internal organisasi. Organisasi juga memiliki kontrol penuh atas penggunaan API. API partner merupakan API yang dirancang oleh dua atau lebih organisasi yang saling bermitra. Perancangan API partner bertujuan untuk saling berkolaborasi menyediakan akses ke data atau fungsionalitas tertentu yang tidak dimiliki oleh salah satu organisasi. API publik dirancang untuk dapat diakses oleh siapa pun yang membutuhkan fungsionalitasnya. API publik juga seringkali disebut dengan *third-party* API. Tujuan dari API publik adalah untuk mempermudah akses ke fungsionalitas atau data tertentu tanpa harus mengembangkannya dari awal. Salah satu contoh API publik yang banyak digunakan adalah Google Maps API.

2.2.6 Bahasa Pemrograman Go

Go, atau yang disebut dengan Golang, adalah bahasa pemrograman *open source* yang dirancang, dikembangkan, dan dikelola oleh Google [45]. Golang dirancang untuk menciptakan bahasa yang lebih sederhana dan efisien dibandingkan dengan bahasa pemrograman sebelumnya seperti C dan C++. Sintaks pada Golang sederhana sehingga dapat dipahami dan ditulis dengan mudah oleh pengembang untuk mengembangkan sistem. Golang juga memiliki *learning curve* yang landai dan dapat dipelajari dengan cepat oleh siapapun.

Golang mendukung pemrograman konkuren, yaitu teknik pemrograman yang memungkinkan eksekusi beberapa tugas secara bersamaan. Dengan adanya dukungan tersebut, Golang dapat memanfaatkan sumber daya CPU dengan baik sehingga meningkatkan kinerja aplikasi ketika memproses permintaan dalam jumlah yang besar serta mendukung skalabilitas sistem. Proses kompilasi pada Golang juga cepat, kode akan kompilasi menjadi bahasa mesin yang dapat dijalankan dengan cepat oleh komputer. Selain itu, Golang juga memiliki fitur *garbage collector* yang mampu mengelola penggunaan memori untuk mengurangi risiko terjadinya *memory leak*.

Dalam penggunaannya, Golang banyak digunakan untuk mengembangkan berbagai macam sistem karena sifatnya sebagai *general-purpose language*. Beberapa contoh penggunaan Golang yaitu pada pengembangan aplikasi web, sistem *cloud*, dan layanan *backend*. Golang juga memiliki dukungan komunitas yang aktif dan berkembang. Seca-

ra keseluruhan, Golang menawarkan kesederhanaan, kecepatan, dan kinerja yang tinggi sehingga menjadikannya pilihan yang menarik bagi pengembang untuk mengembangkan suatu sistem.

2.3 Analisis Perbandingan Metode

2.3.1 Perbandingan Hyperledger Fabric dengan Kerangka Kerja *Blockchain* Lain

Pada saat ini, banyak muncul platform atau kerangka kerja *blockchain* yang mendukung penggunaan di berbagai kasus penggunaan industri. Umumnya, dalam penggunaan industri, *blockchain* yang digunakan adalah *blockchain* privat berizin karena mendukung privasi dan kerahasiaan data. Selain itu, *blockchain* privat berizin juga mendukung penulisan *smart contract* yang mengatur logika bisnis aplikasi. Di antara banyak kerangka kerja *blockchain* privat berizin, terdapat tiga kerangka kerja yang dapat menjadi pilihan utama, yaitu Hyperledger Fabric, Hyperledger Besu, dan R3 Corda. Masing-masing kerangka kerja *blockchain* tersebut memiliki karakteristik yang berbeda satu sama lain yang dijelaskan pada Tabel 2.2.

Tabel 2.2. Perbandingan Hyperledger Fabric, Hyperledger Besu, dan R3 Corda [1–4]

Karakteristik	Hyperledger Fabric	Hyperledger Besu	R3 Corda
Tipe	Privat atau konsorsium berizin	Publik atau privat berbasis Ethereum	Privat atau konsorsium berizin
Lisensi	<i>Open-source</i>	<i>Open-source</i>	<i>Open-source</i> dengan fitur terbatas dan <i>enterprise</i> berbayar
<i>Governance</i>	Linux Foundation	Linux Foundation	R3
Tujuan	Berbagai kasus penggunaan industri	Berbagai kasus penggunaan industri	Industri layanan keuangan
Peran aplikasi	Berkoordinasi langsung dengan semua partisipan untuk mengesahkan transaksi dan mengelola penguncian konkurensi pada <i>state</i> , <i>signature</i> , dan <i>submission</i>	Mengirimkan transaksi yang telah ditandatangani ke satu <i>node</i> dalam jaringan untuk divalidasi	Berkoordinasi dengan pihak lawan transaksi untuk menegosiasikan pembaruan yang diusulkan, mendapatkan <i>signature</i> , dan menyelesaikan transaksi dengan layanan notaris

Alur transaksi	<i>Execute, Order, Validate</i>	<i>Order, Execute/Validate</i>	<i>Execute/Validate, Order/Notarize</i>
Bahasa pemrograman <i>smart contract</i>	Go, Java, Node.js	Solidity	Java, Kotlin
Mekanisme konsensus	Raft, PBFT	<i>Proof-of-Authority, Proof-of-Stake, Proof-of-Work</i>	Raft, BFT
<i>Currency</i>	Tidak menggunakan, tetapi dapat dibuat menggunakan <i>chaincode</i>	Ether	Tidak menggunakan <i>native cryptocurrency</i> , tetapi menggunakan Corda Coin

Berdasarkan Tabel 2.2, Hyperledger Fabric dan Besu memiliki lisensi *open-source* yang memungkinkan pengembang untuk memodifikasi kode sesuai dengan kebutuhan pengembangan. Meskipun R3 Corda juga memiliki lisensi *open-source*, terdapat beberapa limitasi fitur seperti keterbatasan penggunaan *node* tunggal yang rentan terhadap *single point of failure*, tidak adanya *caching* dinamis dan peningkatan kinerja, serta hanya mendukung proses *single thread* [46]. Limitasi pada R3 Corda dapat dihilangkan hanya dengan menggunakan versi *enterprise* yang berbayar. Selain itu, alur transaksi pada Hyperledger Fabric memungkinkan eksekusi transaksi secara konkuren yang berpengaruh terhadap kinerja sistem seperti *throughput* dalam pemrosesan transaksi [15]. Berbeda dengan Hyperledger Besu yang menggunakan alur transaksi *order-execute/validate*, pemrosesan transaksi masih dilakukan secara berurutan yang membatasi kinerja sistem dan dapat menyebabkan *bottleneck* ketika banyak transaksi yang masuk [15].

Hyperledger Fabric mendukung penulisan *smart contract* menggunakan bahasa pemrograman *general-purpose* Go, Java, dan Node.js yang banyak digunakan pada pengembangan aplikasi. Dukungan Hyperledger Fabric terhadap penggunaan tiga bahasa pemrograman membuat pengembang tidak perlu mempelajari bahasa khusus yang digunakan dalam domain tertentu (*domain-specific language* atau DSL) seperti Solidity pada Ethereum. Berbeda dengan Hyperledger Fabric, R3 Corda hanya mendukung penulisan *smart contract* menggunakan Java dan Kotlin yang berjalan pada Java Virtual Machine (JVM). Selain karakteristik yang disebutkan pada Tabel 2.2, Hyperledger Fabric memiliki arsitektur modular yang dapat disesuaikan dengan berbagai kasus penggunaan industri. Arsitektur modular tersebut meliputi penggunaan konsensus yang dapat dibongkar pasang atau *pluggable*, manajemen identitas dan kebijakan pengesahan transaksi yang juga *pluggable*, dan buku besar yang dapat dikonfigurasi untuk mendukung berbagai DBMS.

Berdasarkan uraian di atas, Hyperledger Fabric dipilih karena memiliki fleksibilitas tinggi dalam perancangan arsitektur serta memungkinkan proses konkurensi yang dapat meningkatkan kinerja sistem.

2.3.2 Pemilihan Algoritme Konsensus Hyperledger Fabric

Dalam mengembangkan sistem *blockchain* yang bersifat terdesentralisasi, dibutuhkan suatu mekanisme konsensus untuk mencapai kesepakatan pada suatu keadaan dan untuk menjaga integritas jaringan. Hyperledger Fabric mendukung penggunaan mekanisme konsensus yang dapat disesuaikan dengan kebutuhan dan persyaratan penggunaan. Selain itu, pemilihan mekanisme konsensus dapat berpengaruh terhadap karakteristik kinerja dan penggunaan sumber daya jaringan [6]. Pada Hyperledger Fabric, terdapat tiga mekanisme konsensus, yaitu Kafka, Raft, dan PBFT.

Tabel 2.3. Perbandingan Mekanisme Konsensus Kafka, Raft, dan PBFT [5–7]

Karakteristik	Kafka	Raft	PBFT
Perbedaan	• Berbasis Apache Kafka dan CFT • Menggunakan ZooKeeper untuk manajemen konsensus • Menggunakan arsitektur <i>producer</i>, <i>consumer</i>, dan <i>broker</i> • Replikasi data pada tingkat transaksi • Toleransi kesalahan <i>node</i> dapat ditentukan berdasarkan parameter yang dapat dikonfigurasi	• Berbasis CFT • Menggunakan etcd sebagai sistem manajemen konsensus • Menggunakan arsitektur <i>leader</i>, <i>follower</i>, dan <i>candidate</i> • Replikasi data pada tingkat blok • Raft dapat menoleransi kesalahan hingga kurang dari $1/2$ <i>node</i> yang melakukan kesalahan	• Berbasis BFT • Menggunakan arsitektur <i>leader</i> dan <i>follower</i> • Menggunakan konsep Byzantine yang merotasi <i>leader</i> secara Round-robin ketika mengalami kesalahan atau bertindak jahat • Replikasi data pada tingkat blok • Memerlukan minimal tiga <i>node</i> untuk beroperasi • PBFT dapat menoleransi kesalahan hingga $1/3$ dari jumlah <i>node</i> yang melakukan kesalahan atau bertindak jahat

Kelebihan	• Menghasilkan <i>throughput</i> yang tinggi • Dapat mempertahankan konsistensi dalam keadaan <i>network failure</i> atau <i>crash node</i> • Memiliki skalabilitas yang tinggi sehingga cocok untuk jaringan skala besar	• Menghasilkan <i>throughput</i> yang paling tinggi dibandingkan dengan Kafka dan PBFT • Memiliki cara kerja dan implementasi yang sederhana • Pemilihan <i>leader</i> yang dinamis dan cepat • Memiliki penggunaan sumber daya yang paling rendah dibandingkan dengan Kafka dan PBFT	• Memiliki efisiensi tinggi dan cepat dalam mencapai konsensus • Toleran terhadap <i>node</i> yang bertindak jahat dengan mengisolasi <i>node</i> tersebut dalam mencapai konsensus • Memiliki keamanan yang tinggi di jaringan yang tidak sepenuhnya dapat dipercaya
-----------	---	--	---

Kekurangan	• Memiliki kompleksitas yang tinggi pada arsitektur dan konfigurasi • Memiliki ketergantungan terhadap komponen eksternal ZooKeeper • Sudah <i>deprecated</i> dan tidak mendapat dukungan mulai Hyperledger Fabric versi 2	• Raft dapat mengalami penurunan performa dalam jaringan yang memiliki <i>latency</i> tinggi • Memiliki keterbatasan dalam toleransi kesalahan yang hanya diakibatkan oleh kegagalan <i>node</i> • Memiliki ketergantungan pada <i>leader</i>, jika <i>leader</i> gagal dalam menerima transaksi maka transaksi akan hilang	• Memerlukan jumlah <i>node</i> yang tetap dan diketahui dalam jaringan • Memiliki penggunaan sumber daya yang paling tinggi dibandingkan dengan Raft dan Kafka • Memiliki skalabilitas yang terbatas karena <i>overhead</i> komunikasi antar <i>node</i> yang tinggi sehingga tidak cocok untuk jaringan yang besar
------------	--	---	--

Pemilihan mekanisme konsensus didasarkan pada kebutuhan jaringan sistem. Pada tugas akhir ini, mekanisme konsensus yang dipilih adalah Raft. Pemilihan mekanisme konsensus Raft didasarkan pada implementasi dan cara kerja Raft yang sederhana serta memiliki penggunaan yang paling rendah. Hal tersebut membuat Raft memiliki kinerja yang paling baik di antara Kafka dan PBFT. Raft memiliki keterbatasan hanya dapat menoleransi *node* yang mengalami kegagalan, berbeda dengan PBFT yang juga dapat menangani *node* yang bertindak jahat seperti mengirimkan pesan yang salah. Meskipun Raft hanya menoleransi kesalahan akibat kegagalan *node*, di dalam jaringan privat berizin yang setiap identitas diketahui dan memiliki tata kelola ketat, risiko terhadap *node* yang bertindak jahat sangat rendah dibandingkan dengan jaringan publik tanpa izin.

2.3.3 Pemilihan State Database Hyperledger Fabric

Untuk menyimpan data aset pada *world state*, Hyperledger Fabric memberikan dua opsi basis data, yaitu LevelDB & CouchDB. LevelDB merupakan *state database* bawaan Hyperledger Fabric. LevelDB menyatu atau *embed* dengan setiap *peer* yang tergabung di dalam jaringan Fabric. Berbeda dengan LevelDB, CouchDB merupakan

alternatif pilihan *state database* eksternal yang terpisah dari *peer*. LevelDB menyimpan data dalam struktur pasangan *key-value*, sedangkan CouchDB menyimpan data dalam struktur objek dokumen seperti JSON. Kedua *state database* tersebut dapat melakukan operasi inti *chaincode* seperti membuat dan mengambil data aset serta melakukan *query* data.

Tabel 2.4. Perbandingan *State Database* LevelDB dan CouchDB [8–10]

Karakteristik	LevelDB	CouchDB
Kelebihan	• Implementasi yang sederhana dengan penggunaan struktur data pasangan <i>key-value</i> • Memiliki performa yang tinggi dalam melakukan <i>query</i> sederhana • Penggunaan sumber daya rendah karena bersifat <i>embed</i> pada tiap <i>peer</i> • Mendukung operasi baca/tulis <i>atomic</i> yang memastikan konsistensi data	• Dapat melakukan <i>query</i> data yang kompleks menggunakan struktur data JSON • Mendukung <i>indexing</i> untuk membuat relasi dan meningkatkan kinerja <i>query</i> data • Mendukung <i>off-chain database</i> yang mereplikasi data dari jaringan <i>blockchain</i> • Memiliki dan mendukung semua properti ACID (<i>Atomic, Consistent, Isolated, Durable</i>)

Kekurangan	• Hanya dapat melakukan <i>query key-value</i> sederhana seperti <i>Put</i>, <i>Get</i>, dan <i>Delete</i> • Tidak memiliki skema data terstruktur dan model data relasional • Tidak mendukung <i>query</i> kompleks dan <i>indexing</i> • Tidak dapat mereplikasi data dari jaringan <i>blockchain</i>	• Memerlukan penggunaan sumber daya yang lebih besar karena merupakan <i>state database</i> eksternal • Memerlukan waktu yang lama untuk melakukan <i>query</i> dibandingkan dengan LevelDB • Kinerja menurun lebih banyak dibanding LevelDB seiring bertambahnya data aset • Kompleks dalam implementasi dan konfigurasi
------------	--	--

Setiap *state database* memiliki kelebihan dan kekurangannya masing-masing seperti yang dijelaskan pada Tabel 2.4. Pada tugas akhir ini, *state database* yang dipilih adalah LevelDB. Penggunaan sumber daya yang lebih rendah dan implementasi struktur *key-value* sederhana membuat LevelDB memiliki kinerja komputasi yang lebih unggul dibandingkan dengan CouchDB. Penggunaan LevelDB juga tidak memerlukan konfigurasi tambahan karena merupakan *state database* bawaan yang bersifat *embed* dengan setiap *peer* di dalam jaringan. Sifat *embed* yang dimiliki oleh LevelDB membuat jaringan dan *peer* dapat diskalakan tanpa harus menambah penggunaan sumber daya yang besar seperti pada CouchDB yang merupakan *state database* eksternal. Meskipun hanya mendukung *query* sederhana seperti *Put*, *Get*, dan *Delete*, LevelDB cukup untuk melakukan *query* data di dalam jaringan karena semakin kompleks *query* data maka kinerjanya akan semakin menurun.

2.3.4 Perbandingan Hyperledger Caliper dengan Alat Pengujian Lain

Munculnya banyak platform atau kerangka kerja *blockchain* yang memiliki berbagai fitur dan fungsionalitas menyebabkan munculnya kebutuhan untuk melakukan evaluasi kinerja sistem *blockchain* yang dikembangkan. Oleh karenanya, muncul beberapa alat pengujian *blockchain*, di antaranya yaitu Hyperledger Caliper dan BlockBench. Kedua alat tersebut memiliki kesamaan untuk melakukan pengujian sistem *blockchain* privat. Perbandingan antara Hyperledger Fabric dan BlockBench dijelaskan pada Tabel 2.5.

Tabel 2.5. Perbandingan Hyperledger Caliper dan BlockBench [11, 12]

Karakteristik	Hyperledger Caliper	BlockBench
Tujuan	Mengukur kinerja <i>blockchain</i> dengan menggunakan skenario pengujian yang dapat ditentukan	Mengukur kinerja <i>blockchain</i> secara keseluruhan dengan menggunakan skenario pengujian mikro dan makro
Lisensi	<i>Open-source</i>	<i>Open-source</i>
Dukungan <i>blockchain</i>	Hyperledger Fabric, Hyperledger Besu, Ethereum	Ethereum, Parity, Hyperledger Fabric
Metrik evaluasi	<i>Success rate, transaction & read latency, transaction & read throughput, resource consumption</i>	<i>Throughput, latency, scalability, fault tolerance</i>
Kelebihan	Skenario pengujian dapat dibuat sesuai dengan kebutuhan, menyediakan contoh skenario sederhana, dan memiliki dokumentasi yang lengkap	Dapat mengukur kinerja <i>blockchain</i> dengan detail mulai dari lapisan individu seperti operasi baca/tulis hingga sistem secara keseluruhan
Kekurangan	Memiliki struktur yang kompleks dan kurang dapat mengukur kinerja dari lapisan individu <i>blockchain</i>	Skenario dan dokumentasi yang terbatas serta kompleksitas tinggi terkait teknis penggunaannya

Berdasarkan Tabel 2.5, Hyperledger Caliper dipilih karena memberikan fleksibilitas kepada pengguna dalam menentukan skenario pengujian yang akan dilakukan. Contoh skenario pengujian yang disediakan dapat membantu mempermudah penulisan skenario yang lebih relevan dengan sistem yang dikembangkan. Metrik evaluasi yang didukung Hyperledger Caliper juga lebih lengkap dan dapat merepresentasikan kinerja dari sistem *blockchain* yang sedang diuji. Meskipun Hyperledger Caliper memiliki struktur yang kompleks, adanya dokumentasi lengkap dapat membantu dan mempermudah pengguna untuk menggunakan Hyperledger Caliper dalam melakukan pengujian sistem *blockchain*.

BAB III

METODE PENELITIAN

3.1 Alat dan Bahan Tugas Akhir

3.1.1 Alat Tugas Akhir

Dalam melakukan tugas akhir, diperlukan peralatan baik berupa perangkat keras maupun perangkat lunak sebagai sarana pendukung untuk merancang dan membangun ulang sistem *blockchain* MeetCoin serta untuk melakukan evaluasi kinerja sistem. Berikut adalah peralatan yang digunakan pada tugas akhir ini.

1. Laptop dengan spesifikasi sistem operasi Windows 11 64-bit versi 23H2, *processor* AMD Ryzen 7 6800H (16 CPUs) @ 3,2 GHz, memori 16GB DDR5 4800MHz, grafis NVIDIA GeForce RTX 3060 Laptop (6GB), dan SSD 512GB.
2. WSL2 (Windows Subsystem for Linux) *Backend* dengan distribusi Linux Ubuntu 22.04 LTS.
3. Visual Studio Code versi 1.92.1.
4. Postman versi 11.7.0.
5. Docker versi 25.0.2 dan Docker Compose versi 2.24.3.
6. Bahasa pemrograman Go versi 1.22.5.
7. Hyperledger Fabric versi 2.5.9.
8. Hyperledger Caliper versi 0.6.0.

3.1.2 Bahan Tugas Akhir

Bahan yang digunakan pada tugas akhir ini adalah sebagai berikut.

1. Dokumentasi resmi Hyperledger Fabric yang berisi standar dan panduan mengenai perancangan dan pengembangan sistem *blockchain* menggunakan kerangka kerja Hyperledger Fabric.
2. Dokumentasi resmi Hyperledger Caliper yang berisi panduan untuk melakukan konfigurasi alat Hyperledger Caliper dan pengujian sistem *blockchain*.
3. *Repository* GitHub resmi Hyperledger Fabric yang menyediakan konfigurasi, jaringan pengujian, dan contoh pengembangan sistem *blockchain* menggunakan kerangka kerja Hyperledger Fabric.

3.2 Metode yang Digunakan

Metode yang digunakan pada tugas akhir ini meliputi metode perancangan, metode implementasi, dan metode pengujian. Berikut adalah penjelasan mengenai masing-masing metode yang digunakan.

3.2.1 Metode Perancangan

Pada tugas akhir ini, metode perancangan yang meliputi analisis kebutuhan sistem, perancangan *activity diagram*, dan perancangan arsitektur sistem *blockchain* disusun untuk membantu proses pengembangan sistem agar sesuai dengan rencana dan kebutuhan sistem.

3.2.1.1 Analisis Kebutuhan Sistem

Analisis kebutuhan sistem bertujuan untuk mengidentifikasi dan mendefinisikan kebutuhan sistem yang akan dikembangkan. Pada tahap ini, kebutuhan sistem dapat dibagi menjadi dua, yaitu kebutuhan fungsional dan non-fungsional. Kebutuhan fungsional merupakan kebutuhan sistem yang berhubungan dengan proses bisnis yang akan dikembangkan, sedangkan kebutuhan non-fungsional merupakan kebutuhan atau persyaratan sistem yang terkait dengan batasan dan standar dari sistem yang akan dikembangkan.

1. Kebutuhan Fungsional Sistem

Berdasarkan aplikasi MeetCoin sebelumnya, fungsionalitas sistem yang menjadi kebutuhan pengguna dalam menggunakan aplikasi MeetCoin adalah sebagai berikut.

- (a) Pengguna dapat mendaftarkan akun MeetCoin.
- (b) Pengguna dapat melakukan autentikasi akun miliknya.
- (c) Pengguna dapat melihat profil akun.
- (d) Pengguna dapat melakukan pengisian saldo miliknya.
- (e) Pengguna dapat mengecek saldo miliknya.
- (f) Pengguna dapat melakukan transfer saldo miliknya ke pengguna lain.
- (g) Pengguna dapat melakukan penarikan saldo ke akun rekening bank miliknya.
- (h) Pengguna dapat melihat riwayat transaksi miliknya.

2. Kebutuhan Non-Fungsional Sistem

Berdasarkan kebutuhan fungsional sistem dan dokumentasi resmi Hyperledger Fabric, hal-hal yang menjadi kebutuhan non-fungsional sistem adalah sebagai berikut.

- (a) Pengguna diidentifikasi berdasarkan *username* yang bersifat unik dan tidak dapat mendaftarkan akun dengan *username* yang sama.
- (b) *Password* pengguna dienkrpsi terlebih dahulu sebelum disimpan di dalam basis data untuk memastikan keamanan akun.
- (c) Terdapat salinan data transaksi *on-chain* pada basis data di luar jaringan untuk meningkatkan ketersediaan dan aksesibilitas data tanpa memengaruhi kinerja jaringan.
- (d) Fungsionalitas sistem dapat diakses melalui API sehingga tidak memerlukan perintah berbasis *command line interface* (CLI) untuk menjalankan proses bisnisnya.
- (e) Sistem *blockchain* terdiri dari dua *peer node* untuk mencegah kegagalan sistem secara keseluruhan yang diakibatkan oleh hanya adanya satu *peer node* yang berjalan pada sistem.
- (f) Kebijakan pengesahan transaksi harus disetujui oleh kedua *peer node* untuk menjaga integritas data transaksi dan mencegah salah satu *peer node* melakukan serta mengesahkan transaksi yang tidak sah.

3.2.1.2 Perancangan *Activity Diagram*

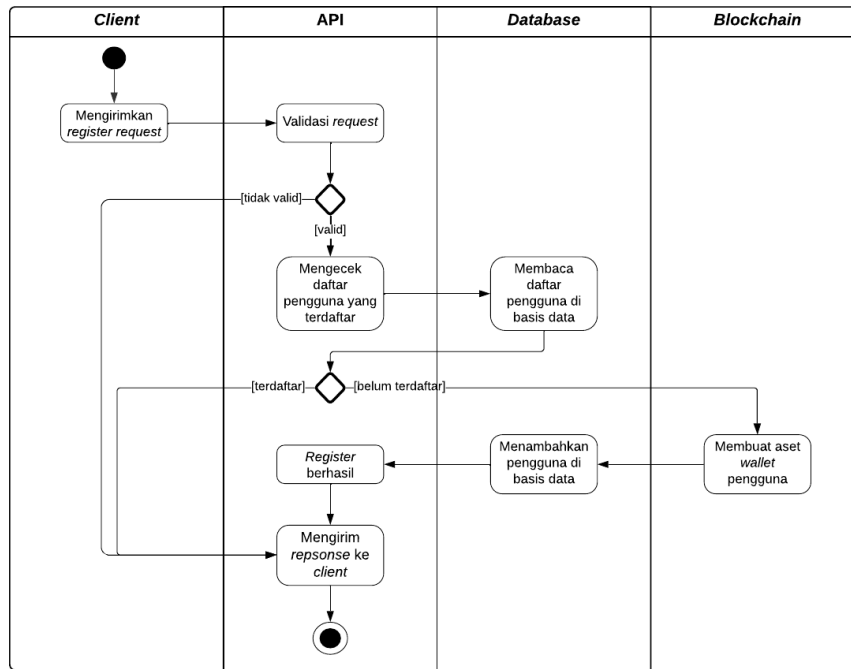
Setelah menganalisis kebutuhan sistem, dirancang *activity diagram* yang bertujuan untuk memodelkan proses bisnis dan alur kerja antara pengguna dengan sistem yang akan dikembangkan.

1. Pendaftaran Akun

Pada proses pendaftaran akun, pengguna mengirimkan *request* berupa nama, *e-mail*, *username*, nomor telepon, dan *password* ke API. *Request* tersebut kemudian akan diproses dan diteruskan ke basis data dan *blockchain* seperti yang ditunjukkan pada Gambar 3.1. Setelah selesai memproses *request* yang diterima, API akan mengirimkan *response* ke pengguna bahwa proses pendaftaran akun berhasil atau gagal.

2. Autentikasi Akun

Proses autentikasi akun juga dapat disebut sebagai proses *log in*. Pengguna yang ingin melakukan *log in* ke sistem mengirimkan *request* berupa *username* dan *password* yang telah didaftarkan sebelumnya. Setelah diterima oleh API, proses autentikasi akun pengguna akan diproses seperti yang ditunjukkan pada Gambar 3.2. API akan mengirim *response* berupa *token* pengguna setelah berhasil memproses *request* tersebut. *Token* tersebut berfungsi untuk mengautentikasi identitas pengguna ketika akan melakukan suatu proses di dalam sistem.



Gambar 3.1. Activity Diagram Proses Pendaftaran Akun

3. Melihat Profil Akun

Untuk melihat profil akun, pengguna harus *log in* ke dalam sistem terlebih dahulu. *Token* yang didapat setelah berhasil *log in* akan diautentikasi terlebih dahulu untuk memeriksa apakah pengguna tersebut valid untuk dapat melihat profil akunnya. Gambar 3.3 menunjukkan proses untuk melihat profil akun pengguna.

4. Pengisian Saldo

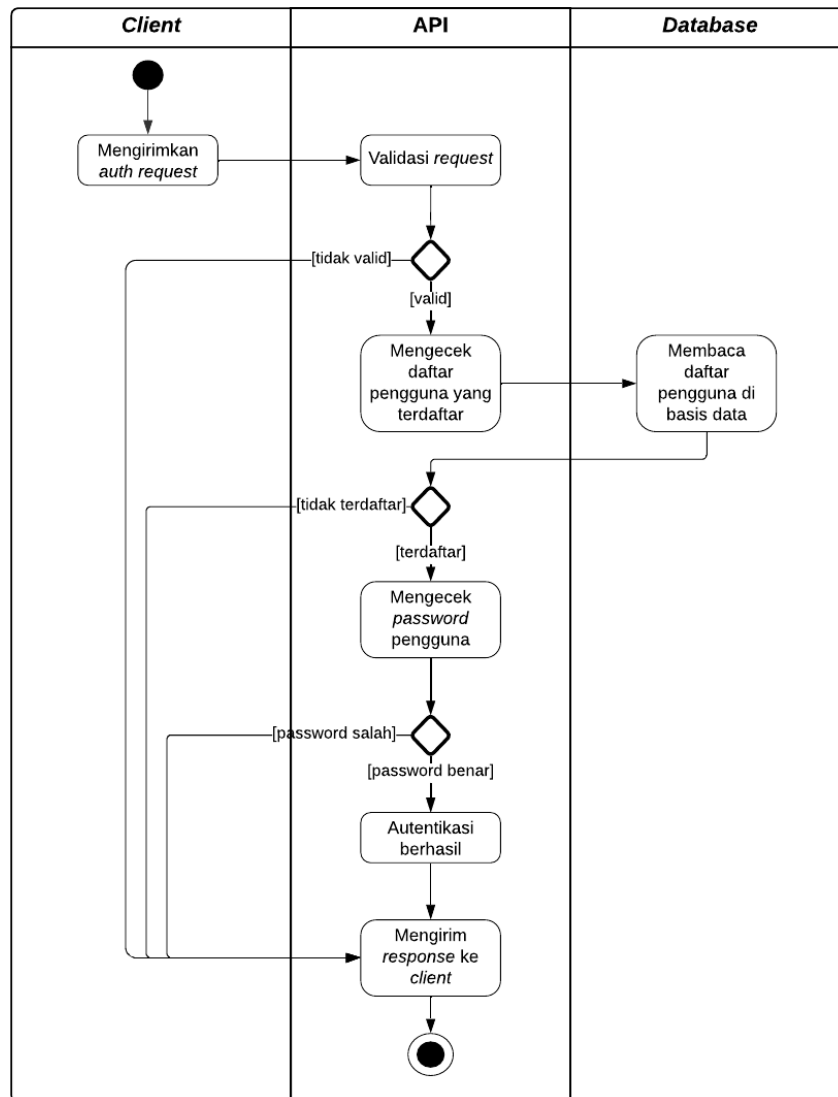
Pada proses pengisian atau *top up* saldo, pengguna dapat mengisi saldo akun melalui *request* yang dikirimkan ke API berupa jumlah koin yang diinginkan. *Request* tersebut diproses oleh API dan diteruskan ke *blockchain* dan basis data seperti yang ditunjukkan pada Gambar 3.4. Setelah proses pengisian saldo berhasil, saldo pengguna bertambah dan riwayat pengisian saldo akan dicatat di basis data.

5. Pengecekan Saldo

Proses pengecekan saldo pengguna mirip seperti proses melihat profil akun. API akan mengecek *token* yang dimiliki pengguna terlebih dahulu untuk memvalidasi *request* sebelum diproses lebih lanjut. Namun, berbeda dengan proses melihat profil akun yang membaca data pengguna di basis data, proses pengecekan saldo membaca data saldo pengguna dari *blockchain*. Gambar 3.5 menunjukkan proses pengecekan saldo pengguna.

6. Transfer Saldo

Pada proses transfer saldo, pengguna mengirimkan *request* berupa *username* tujuan dan jumlah koin yang ingin ditransferkan ke penerima. Sebelum melakukan proses

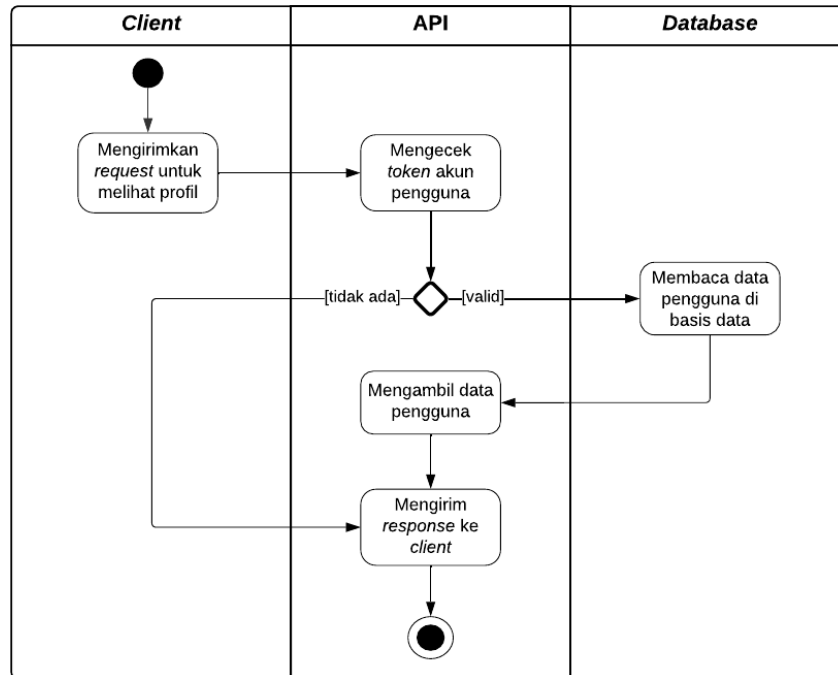


Gambar 3.2. Activity Diagram Proses Autentikasi Akun

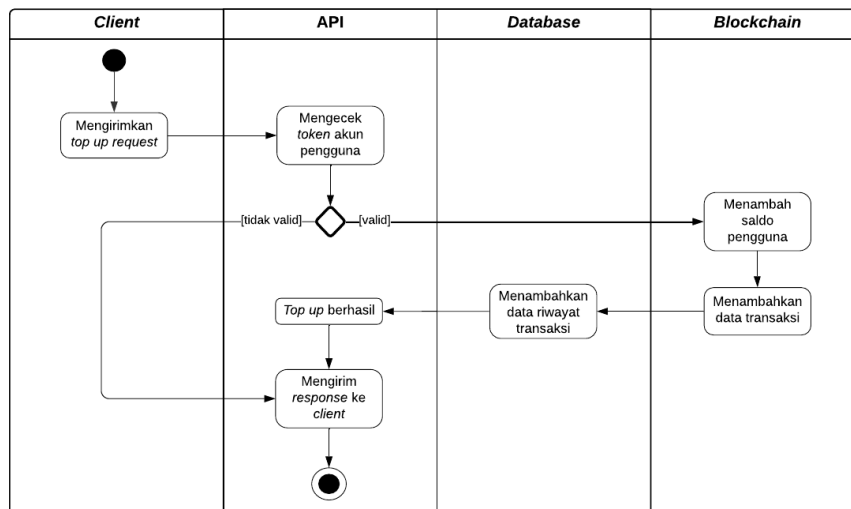
transfer, API akan memvalidasi *request* tersebut dengan mengecek saldo pengguna dan akun penerima seperti yang ditunjukkan pada Gambar 3.6. Apabila kedua hal tersebut valid, maka proses transaksi akan dilakukan. Transaksi yang berhasil akan memperbarui saldo dari kedua belah pihak serta menambahkan data transaksi di *blockchain*. Riwayat transaksi tersebut juga akan dicatat di basis data.

7. Penarikan Saldo

Proses penarikan atau *withdraw* saldo dilakukan dengan pengguna mengirimkan *request* yang berisi jumlah saldo yang ingin ditarik dan rekening bank tujuan. Sebelum melakukan penarikan saldo, jumlah saldo yang ingin ditarik akan dicek terlebih dahulu. Jika jumlah saldo yang ingin ditarik tidak melebihi atau sama dengan saldo yang dimiliki, proses penarikan saldo akan dilakukan. Proses penarikan saldo yang berhasil akan memperbarui saldo pengguna di *blockchain* serta mencatat



Gambar 3.3. Activity Diagram Proses Melihat Profil Akun

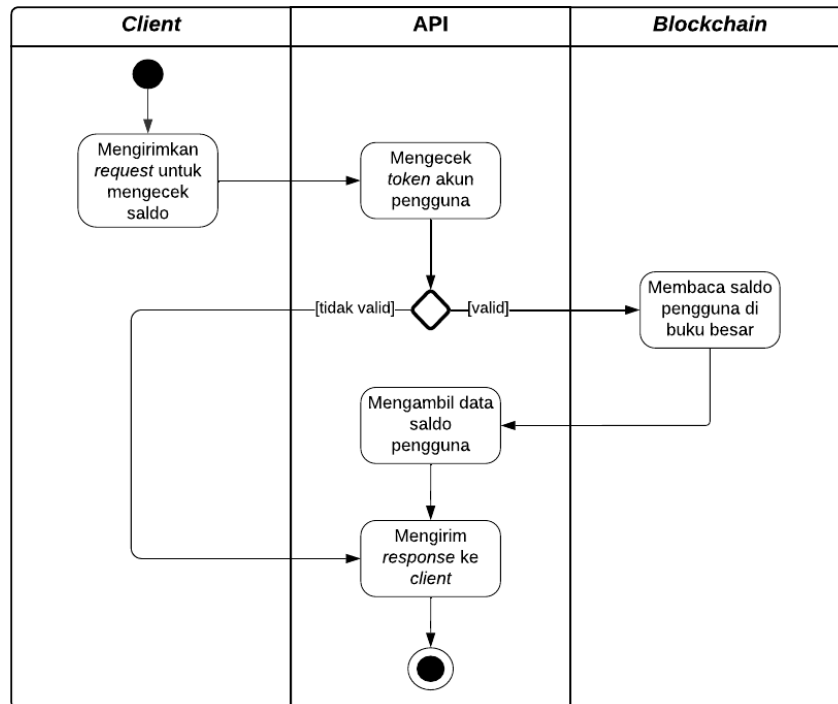


Gambar 3.4. Activity Diagram Proses Pengisian Saldo

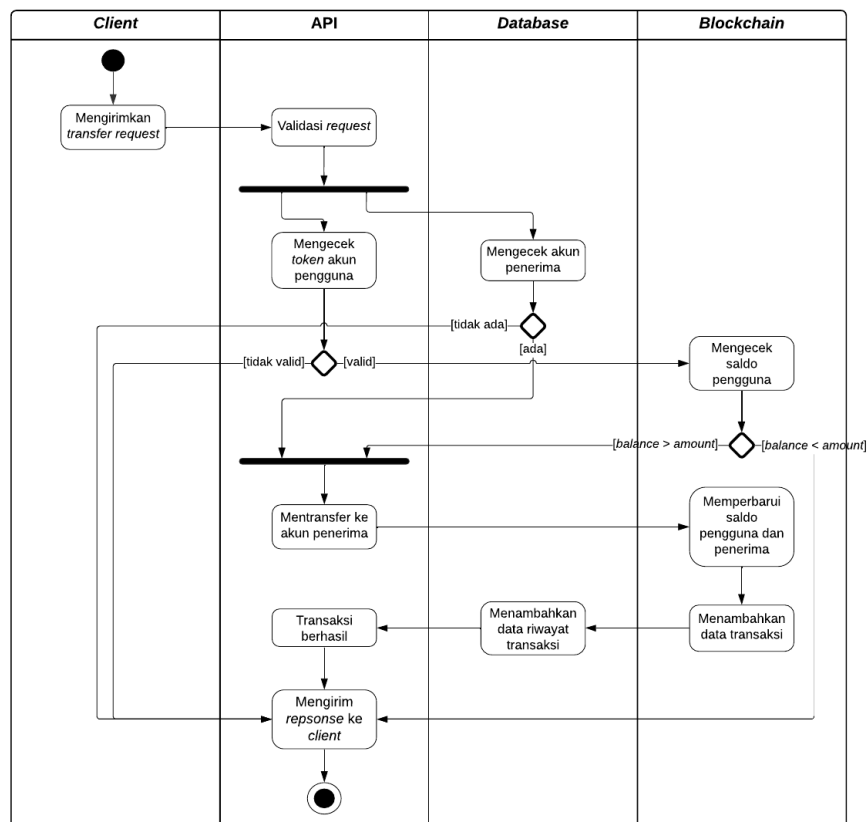
riwayatnya di basis data. Gambar 3.7 menunjukkan proses penarikan saldo pengguna ke rekening bank tujuan.

8. Melihat Riwayat Transaksi

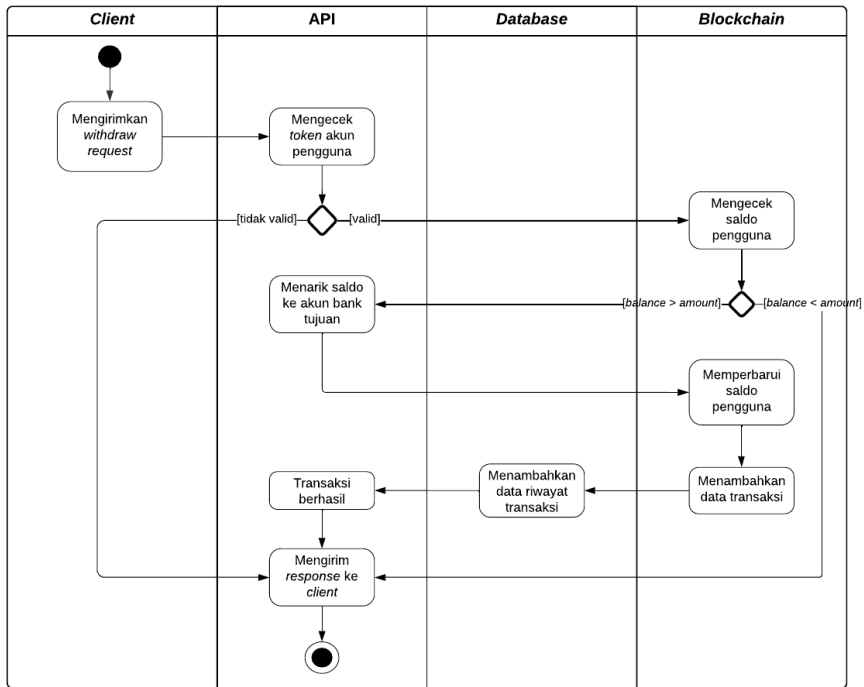
Proses melihat riwayat transaksi pengguna mirip dengan proses melihat profil akun dan juga proses mengecek saldo. *Token* yang dimiliki pengguna akan divalidasi terlebih dahulu untuk membaca riwayat transaksi dari basis data seperti yang ditunjukkan pada Gambar 3.8. Riwayat transaksi yang sesuai dengan pengguna tersebut



Gambar 3.5. Activity Diagram Proses Pengecekan Saldo

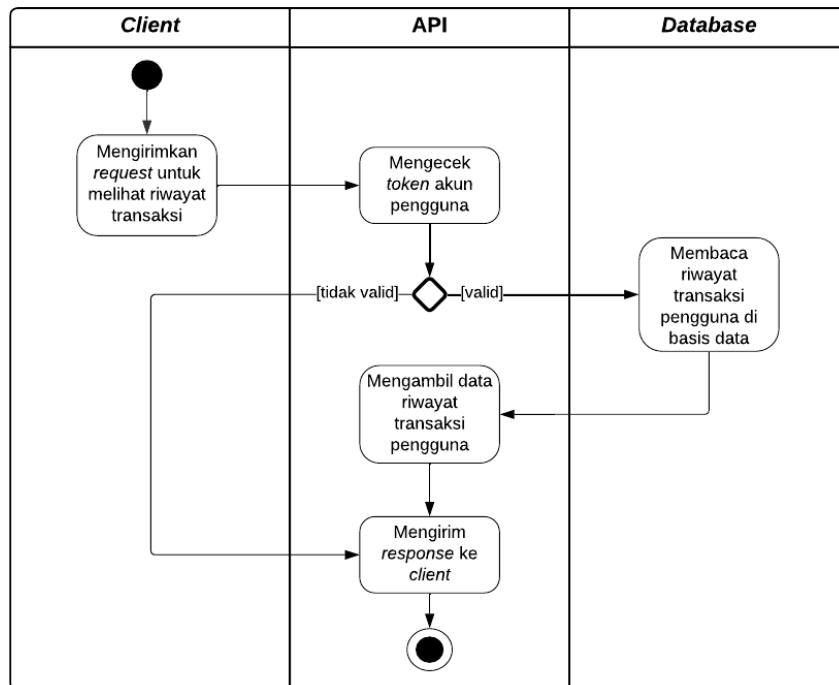


Gambar 3.6. Activity Diagram Proses Transfer Saldo



Gambar 3.7. Activity Diagram Proses Penarikan Saldo

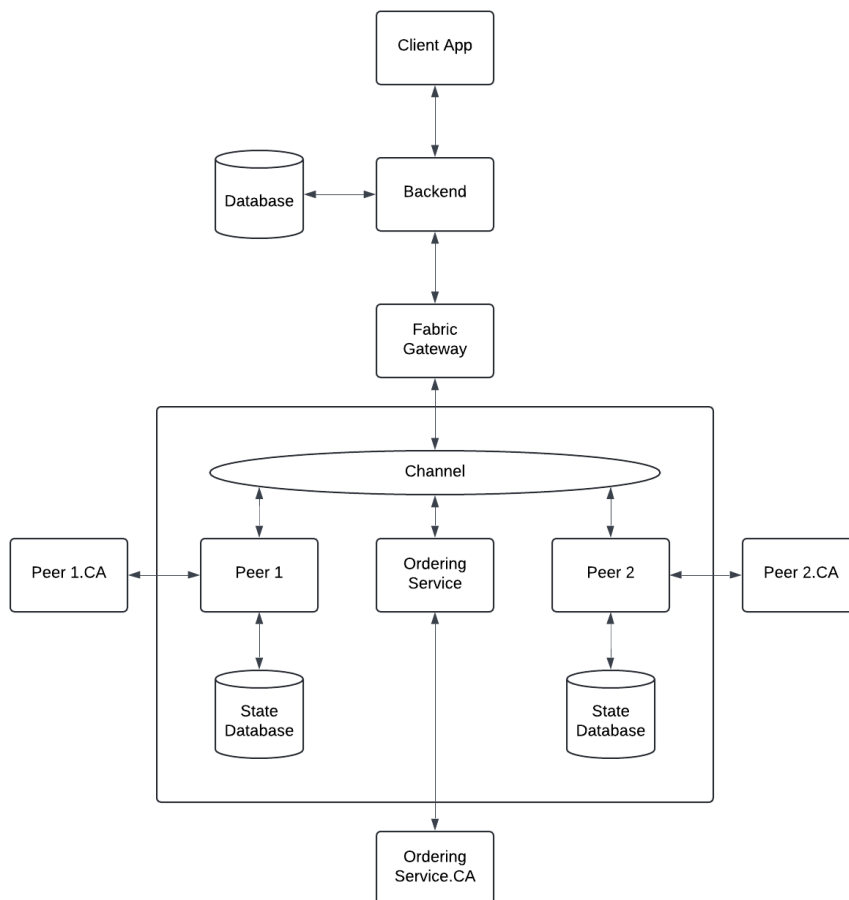
akan dikirimkan sebagai *response*.



Gambar 3.8. Activity Diagram Proses Melihat Riwayat Transaksi

3.2.1.3 Perancangan Arsitektur Sistem *Blockchain*

Perancangan arsitektur sistem *blockchain* MeetCoin bertujuan untuk memberikan gambaran umum mengenai arsitektur sistem yang akan dikembangkan. Rancangan dari arsitektur sistem *blockchain* MeetCoin yang akan dikembangkan ditunjukkan pada Gambar 3.9. Arsitektur sistem *blockchain* MeetCoin terdiri dari dua *peer node* dan *ordering service node* yang tergabung dalam sebuah *channel* di dalam jaringan Fabric. *Channel* memastikan bahwa data transaksi dan buku besar hanya dapat diakses oleh *node* yang tergabung dalam *channel* tersebut. Adanya dua *peer node* bertujuan untuk mengeliminasi risiko *single point of failure* sehingga sistem masih dapat beroperasi ketika salah satu *peer node* mengalami kegagalan. Setiap *node* juga memiliki salinan buku besar yang disimpan dalam *state database* (*world state* dan *blockchain*) untuk menjaga integritas data yang disimpan di dalam jaringan. *Chaincode* yang berisi logika bisnis terpasang di setiap *peer node*. Setiap *chaincode* yang terpasang di setiap *peer node* juga memiliki kebijakan penyesahan yang menentukan *peer node* mana yang harus mengeksekusi dan mengesahkan transaksi. *Ordering service node* akan mengurutkan transaksi yang telah disahkan hingga mencapai konsensus sebelum transaksi tersebut di-*commit* ke dalam buku besar.



Gambar 3.9. Arsitektur Sistem *Blockchain* MeetCoin

Pada arsitektur sistem *blockchain* MeetCoin, terdapat Fabric Gateway yang memungkinkan melakukan eksekusi *chaincode* dalam jaringan Fabric melalui API. Dengan adanya Fabric Gateway, pengguna tidak perlu menggunakan perintah berbasis CLI untuk berinteraksi dengan jaringan Fabric ataupun mengeksekusi *chaincode*. Fabric Gateway juga mempermudah integrasi sistem *blockchain* dengan aplikasi klien. Selain itu, dalam jaringan Fabric terdapat *Certificate Authority* (CA) yang menerbitkan sertifikat digital sebagai identitas dari *node* ataupun pengguna yang terdaftar dan diizinkan dalam jaringan. CA melekat pada setiap *node* yang tergabung dalam jaringan. Selain sertifikat digital, setiap identitas dari pengguna memiliki tanda tangan digital berupa pasangan kunci privat dan publik. Tanda tangan digital tersebut digunakan untuk mengautentikasi dan menjaga integritas pesan yang dikirimkan sehingga komunikasi di dalam jaringan dapat dijamin keamanannya.

Fabric Gateway pada arsitektur sistem *blockchain* MeetCoin terhubung ke *backend* yang memiliki basis data tersendiri. *Backend* juga mengimplementasikan API yang memungkinkan pengguna untuk mengakses proses bisnis sistem *blockchain* MeetCoin. Namun, terdapat perbedaan antara Fabric Gateway dengan *backend*. Fabric Gateway merupakan penghubung antara *backend* dengan jaringan Fabric, sedangkan *backend* merupakan aplikasi yang berinteraksi dengan pengguna untuk mengirimkan transaksi dan menerima hasil transaksi dari jaringan Fabric. Basis data yang terdapat pada *backend* berperan sebagai tempat untuk menyimpan data pengguna. Selain itu, basis data *backend* berfungsi untuk mencatat salinan transaksi yang terjadi di dalam jaringan *blockchain*.

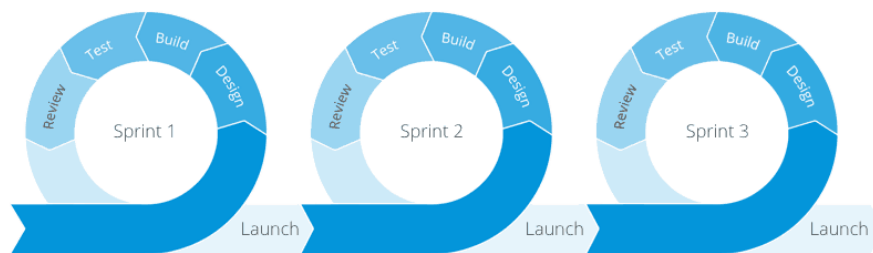
Secara keseluruhan, arsitektur sistem *blockchain* MeetCoin terdiri dari tiga komponen, yaitu jaringan Fabric, Fabric Gateway, dan *backend*. Ketiga komponen tersebut memiliki fungsi dan peran masing-masing yang saling melengkapi sehingga membentuk satu kesatuan sistem. Jaringan Fabric berfungsi sebagai pemroses transaksi yang aman dan transparan untuk menjaga integritas transaksi aset digital dalam sistem. Fabric Gateway berfungsi sebagai jembatan untuk mengirimkan transaksi dan melakukan eksekusi *chaincode* dari aplikasi klien ke jaringan Fabric. *Backend* berfungsi sebagai manajemen data pengguna, autentikasi akun pengguna, dan mencatat salinan data transaksi.

3.2.2 Metode Implementasi

Dalam proses implementasi sistem *blockchain* MeetCoin, metode implementasi yang digunakan meliputi aspek *Software Development Life Cycle* (SDLC), arsitektur sistem *blockchain* (meliputi kerangka kerja, algoritme konsensus, dan *state database*), serta bahasa pemrograman yang digunakan.

3.2.2.1 *Software Development Life Cycle*

Pada pengembangan sistem *blockchain* MeetCoin, metode pengembangan atau SDLC yang digunakan adalah metode *Agile*. Metode *Agile* berfokus pada rangkaian pengembangan sistem dengan pendekatan berulang dan bertahap. Berbeda dengan metode *Waterfall* yang harus dijalankan secara linear dan tidak dapat diubah di tengah proses pengembangan, metode *Agile* memiliki kemampuan beradaptasi terhadap kebutuhan atau persyaratan sistem yang dapat berubah-ubah. Pada metode *Agile*, proses pengembangan dibagi menjadi bagian-bagian kecil (*backlog*) dalam iterasi pendek atau satu siklus yang disebut dengan *sprint* seperti yang ditunjukkan pada Gambar 3.10. Setiap *sprint* akan menghasilkan peningkatan suatu bagian atau fungsi sistem dan *sprint* akan dilakukan berulang hingga membentuk sistem secara keseluruhan. Metode *Agile* juga mengurangi risiko kesalahan sistem karena setiap bagian atau fungsi yang dikembangkan akan diuji terlebih dahulu sebelum memulai *sprint* selanjutnya.



Gambar 3.10. SDLC dengan Metode *Agile* [16]

3.2.2.2 *Kerangka Kerja Blockchain*

Kerangka kerja *blockchain* yang digunakan dalam pengembangan sistem *blockchain* MeetCoin adalah Hyperledger Fabric dengan versi 2.5.9 LTS (*Long-Term Support*). Kerangka kerja Hyperledger Fabric memiliki lisensi *open-source* dan arsitektur modular yang mudah untuk dikonfigurasi sesuai dengan kebutuhan pengembangan sistem. Karena arsitektur modularnya, Hyperledger Fabric juga menjadi lebih mudah untuk diskalakan. Dukungan penulisan *chaincode* menggunakan bahasa pemrograman *general-purpose* seperti Go, Java, dan Node.js membuat pengembang memiliki banyak pilihan dalam mengembangkan *chaincode* sesuai dengan bahasa pemrograman yang dikuasai. Dari sisi kinerja, alur transaksi *execute-order-validate* pada Hyperledger Fabric memungkinkan eksekusi transaksi secara konkuren yang membuat sistem yang dikembangkan memiliki kinerja yang lebih baik dibandingkan dengan kerangka kerja atau platform *blockchain* yang mengeksekusi transaksi secara *order-execute* atau berurutan.

3.2.2.3 Mekanisme Konsensus

Mekanisme konsensus yang digunakan untuk mencapai kesepakatan mengenai urutan blok data di Hyperledger Fabric adalah Raft. Raft memiliki cara kerja dan implementasi yang sederhana serta pemilihan *leader* yang dinamis dan cepat. Hal tersebut membuat Raft memiliki kinerja yang paling cepat dibandingkan dengan Kafka dan PBFT dalam mencapai konsensus. Selain itu, Raft juga memiliki penggunaan sumber daya yang paling rendah dibandingkan dengan Kafka dan PBFT. Berbeda dengan PBFT yang dapat menoleransi *node* yang bertindak jahat seperti mengirimkan pesan yang salah, Raft hanya mampu menoleransi kesalahan akibat kegagalan *node*. Namun, dalam jaringan privat berizin seperti Hyperledger Fabric yang memiliki tata kelola yang ketat dan setiap identitas diketahui, risiko *node* yang bertindak jahat sangat rendah dibandingkan dengan jaringan publik tanpa izin.

3.2.2.4 State Database

State database yang digunakan untuk menyimpan data aset buku besar adalah LevelDB. LevelDB merupakan *state database* bawaan dari Hyperledger Fabric yang bersifat *embed* dengan setiap *peer node*. Karena sifatnya yang *embed*, LevelDB tidak memerlukan konfigurasi tambahan serta memiliki penggunaan sumber daya yang rendah. LevelDB juga memiliki implementasi sederhana dengan struktur data pasangan *key-value*, berbeda dengan CouchDB yang menggunakan struktur data JSON. Meskipun LevelDB tidak mendukung *indexing* untuk dapat melakukan *query* kompleks seperti CouchDB, hal tersebut menjadi kelebihan bagi LevelDB untuk dapat melakukan *query* sederhana seperti *Put*, *Get*, dan *Delete* dengan lebih cepat. Selain itu, LevelDB juga mendukung operasi baca/tulis *atomic* untuk memastikan konsistensi data yang disimpan.

3.2.2.5 Bahasa Pemrograman

Bahasa pemrograman yang digunakan pada *chaincode*, Fabric Gateway API, dan *backend* API adalah bahasa Go. Go dipilih karena memiliki sintaks penulisan yang sederhana serta proses komputasi yang cepat dan efisien dengan adanya fitur konkurensi dan *garbage collector*. Fitur konkurensi dalam Go dapat memanfaatkan penggunaan semua CPU yang tersedia dan fitur *garbage collector* untuk membantu pengelolaan memori yang menghapus *entry* secara otomatis ketika sudah tidak digunakan lagi. Selain itu, dalam pengembangan *chaincode*, *chaincode* yang dikembangkan menggunakan Go memiliki kinerja paling baik dalam jaringan Hyperledger Fabric dibandingkan dengan *chaincode* yang dikembangkan dengan Node.js dan Java [47].

3.2.3 Metode Pengujian

Pengujian sistem *blockchain* MeetCoin dilakukan untuk mengevaluasi kinerja dari sistem yang dikembangkan. Adanya evaluasi kinerja sistem digunakan untuk mengukur beban kerja yang dapat diterima oleh sistem dan sebagai standar untuk optimalisasi sistem dalam pengembangan lebih lanjut. Pengujian sistem *blockchain* MeetCoin dilakukan menggunakan alat Hyperledger Caliper. Lebih spesifik, pengujian hanya dilakukan pada jaringan Fabric karena proses inti transaksi terjadi di dalam jaringan Fabric. Selain itu, data transaksi dan saldo pengguna juga disimpan dalam buku besar yang terdapat di dalam jaringan Fabric.

Metrik pengujian Hyperledger Caliper atau variabel yang diamati pada pengujian ini adalah *transaction success rate*, *throughput*, *latency*, dan *resource consumption*. Berikut adalah penjelasan mengenai masing-masing metrik pengujian Hyperledger Caliper.

1. Success Rate

$$Success\ Rate = \frac{\text{Total transaksi berhasil}}{\text{Total transaksi dikirimkan}} \times 100\% \quad (3-1)$$

Success rate merupakan metrik pengujian yang digunakan untuk mengukur tingkat keberhasilan transaksi yang diproses oleh sistem. *Success rate* dapat dihitung dengan membagi total transaksi yang berhasil diproses sistem dengan total transaksi yang dikirimkan ke sistem seperti yang ditunjukkan pada persamaan 3-1. Metrik ini menunjukkan persentase keberhasilan sistem dalam memproses transaksi.

2. Throughput

$$Throughput = \frac{\text{Total transaksi berhasil}}{\text{Total waktu dalam detik}} \quad (3-2)$$

Throughput merupakan metrik yang digunakan untuk menghitung transaksi yang berhasil diproses oleh sistem dalam periode waktu tertentu, umumnya dalam waktu satu detik. Satuan *throughput* dinyatakan sebagai *transactions per second* (TPS). Persamaan untuk menghitung *throughput* ditunjukkan pada persamaan 3-2 dengan membagi total transaksi yang berhasil dengan total waktu dalam detik. *Throughput* bertujuan untuk menunjukkan kinerja dari sistem dalam memproses transaksi dalam waktu satu detik.

3. Latency

$$Latency = \text{Waktu saat transaksi dikonfirmasi} - \text{Waktu saat transaksi dikirim} \quad (3-3)$$

Latency merupakan metrik untuk menghitung total waktu yang dibutuhkan sistem untuk memproses transaksi hingga efek transaksinya dapat digunakan di seluruh jaringan. *Latency* dapat dihitung dengan mencari selisih waktu saat transaksi

dikirimkan dan waktu saat transaksi dikonfirmasi seperti yang ditunjukkan pada persamaan 3-3. Metrik *latency* akan menghasilkan luaran statistik berupa *latency* minimum, maksimum, dan rerata. Metrik *latency* bertujuan untuk menunjukkan seberapa cepat sistem dalam memproses transaksi.

4. *Resource Consumption*

Resource consumption merupakan metrik untuk memonitor sumber daya yang dikonsumsi oleh jaringan ketika menjalankan suatu proses. Luaran yang dihasilkan oleh metrik *resource consumption* berupa penggunaan memori maksimum dan rerata, CPU maksimum dan rerata, *network I/O*, dan *disc I/O*. Metrik *resource consumption* bertujuan untuk menunjukkan seberapa besar sumber daya yang dibutuhkan jaringan untuk menjalankan suatu proses.

Pengujian tidak dilakukan pada seluruh proses bisnis sistem, melainkan hanya pada proses inti transaksi yang meliputi pengisian saldo, pengecekan saldo, dan transfer saldo. Ketiga proses bisnis tersebut dipilih karena merupakan proses yang paling sering berinteraksi dengan jaringan Fabric dan *chaincode*, seperti membaca dan melakukan perubahan pada buku besar. Variabel yang diatur dalam pengujian ini adalah sebagai berikut.

1. Jenis transaksi yang akan diuji yaitu pengisian saldo, pengecekan saldo, dan transfer saldo.
2. *Worker* merupakan satuan *worker process* yang akan mengirimkan transaksi. Variabel *worker* dalam pengujian ini ditetapkan ke tiga nilai yang berbeda: 5, 10, dan 15 *worker*.
3. Variabel jumlah transaksi dalam pengujian ini ditetapkan ke tiga nilai yang berbeda: 1000, 2000, dan 3000 transaksi.
4. Variabel laju transaksi yang dikirimkan dalam pengujian ini juga ditetapkan ke tiga nilai berbeda: 50, 100, dan 150 transaksi per detik.

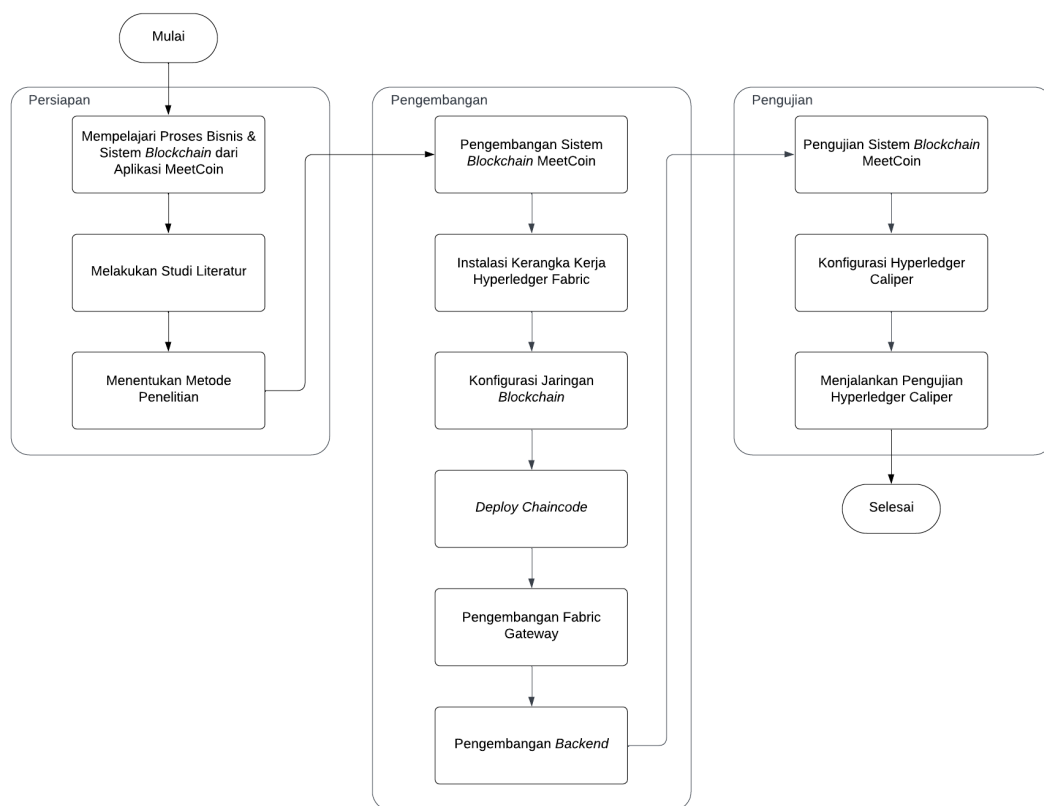
Skenario pengujian yang dilakukan adalah sebagai berikut.

1. Skenario pertama akan mengubah jumlah *worker* pada setiap jenis transaksi untuk mengetahui dampaknya terhadap indikator kinerja sistem. Variabel *worker* ditetapkan sebanyak 5, 10, dan 15 *worker*. Untuk variabel lain seperti jumlah transaksi ditetapkan sebanyak 1000 transaksi dan laju transaksi sebanyak 50 transaksi per detik.
2. Skenario kedua akan mengubah jumlah transaksi pada setiap jenis transaksi untuk melihat efeknya terhadap indikator kinerja sistem. Variabel jumlah transaksi ditetapkan sebanyak 1000, 2000, dan 3000 transaksi. Variabel *worker* ditetapkan sebanyak 5 *worker* dan laju transaksi ditetapkan sebanyak 50 transaksi per detik.

3. Skenario ketiga akan mengubah laju transaksi yang dikirimkan pada setiap jenis transaksi untuk mengetahui efeknya terhadap indikator kinerja sistem. Variabel laju transaksi ditetapkan sebanyak 50, 100, dan 150 transaksi per detik. Variabel *worker* ditetapkan sebanyak 5 *worker* dan variabel jumlah transaksi ditetapkan sebanyak 1000 transaksi.

3.3 Alur Tugas Akhir

Alur pengerjaan tugas akhir meliputi beberapa tahapan atau prosedur yang dilakukan secara berurutan dan sistematis. Alur dari tugas akhir disajikan dalam bentuk diagram *flowchart* yang ditunjukkan pada Gambar 3.11.



Gambar 3.11. Alur Tugas Akhir

3.3.1 Mempelajari Proses Bisnis & Sistem *Blockchain* dari Aplikasi MeetCoin

Pada tahapan ini, dilakukan analisis terhadap proses bisnis dan sistem *blockchain* dari aplikasi MeetCoin sebelumnya dengan membaca laporan penelitian dari aplikasi MeetCoin, membaca dokumentasi aplikasi, dan *hands-on* aplikasi. Tahapan ini bertujuan untuk mendapatkan pemahaman mengenai kebutuhan sistem yang mendasari proses bisnis dan mengetahui cara kerja dari sistem *blockchain* MeetCoin. Selain itu, tahapan ini juga menganalisis kelebihan dan kekurangan sistem *blockchain* MeetCoin untuk dapat diperbaiki dan ditingkatkan pada tugas akhir ini.

3.3.2 Melakukan Studi Literatur

Tahapan selanjutnya adalah melakukan studi literatur. Studi literatur dilakukan dengan merumuskan permasalahan yang muncul setelah mempelajari proses bisnis dan sistem *blockchain* dari aplikasi MeetCoin, mengidentifikasi batasan tugas akhir yang terkait dengan rumusan masalah untuk menentukan cakupan tugas akhir, dan mengumpulkan serta menganalisis literatur yang terkait dengan rumusan masalah. Literatur yang dikumpulkan dan dianalisis pada tugas akhir ini meliputi literatur mengenai penerapan teknologi *blockchain* di sektor keuangan dan literatur yang berkaitan dengan perancangan sistem *blockchain* menggunakan kerangka kerja Hyperledger Fabric. Selain itu, tahapan ini juga membandingkan beberapa metode yang akan digunakan dalam pengerjaan tugas akhir.

3.3.3 Menentukan Metode Penelitian

Setelah melakukan studi literatur, tahapan berikutnya adalah menentukan metode penelitian. Metode yang digunakan dalam tugas akhir ini meliputi metode perancangan, metode implementasi, dan metode pengujian. Pada metode perancangan, terdapat beberapa tahapan yang meliputi analisis kebutuhan fungsional dan non-fungsional sistem, perancangan *activity diagram* yang merepresentasikan proses bisnis sistem, dan perancangan arsitektur sistem *blockchain* MeetCoin. Metode implementasi terdiri dari aspek SDLC, arsitektur sistem *blockchain* (kerangka kerja, mekanisme konsensus, dan *state database*), dan bahasa pemrograman yang digunakan untuk mengembangkan sistem *blockchain* MeetCoin. Metode pengujian yang dilakukan yaitu penentuan variabel yang diamati berupa metrik *transaction success rate*, *throughput*, *latency*, dan *resource consumption* serta variabel yang diatur berupa jenis transaksi, jumlah *worker*, jumlah transaksi, dan laju transaksi yang kemudian akan diuji pada tiga skenario yang berbeda.

3.3.4 Pengembangan Sistem *Blockchain* MeetCoin

Tahapan pengembangan sistem *blockchain* MeetCoin terdiri dari instalasi kerangka kerja Hyperledger Fabric, konfigurasi jaringan *blockchain*, pengembangan *chaincode*, pengembangan Fabric Gateway API, dan pengembangan *backend*. Berikut adalah penjelasan mengenai masing-masing tahapan pengembangan sistem *blockchain* MeetCoin.

3.3.4.1 Instalasi Kerangka Kerja Hyperledger Fabric

Instalasi kerangka kerja Hyperledger Fabric dilakukan dengan mengunduh *script file* dengan perintah `curl -sSLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh && chmod +x install-fabric.sh` pada terminal WSL2. Perintah tersebut akan mengunduh *script*

file dengan nama `install-fabric.sh`. *Script file* tersebut kemudian dijalankan dengan perintah `./install-fabric.sh -f 2.5.9 binary docker`. *Script file* `install-fabric.sh` akan mengunduh *binary file* dan Docker *image* dari Fabric versi 2.5.9 yang dibutuhkan untuk menjalankan jaringan *blockchain*. Setelah proses unduh selesai, kerangka kerja Hyperledger Fabric berhasil diinstalasi.

3.3.4.2 Konfigurasi Jaringan *Blockchain*

Setelah berhasil melakukan instalasi Hyperledger Fabric, tahapan selanjutnya adalah melakukan konfigurasi jaringan *blockchain*. Jaringan *blockchain* yang digunakan pada tugas akhir ini menggunakan jaringan pengujian yang disediakan oleh Hyperledger Fabric yang diunduh melalui *repository* GitHub resmi Hyperledger Fabric. Jaringan pengujian Hyperledger Fabric juga menyediakan *script file* yang berfungsi untuk membantu pengembang dalam melakukan konfigurasi ataupun menjalankan jaringan *blockchain*. Jaringan *blockchain* terdiri dari dua *peer node* dan *ordering service node*. *Node* tersebut dikonfigurasi dalam *file* YAML yang akan berjalan dalam bentuk Docker *container* ketika jaringan *blockchain* dijalankan. Gambar 3.12 menunjukkan cuplikan kode konfigurasi untuk *node* dalam jaringan *blockchain*.

Selain *node*, yang perlu dikonfigurasi dalam jaringan *blockchain* adalah mekanisme konsensus. Mekanisme konsensus yang digunakan pada jaringan *blockchain* adalah Raft. Konfigurasi mekanisme konsensus Raft dalam jaringan *blockchain* ditunjukkan pada Gambar 3.13. Pada gambar tersebut, konfigurasi penggunaan Raft didefinisikan mulai pada baris kedua yaitu `ChannelUsingRaft`. Baris kode selanjutnya merupakan detail konfigurasi dari Raft seperti mendefinisikan *node* mana yang akan menjadi *orderer*.

Setelah melakukan konfigurasi *node* dan mekanisme konsensus, jaringan *blockchain* dapat mulai dijalankan. Untuk menjalankan jaringan *blockchain*, perintah yang digunakan adalah `./network.sh up createChannel -c mychannel -ca`. Perintah tersebut akan melakukan *bootstrapping* jaringan *blockchain*. Argumen `createChannel -c mychannel` merupakan perintah tambahan untuk membuat sebuah *channel* yang bernama `mychannel` dan menetapkan *node* untuk bergabung di dalam *channel* tersebut, sedangkan argumen `-ca` merupakan perintah untuk menggunakan Certificate Authority (CA) sebagai manajemen identitas pengguna di dalam jaringan. Meskipun argumen mengenai *state database* tidak disebutkan secara eksplisit pada perintah di atas, LevelDB juga ikut diinisiasi dalam jaringan *blockchain* karena LevelDB merupakan *state database* bawaan dari Hyperledger Fabric.

3.3.4.3 Deploy Chaincode

Tahapan berikutnya setelah melakukan konfigurasi jaringan *blockchain* adalah *men-deploy chaincode* yang merupakan inti proses bisnis dari jaringan *blockchain*. Cha-

```

1  version: "3.7"
2
3  volumes:
4    orderer.example.com:
5    peer0.org1.example.com:
6    peer0.org2.example.com:
7
8  networks:
9    test:
10     name: fabric_test
11
12  services:
13    peer0.org1.example.com:
14     container_name: peer0.org1.example.com
15     image: hyperledger/fabric-peer:2.5.9
16     labels:
17       service: hyperledger-fabric
18     environment:
19       - FABRIC_CFG_PATH=/etc/hyperledger/peerconfig
20       - FABRIC_LOGGING_SPEC=INFO
21       #- FABRIC_LOGGING_SPEC=DEBUG
22       - CORE_PEER_TLS_ENABLED=true
23       - CORE_PEER_PROFILE_ENABLED=false
24       - CORE_PEER_TLS_CERT_FILE=/etc/hyperledger/fabric/tls/server.crt
25       - CORE_PEER_TLS_KEY_FILE=/etc/hyperledger/fabric/tls/server.key
26       - CORE_PEER_TLS_ROOTCERT_FILE=/etc/hyperledger/fabric/tls/ca.crt
27       # Peer specific variables
28       - CORE_PEER_ID=peer0.org1.example.com
29       - CORE_PEER_ADDRESS=peer0.org1.example.com:7051
30       - CORE_PEER_LISTENADDRESS=0.0.0.0:7051
31       - CORE_PEER_CHAINCODEADDRESS=peer0.org1.example.com:7052
32       - CORE_PEER_CHAINCODELISTENADDRESS=0.0.0.0:7052
33       - CORE_PEER_GOSSIP_BOOTSTRAP=peer0.org1.example.com:7051
34       - CORE_PEER_GOSSIP_EXTERNALENDPOINT=peer0.org1.example.com:7051
35       - CORE_PEER_LOCALMSPID=Org1MSP
36       - CORE_PEER_MSPCONFIGPATH=/etc/hyperledger/fabric/msp
37       - CORE_OPERATIONS_LISTENADDRESS=peer0.org1.example.com:9444
38       - CORE_METRICS_PROVIDER=prometheus
39       - CHAINCODE_AS_A_SERVICE_BUILDER_CONFIG={"peername": "peer0org1"}
40       - CORE_CHAINCODE_EXECUTETIMEOUT=300s
41     volumes:
42     -
43     ../organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com:/etc/hyperledger
44     - peer0.org1.example.com:/var/hyperledger/production
45     working_dir: /root
46     command: peer node start
47     ports:
48       - 7051:7051
49       - 9444:9444
50     networks:
51     - test

```

Gambar 3.12. Konfigurasi *Node*

incode yang digunakan pada sistem *blockchain* MeetCoin mengadopsi standar *token* ERC-20 yang ada pada Ethereum. Standar *token* ERC-20 merupakan standar yang digunakan untuk membuat *token* atau koin yang dapat dipertukarkan atau ditransaksikan. *Chaincode* yang digunakan diunduh melalui *repository* GitHub resmi Hyperledger Fabric. *Chaincode* dengan standar ERC-20 memiliki berbagai fungsi yang diasosiasikan dengan proses bisnis sistem *blockchain* MeetCoin. Secara umum, fungsi yang terdapat

```

1 Profiles:
2   ChannelUsingRaft:
3     <<: *ChannelDefaults
4     Orderer:
5       <<: *OrdererDefaults
6       OrdererType: etcdraft
7       EtcdRaft:
8         Consenters:
9           - Host: orderer.example.com
10            Port: 7050
11            ClientTLSCert:
12              ../organizations/ordererOrganizations/example.com/orderers/orderer.example.com/tls/server.crt
13            ServerTLSCert:
14              ../organizations/ordererOrganizations/example.com/orderers/orderer.example.com/tls/server.crt
15          Organizations:
16            - *OrdererOrg
17          Capabilities: *OrdererCapabilities
18        Application:
19          <<: *ApplicationDefaults
20          Organizations:
21            - *Org1
22            - *Org2
23          Capabilities: *ApplicationCapabilities

```

Gambar 3.13. Konfigurasi Mekanisme Konsensus Raft

pada standar ERC-20 yaitu *Mint*, *Burn*, *ClientAccountBalance*, dan *Transfer*. Fungsi *Mint* diasosiasikan dengan proses bisnis pengisian saldo, fungsi *Burn* diasosiasikan dengan penarikan saldo, fungsi *ClientAccountBalance* diasosiasikan dengan pengecekan saldo, dan fungsi *Transfer* diasosiasikan dengan transfer saldo.

Gambar 3.14, Gambar 3.15, Gambar 3.16, dan Gambar 3.17 menunjukkan cuplikan kode dari setiap fungsi *Mint*, *Burn*, *ClientAccountBalance*, dan *Transfer*. Fungsi *Mint* dan *Burn* memiliki satu parameter yaitu *amount* saldo. Fungsi *Mint* akan menambahkan token atau koin ke akun pengguna sesuai dengan parameter yang diberikan, sedangkan fungsi *Burn* akan mengurangi saldo pengguna. Pada fungsi *Burn* terdapat pengecekan jumlah saldo agar saldo yang ditarik tidak melebihi jumlah saldo yang dimiliki pengguna. Fungsi *ClientAccountBalance* akan membaca dan mengembalikan nilai berupa saldo yang dimiliki oleh pengguna dari buku besar. Fungsi *Transfer* memiliki dua parameter, yaitu *receiver* atau penerima dan *amount* saldo. Parameter *receiver* diisi dengan *account ID* pengguna. Setiap fungsi tersebut akan mengecek identitas dari pengguna terlebih dahulu, jika identitas tersebut valid maka fungsi akan dieksekusi.

Sebelum dapat digunakan, *chaincode* perlu di-*deploy* ke jaringan *blockchain* terlebih dahulu. Proses *deploy chaincode* dilakukan dengan perintah `./network.sh deployCC -ccn basic -ccp ../token-erc-20/chaincode-go/ -ccl go -ccep "AND('Org1MSP.peer', 'Org2MSP.peer')"`. Perintah `./network.sh deployCC` akan melakukan *bundling* pada folder *chaincode* sehingga menjadi *pac-*

```

1 func (s *SmartContract) Mint(ctx contractapi.TransactionContextInterface, amount int)
  error {
2
3   ...
4
5   minter, err := ctx.GetClientIdentity().GetID()
6   if err != nil {
7       return fmt.Errorf("failed to get client id: %v", err)
8   }
9
10  if amount <= 0 {
11      return fmt.Errorf("mint amount must be a positive integer")
12  }
13
14  currentBalanceBytes, err := ctx.GetStub().GetState(minter)
15  if err != nil {
16      return fmt.Errorf("failed to read minter account %s from world state: %v", minter,
17      err)
18  }
19
20  var currentBalance int
21
22  if currentBalanceBytes == nil {
23      currentBalance = 0
24  } else {
25      currentBalance, _ = strconv.Atoi(string(currentBalanceBytes))
26  }
27
28  updatedBalance, err := add(currentBalance, amount)
29  if err != nil {
30      return err
31  }
32
33  err = ctx.GetStub().PutState(minter, []byte(strconv.Itoa(updatedBalance)))
34  if err != nil {
35      return err
36  }
37
38  ...
39
40  transferEvent := event{"0x0", minter, amount}
41  transferEventJSON, err := json.Marshal(transferEvent)
42  if err != nil {
43      return fmt.Errorf("failed to obtain JSON encoding: %v", err)
44  }
45
46  err = ctx.GetStub().SetEvent("Transfer", transferEventJSON)
47  if err != nil {
48      return fmt.Errorf("failed to set event: %v", err)
49  }
50
51  log.Printf("minter account %s balance updated from %d to %d", minter, currentBalance,
52  updatedBalance)
53
54  return nil
55 }

```

Gambar 3.14. Fungsi *Mint*

kage yang kemudian akan diinstalasi pada setiap *peer node*. Argumen `-ccn basic` `-ccp ../token-erc-20/chaincode-go/` `-ccl go` secara berturut-turut mendefinisikan nama dari *chaincode*, *path* dari folder *chaincode*, dan bahasa pemrograman yang digunakan pada *chaincode* tersebut. Argumen `-ccep "AND ('Org1MSP.peer',`

```

1 func (s *SmartContract) Burn(ctx contractapi.TransactionContextInterface, amount int)
  error {
2
3   ...
4
5   minter, err := ctx.GetClientIdentity().GetID()
6   if err != nil {
7       return fmt.Errorf("failed to get client id: %v", err)
8   }
9
10  if amount <= 0 {
11      return errors.New("burn amount must be a positive integer")
12  }
13
14  currentBalanceBytes, err := ctx.GetStub().GetState(minter)
15  if err != nil {
16      return fmt.Errorf("failed to read minter account %s from world state: %v", minter,
17      err)
18  }
19
20  var currentBalance int
21
22  if currentBalanceBytes == nil {
23      return errors.New("The balance does not exist")
24  }
25
26  currentBalance, _ = strconv.Atoi(string(currentBalanceBytes))
27
28  updatedBalance, err := sub(currentBalance, amount)
29  if err != nil {
30      return err
31  }
32
33  err = ctx.GetStub().PutState(minter, []byte(strconv.Itoa(updatedBalance)))
34  if err != nil {
35      return err
36  }
37
38  ...
39
40  transferEvent := event{minter, "0x0", amount}
41  transferEventJSON, err := json.Marshal(transferEvent)
42  if err != nil {
43      return fmt.Errorf("failed to obtain JSON encoding: %v", err)
44  }
45
46  err = ctx.GetStub().SetEvent("Transfer", transferEventJSON)
47  if err != nil {
48      return fmt.Errorf("failed to set event: %v", err)
49  }
50
51  log.Printf("minter account %s balance updated from %d to %d", minter, currentBalance,
52  updatedBalance)
53
54  return nil
55 }

```

Gambar 3.15. Fungsi *Burn*

'Org2MSP.peer') " merupakan perintah untuk menentukan kebijakan pengesahan transaksi. Argumen tersebut mendefinisikan bahwa transaksi harus disahkan oleh kedua *peer node* sebelum dapat diproses lebih lanjut.

```

1 func (s *SmartContract) ClientAccountBalance(ctx contractapi.TransactionContextInterface)
  (int, error) {
2
3   ...
4
5   clientID, err := ctx.GetClientIdentity().GetID()
6   if err != nil {
7       return 0, fmt.Errorf("failed to get client id: %v", err)
8   }
9
10  balanceBytes, err := ctx.GetStub().GetState(clientID)
11  if err != nil {
12      return 0, fmt.Errorf("failed to read from world state: %v", err)
13  }
14  if balanceBytes == nil {
15      return 0, fmt.Errorf("the account %s does not exist", clientID)
16  }
17
18  balance, _ := strconv.Atoi(string(balanceBytes))
19
20  return balance, nil
21 }

```

Gambar 3.16. Fungsi *ClientAccountBalance*

```

1 func (s *SmartContract) Transfer(ctx contractapi.TransactionContextInterface, recipient
  string, amount int) error {
2
3   ...
4
5   clientID, err := ctx.GetClientIdentity().GetID()
6   if err != nil {
7       return fmt.Errorf("failed to get client id: %v", err)
8   }
9
10  err = transferHelper(ctx, clientID, recipient, amount)
11  if err != nil {
12      return fmt.Errorf("failed to transfer: %v", err)
13  }
14
15  transferEvent := event{clientID, recipient, amount}
16  transferEventJSON, err := json.Marshal(transferEvent)
17  if err != nil {
18      return fmt.Errorf("failed to obtain JSON encoding: %v", err)
19  }
20  err = ctx.GetStub().SetEvent("Transfer", transferEventJSON)
21  if err != nil {
22      return fmt.Errorf("failed to set event: %v", err)
23  }
24
25  return nil
26 }

```

Gambar 3.17. Fungsi *Transfer*

3.3.4.4 Pengembangan Fabric Gateway

Fabric Gateway dikembangkan dalam bentuk API yang menghubungkan *backend* dengan jaringan *blockchain*. Pada Fabric Gateway, terdapat beberapa konfigurasi yang dibutuhkan agar dapat membentuk koneksi dengan jaringan *blockchain*. Konfigurasi tersebut ditunjukkan pada Gambar 3.18. Beberapa konfigurasi yang dibutuhkan seperti sertifikat digital dan *private key* pengguna. Sertifikat digital dan *private key* tersebut kemudian akan divalidasi terlebih dahulu. Jika identitas pengguna valid, pengguna dapat terhubung dengan jaringan *blockchain* dan dapat melakukan eksekusi *chaincode*. Koneksi pengguna ke jaringan *blockchain* akan ditutup setelah pengguna melakukan eksekusi *chaincode*.

```
1 certPath = fmt.Sprintf(cryptoPath+"/users/%s@org1.example.com/msp/signcerts/cert.pem",
2   username)
3
4 // The gRPC client connection should be shared by all Gateway connections to this
5   endpoint
6   clientConnection := newGrpcConnection()
7   defer clientConnection.Close()
8
9   id := newIdentity()
10  sign := newSign()
11
12 // Create a Gateway connection for a specific client identity
13 gateway, err := client.Connect(
14   id,
15   client.WithSign(sign),
16   client.WithClientConnection(clientConnection),
17   // Default timeouts for different gRPC calls
18   client.WithEvaluateTimeout(5*time.Second),
19   client.WithEndorseTimeout(15*time.Second),
20   client.WithSubmitTimeout(5*time.Second),
21   client.WithCommitStatusTimeout(1*time.Minute),
22 )
23 if err != nil {
24   panic(err)
25 }
26 defer gateway.Close()
```

Gambar 3.18. Konfigurasi Fabric Gateway

API yang dikembangkan pada Fabric Gateway ditujukan untuk mengakses proses bisnis yang ada pada *chaincode*, yaitu *mint*, *burn*, *balance*, dan *transfer*. Selain itu, terdapat tiga API tambahan, yaitu *initializer*, *account ID*, dan *register*. API *initializer* digunakan untuk menginisiasi *chaincode*, sedangkan API *account ID* digunakan untuk mengambil data *account ID* pengguna. API *register* digunakan untuk mendaftarkan identitas pengguna berdasarkan *username* yang bersifat unik. API *register* tidak mengakses *chaincode*, melainkan mengakses *script file* yang memberi perintah Certificate Authority (CA) untuk mengeluarkan sertifikat digital yang terkait dengan pengguna tersebut. *Script file* untuk mendaftarkan identitas baru ditunjukkan pada Gambar 3.19. Fabric Gateway

API dijalankan dengan perintah `go run main.go` dan akan berjalan pada *port* 8082.

```
1  #!/bin/bash
2
3  export PATH=${PWD}/../../bin:${PWD}:${PATH}
4  export FABRIC_CFG_PATH=${PWD}/../../config/
5  export FABRIC_CA_CLIENT_HOME=${PWD}/../organizations/peerOrganizations/org1.example.com/
6
7  fabric-ca-client register --caname ca-org1 --id.name $1 --id.secret $2 --id.type client --
  tls.certfiles ${PWD}/../organizations/fabric-ca/org1/tls-cert.pem
8
9  fabric-ca-client enroll -u https://$1:$2@localhost:7054 --caname ca-org1 -M
  ${PWD}/../organizations/peerOrganizations/org1.example.com/users/$1@org1.example.com/msp --
  tls.certfiles ${PWD}/../organizations/fabric-ca/org1/tls-cert.pem
10 cp ${PWD}/../organizations/peerOrganizations/org1.example.com/msp/config.yaml
   ${PWD}/../organizations/peerOrganizations/org1.example.com/users/$1@org1.example.com/msp/config
```

Gambar 3.19. *Script File Register*

3.3.4.5 Pengembangan *Backend*

Backend yang dikembangkan berupa API untuk mengakses seluruh proses bisnis sistem *blockchain* MeetCoin dan juga basis data MySQL untuk manajemen data pengguna. Pada *backend*, pengguna didefinisikan dalam tipe data *struct* seperti pada Gambar 3.20. *Field AccountID* merupakan *field* yang akan menyimpan *account ID* milik pengguna setelah melakukan pendaftaran akun. Sebelum data pengguna disimpan di basis data, *password* pengguna akan dienkripsi terlebih dahulu untuk memastikan keamanan akun. Fungsi untuk mengenkripsi *password* pengguna ditunjukkan pada Gambar 3.21. Proses autentikasi pengguna diimplementasikan dengan menggunakan JSON Web Token (JWT). Pengguna perlu memasukkan *username* dan *password* yang sudah didaftarkan sebelumnya. Jika *username* dan *password* pengguna valid, *token* JWT akan di-generate. Fungsi untuk meng-generate *token* JWT ditunjukkan pada Gambar 3.22. *Token* JWT yang di-generate digunakan untuk mengautentikasi identitas pengguna ketika akan menjalankan proses bisnis sistem.

```
1  type User struct {
2      gorm.Model
3      Username string `gorm:"size:225;not null;unique" json:"username"`
4      Password string `gorm:"size:225;not null" json:"password"`
5      AccountID string `gorm:"size:255" json:"accountId"`
6      Email    string `gorm:"size:225" json:"email"`
7      Name     string `gorm:"size:225" json:"name"`
8      Phone    string `gorm:"size:255" json:"phone"`
9  }
```

Gambar 3.20. *Struct Pengguna*

```

1 func (u *User) BeforeSave() error {
2     hashedPassword, err := bcrypt.GenerateFromPassword([]byte(u.Password),
3         bcrypt.DefaultCost)
4     if err != nil {
5         return err
6     }
7     u.Password = string(hashedPassword)
8     u.Username = html.EscapeString(strings.TrimSpace(u.Username))
9
10    return nil
11 }

```

Gambar 3.21. Fungsi Enkripsi *Password*

```

1 func GenerateToken(userID uint, email string, accountId string) (string, error) {
2     token_lifespan, err := strconv.Atoi(os.Getenv("TOKEN_HOUR_LIFESPAN"))
3     if err != nil {
4         return "", err
5     }
6
7     claims := jwt.MapClaims{}
8     claims["authorized"] = true
9     claims["user_id"] = userID
10    claims["email"] = email
11    claims["account_id"] = accountId
12    claims["exp"] = time.Now().Add(time.Hour * time.Duration(token_lifespan)).Unix()
13    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
14
15    return token.SignedString([]byte(os.Getenv("API_SECRET")))
16 }

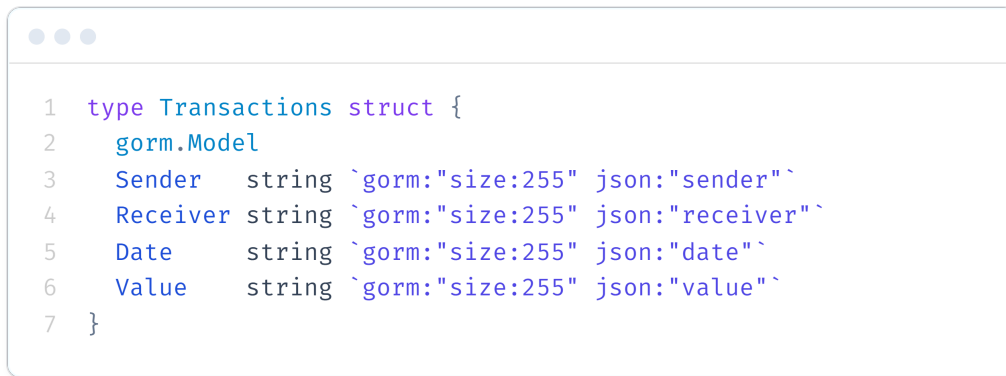
```

Gambar 3.22. Fungsi *Generate Token JWT*

Selain data pengguna, juga didefinisikan data transaksi dalam *struct* yang ditunjukkan pada Gambar 3.23. Data transaksi yang disimpan yaitu pengirim, penerima, waktu terjadinya transaksi, dan nilai transaksi. Data transaksi akan dibuat di basis data ketika transaksi yang terjadi di dalam jaringan *blockchain* berhasil diproses. Jika proses transaksi di dalam jaringan *blockchain* gagal, maka data transaksi tidak akan dibuat. Cuplikan kode untuk membuat data transaksi pada API transfer saldo ditunjukkan pada Gambar 3.24. *Backend* API dijalankan dengan perintah `go run main.go` dan akan berjalan pada *port* 8080.

3.3.5 Pengujian Sistem *Blockchain MeetCoin*

Tahapan pengujian sistem *blockchain* MeetCoin terdiri dari konfigurasi Hyperledger Caliper dan menjalankan pengujian Hyperledger Caliper. Berikut adalah penjelasan mengenai masing-masing tahapan pengujian sistem *blockchain* MeetCoin.



```

1  type Transactions struct {
2      gorm.Model
3      Sender   string `gorm:"size:255" json:"sender"`
4      Receiver  string `gorm:"size:255" json:"receiver"`
5      Date      string `gorm:"size:255" json:"date"`
6      Value     string `gorm:"size:255" json:"value"`
7  }

```

Gambar 3.23. *Struct Transaksi*



```

1  func Transfer(c *gin.Context) {
2
3      ...
4
5      err = blockchain.Transfer(username, receiverAccountID, convertedValue)
6      if err != nil {
7          c.JSON(http.StatusBadRequest, gin.H{"error": err.Error()})
8          return
9      }
10
11     currentTime := time.Now()
12     var transactionLog models.Transactions
13
14     transactionLog.Sender = username
15     transactionLog.Receiver = input.Receiver
16     transactionLog.Value = input.Value
17     transactionLog.Date = fmt.Sprintf(currentTime.Format("2006-01-02 15:04:05"))
18
19     transactionLog.SaveTransaction()
20
21     c.JSON(http.StatusOK, gin.H{"message": "ok"})
22 }

```

Gambar 3.24. Cuplikan Kode untuk Membuat Data Transaksi

3.3.5.1 Konfigurasi Hyperledger Caliper

Dalam melakukan pengujian sistem *blockchain* menggunakan Hyperledger Caliper diperlukan dua *file* konfigurasi dan satu modul, yaitu *network*, *benchmark*, dan modul *workload*. Konfigurasi *network* dibutuhkan untuk melakukan *binding* dengan sistem yang akan diuji. Pada konfigurasi *network*, didefinisikan mengenai sistem *blockchain* yang akan diuji, nama channel, nama *chaincode*, dan identitas pengguna yang akan melakukan pengujian. Konfigurasi *network* pada pengujian ini ditunjukkan pada Gambar 3.25. Pada gambar tersebut, Hyperledger Caliper akan melakukan pengujian pada sistem *blockchain* Fabric yang memiliki *channel* dengan nama *mychannel*, *chaincode* dengan nama *basic*,

dan identitas pengguna dengan nama User1.

```
1  name: Caliper test
2  version: "2.0.0"
3
4  caliper:
5    blockchain: fabric
6
7  channels:
8    - channelName: mychannel
9      contracts:
10        - id: basic
11
12  organizations:
13    - mspid: Org1MSP
14      identities:
15        certificates:
16          - name: 'User1'
17            clientPrivateKey:
18              path: '../hlf-mtcnv2/test-
19network/organizations/peerOrganizations/org1.example.com/users/User1@org1.example.com/msp/keyst
19      clientSignedCert:
20              path: '../hlf-mtcnv2/test-
21network/organizations/peerOrganizations/org1.example.com/users/User1@org1.example.com/msp/signc
21      connectionProfile:
22              path: '../hlf-mtcnv2/test-network/organizations/peerOrganizations/org1.example.com/connec
22org1.yaml'
```

Gambar 3.25. Konfigurasi *Network* pada Hyperledger Caliper

Konfigurasi *benchmark* mendefinisikan skenario dan beban kerja yang akan diberikan ke sistem selama pengujian. Skenario pengujian yang dilakukan pada tugas akhir ini dijelaskan pada bagian 3.2.3. Cuplikan konfigurasi *benchmark* ditunjukkan pada Gambar 3.26. Konfigurasi pada gambar tersebut akan memberikan beban kerja ke sistem pada fungsi *Mint* sebanyak 50, 100, dan 150 transaksi per detik. Selain memberikan beban kerja ke sistem, *benchmark* juga akan memonitor sumber daya yang digunakan selama pengujian.

Modul *workload* merupakan modul yang bertanggung jawab untuk membuat dan mengirimkan transaksi ke sistem selama pengujian. Terdapat tiga modul *workload* yang masing-masing modul mewakili jenis transaksi yang akan diuji, yaitu modul *mint* untuk pengisian saldo, modul *balance* untuk pengecekan saldo, dan modul *transfer* untuk transfer saldo. Cuplikan kode untuk masing-masing modul *workload* ditunjukkan pada Gambar 3.27, Gambar 3.28, dan Gambar 3.29. Gambar 3.27 menunjukkan bahwa fungsi *Mint* yang ada di *chaincode* akan dieksekusi oleh User1 dengan parameter *amount* sebanyak 1000. Gambar 3.28 menunjukkan fungsi *ClientAccountBalance* akan eksekusi oleh User1 dan argumen *readOnly* bernilai *true* karena fungsi tersebut merupakan fungsi *read*. Gambar 3.29 juga menunjukkan bahwa fungsi *Transfer* akan dieksekusi oleh User1 dengan parameter *receiver* yang berisi *account ID* dari penerima dan *amount* sebanyak 1000.

```

1  test:
2    name: basic-contract-benchmark
3    description: test benchmark
4    workers:
5      number: 5
6    rounds:
7      - label: Mint with 50 TPS
8        description: Mint some coins
9        txNumber: 1000
10       rateControl:
11         type: fixed-rate
12         opts:
13           tps: 50
14       workload:
15         module: workload/mint.js
16         arguments:
17           contractId: basic
18      - label: Mint with 100 TPS
19        description: Mint some coins
20        txNumber: 1000
21        rateControl:
22          type: fixed-rate
23          opts:
24            tps: 100
25        workload:
26          module: workload/mint.js
27          arguments:
28            contractId: basic
29      - label: Mint with 150 TPS
30        description: Mint some coins
31        txNumber: 1000
32        rateControl:
33          type: fixed-rate
34          opts:
35            tps: 150
36        workload:
37          module: workload/mint.js
38          arguments:
39            contractId: basic
40
41    monitors:
42      resource:
43        - module: docker
44        options:
45          interval: 1
46          cpuUsageNormalization: true
47      containers:
48        - all

```

Gambar 3.26. Konfigurasi *Benchmark* pada Hyperledger Caliper

3.3.5.2 Menjalankan Pengujian Hyperledger Caliper

Sebelum dapat menjalankan pengujian, Hyperledger Caliper diinstalasi terlebih dahulu dengan perintah `npm install --only=prod @hyperledger/caliper-cli@0.6.0`. Setelah instalasi Hyperledger Caliper berhasil, selanjutnya adalah melakukan *binding* Hyperledger Caliper dengan jaringan *blockchain* MeetCoin. Proses *binding* dilakukan dengan perintah `npx caliper bind --caliper-bind-sut fabric:fabric gateway`. Proses *binding* akan memberitahu Hyperledger Caliper kerangka kerja atau platform *blockchain* yang ditargetkan dan versi SDK kerangka kerja yang digunakan untuk pengujian.

```

1 class MintWorkload extends WorkloadModuleBase {
2   constructor() {
3     super();
4   }
5
6   // Initialize the workload with the parameters needed
7   async initializeWorkloadModule(
8     workerIndex,
9     totalWorkers,
10    roundIndex,
11    roundArguments,
12    sutAdapter,
13    sutContext,
14  ) {
15    await super.initializeWorkloadModule(
16      workerIndex,
17      totalWorkers,
18      roundIndex,
19      roundArguments,
20      sutAdapter,
21      sutContext,
22    );
23
24    console.log(`Worker ${this.workerIndex}: Initializing Mint Test`);
25  }
26
27  async submitTransaction() {
28    // Construct the transaction request for the Mint function
29    const args = {
30      contractId: this.roundArguments.contractId,
31      contractFunction: "Mint",
32      invokerIdentity: "User1",
33      contractArguments: ["1000"],
34      readOnly: false,
35    };
36
37    await this.sutAdapter.sendRequests(args);
38  }
39 }
40
41 function createWorkloadModule() {
42   return new MintWorkload();
43 }

```

Gambar 3.27. Cuplikan Kode Modul *Workload Mint*

Pengujian dijalankan dengan perintah `npx caliper launch manager -caliper-workspace ./ -caliper-networkconfig networks/networkConfig.yaml -caliper-benchconfig benchmarks/mtcnv2Benchmark.yaml -caliper-flow-only-test`. Perintah caliper launch manager akan memulai proses jalannya pengujian dengan mengoordinasikan konfigurasi dari *network*, *benchmark*, dan modul *workload*. Argumen `-caliper-workspace ./ -caliper-networkconfig networks/networkConfig.yaml -caliper-benchconfig benchmarks/mtcnv2Benchmark.yaml` mendefinisikan *path* dari *workspace*, konfigurasi *network*, dan konfigurasi *benchmark* yang digunakan dalam pengujian. Argumen `-caliper-flow-only-test` merupakan perintah tambahan yang menginstruksikan Hyperledger Caliper untuk hanya melakukan fase pengujian. Pengujian menggunakan

```

1 class BalanceWorkload extends WorkloadModuleBase {
2   constructor() {
3     super();
4   }
5
6   // Initialize the workload with the parameters needed
7   async initializeWorkloadModule(
8     workerIndex,
9     totalWorkers,
10    roundIndex,
11    roundArguments,
12    sutAdapter,
13    sutContext,
14  ) {
15    await super.initializeWorkloadModule(
16      workerIndex,
17      totalWorkers,
18      roundIndex,
19      roundArguments,
20      sutAdapter,
21      sutContext,
22    );
23
24    console.log(`Worker ${this.workerIndex}: Initializing Balance Test`);
25  }
26
27  async submitTransaction() {
28    // Construct the transaction request for the Balance function
29    const args = {
30      contractId: this.roundArguments.contractId,
31      contractFunction: "ClientAccountBalance",
32      invokerIdentity: "User1",
33      readOnly: true,
34    };
35
36    await this.sutAdapter.sendRequests(args);
37  }
38 }
39
40 function createWorkloadModule() {
41   return new BalanceWorkload();
42 }

```

Gambar 3.28. Cuplikan Kode Modul *Workload Balance*

Hyperledger Caliper akan menghasilkan *test report* dalam bentuk *file* HTML.

```

1 class TransferWorkload extends WorkloadModuleBase {
2   constructor() {
3     super();
4   }
5
6   // Initialize the workload with the parameters needed
7   async initializeWorkloadModule(
8     workerIndex,
9     totalWorkers,
10    roundIndex,
11    roundArguments,
12    sutAdapter,
13    sutContext,
14  ) {
15    await super.initializeWorkloadModule(
16      workerIndex,
17      totalWorkers,
18      roundIndex,
19      roundArguments,
20      sutAdapter,
21      sutContext,
22    );
23
24    console.log(`Worker ${this.workerIndex}: Initializing Transfer Test`);
25  }
26
27  async submitTransaction() {
28    const receiver =
29      "eDUwOT06Q049Y2FsaXBlcjIsT1U9Y2xpZW50LE89SHlwZXJsZWRnZXIsU1Q9Tm9ydGggQ2Fyb2xpbmEsQz1VUzo6Q
30      049ZmFicmljLWNhLXNlcnZlcixPVT1GYWJyaWMsTz1IeXBldmVlcixTVD10b3J0aCBDYXJvbGluYSxDPVVT";
31
32    // Construct the transaction request for the Transfer function
33    const args = {
34      contractId: this.roundArguments.contractId,
35      contractFunction: "Transfer",
36      invokerIdentity: "User1",
37      contractArguments: [receiver, "1000"],
38      readOnly: false,
39    };
40
41    await this.sutAdapter.sendRequests(args);
42  }
43
44  async cleanupWorkloadModule() {
45    console.log(`Worker ${this.workerIndex}: Transfer test completed`);
46  }
47
48  function createWorkloadModule() {
49    return new TransferWorkload();
50  }

```

Gambar 3.29. Cuplikan Kode Modul *Workload Transfer*

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Pengembangan Sistem *Blockchain* MeetCoin

Sistem *blockchain* MeetCoin yang dirancang dan dikembangkan menggunakan kerangka kerja Hyperledger Fabric memiliki arsitektur seperti yang ditunjukkan pada Gambar 3.9. Arsitektur sistem *blockchain* MeetCoin yang dikembangkan memiliki tiga komponen utama, yaitu jaringan Fabric, Fabric Gateway, dan *backend*. Fungsi dari setiap komponen arsitektur pada sistem *blockchain* MeetCoin yang dikembangkan dijelaskan pada Tabel 4.1.

Tabel 4.1. Komponen Arsitektur Sistem *Blockchain* MeetCoin

Komponen Arsitektur	Fungsi
Jaringan Fabric	Memproses transaksi dengan aman dan transparan untuk menjaga integritas transaksi aset digital dalam sistem
Fabric Gateway	Menjembatani untuk pengiriman transaksi dan eksekusi <i>chaincode</i> dari aplikasi klien ke jaringan Fabric
<i>Backend</i>	Manajemen data pengguna, autentikasi akun pengguna, dan mencatat salinan data transaksi

4.1.1 Jaringan Fabric

Jaringan Fabric dikonfigurasi dengan dua *peer node*, *ordering service node*, dan Certificate Authority (CA) untuk setiap *node*. Setiap *node* tersebut terhubung secara *peer-to-peer* dan tergabung dalam sebuah *channel*. *Channel* dalam jaringan Fabric bertujuan untuk meningkatkan privasi transaksi antar *node* yang tergabung di dalamnya sehingga transaksi tersebut tidak dapat diakses oleh *node* di luar *channel*. Konfigurasi dua *peer node* akan membantu jaringan untuk mencegah terjadinya *single point of failure* sehingga jaringan masih dapat bertahan ketika salah satu *node* mengalami kegagalan. Hal tersebut juga membuat jaringan memiliki keandalan yang lebih baik dibandingkan dengan sistem *blockchain* MeetCoin sebelumnya yang hanya memiliki satu *peer node*. Selain itu, konfigurasi dua *peer node* membuat buku besar terdistribusi ke *peer node* yang lain untuk memastikan konsistensi data serta menjaga integritas data di jaringan Fabric.

Implementasi CA membantu jaringan untuk mengelola identitas pengguna berupa sertifikat digital dan pasangan kunci privat dan publik yang unik untuk setiap pengguna. Berbeda dengan sistem *blockchain* MeetCoin sebelumnya yang tidak memanfaatkan penggunaan CA dan identitas pengguna di dalam jaringan hanya didasarkan pada *user-*

name dan *password* yang disimpan dalam buku besar. Dengan CA, setiap pengguna di dalam jaringan yang ingin melakukan transaksi harus terdaftar di CA dan memiliki sertifikat digital yang sah. Hal tersebut memberikan tingkat keamanan yang lebih tinggi dibandingkan dengan menyimpan identitas pengguna berupa *username* dan *password* di buku besar. Penyimpanan identitas berupa *username* dan *password* di buku besar kurang aman karena data sensitif pengguna disimpan secara terbuka tanpa ada proses verifikasi yang ketat sehingga rentan terhadap serangan *hacking* dan penyalahgunaan data serta berpotensi untuk masuknya pengguna palsu atau tidak sah ke dalam jaringan.

Mekanisme konsensus yang digunakan dalam sistem *blockchain* MeetCoin yaitu Raft. Dari perbandingan mekanisme konsensus yang ditawarkan oleh Hyperledger Fabric, Raft memiliki kinerja yang paling cepat dibandingkan dengan mekanisme konsensus lainnya, yaitu Kafka dan PBFT. Pada Raft, transaksi hanya dapat diurutkan dan dimasukkan ke dalam blok oleh *leader* sehingga memastikan konsistensi pengurutan serta pembuatan blok dan mencegah konflik yang dapat terjadi ketika terdapat lebih dari satu *node* yang mencoba hal tersebut. Selain itu, pemilihan *leader* yang cepat dan dinamis dapat meminimalisir *downtime* ketika terdapat salah satu *node* yang bertugas sebagai *orderer* mengalami kegagalan.

Untuk penyimpanan data aset di dalam jaringan Fabric, *state database* yang digunakan yaitu LevelDB. Selain karena merupakan *state database* bawaan Hyperledger Fabric yang memiliki penggunaan sumber daya yang rendah dan memiliki kinerja yang lebih tinggi dalam melakukan *query* dibandingkan dengan CouchDB, LevelDB dipilih karena data aset yang disimpan di buku besar pada aplikasi *blockchain* MeetCoin hanya sederhana. Data aset yang disimpan di buku besar pada aplikasi *blockchain* MeetCoin berupa *account ID* dan *value* saldo yang dimiliki pengguna. Besarnya ukuran data yang disimpan di buku besar berdampak terhadap kinerja jaringan dalam memproses transaksi. Semakin besar ukuran data yang disimpan, semakin besar juga beban yang harus ditangani oleh jaringan. Hal tersebut dapat membuat penurunan kinerja jaringan dari sisi kecepatan dan efisiensi dalam memproses transaksi.

Kebijakan pengesahan transaksi yang diterapkan pada sistem *blockchain* MeetCoin melibatkan kedua *peer node*. Transaksi harus disahkan oleh kedua *peer node* sebelum dapat diurutkan dan dimasukkan ke dalam blok oleh *ordering service node* serta di-*commit* ke buku besar. Hal tersebut bertujuan untuk mencegah salah satu *peer node* dapat mengirimkan dan mengesahkan transaksi yang tidak sah dalam jaringan. Dengan kebijakan pengesahan transaksi yang mengharuskan transaksi disetujui oleh kedua *peer node*, proses validasi transaksi menjadi lebih ketat dan aman serta mengurangi risiko kesalahan atau manipulasi data transaksi.

Chaincode berfungsi sebagai logika bisnis yang menjalankan proses transaksi di jaringan Fabric. Proses bisnis sistem *blockchain* MeetCoin yang terdapat pada *chaincode*

meliputi pengisian saldo, pengecekan saldo, penarikan saldo, dan transfer saldo. Keempat proses bisnis tersebut merupakan fungsi yang bersifat transaksional sehingga setiap proses transaksi yang dijalankan akan mencatat perubahan pada buku besar. Selain itu, *chaincode* memastikan bahwa setiap transaksi yang dilakukan sesuai dengan aturan yang telah ditetapkan, seperti memverifikasi bahwa saldo cukup sebelum melakukan penarikan atau transfer saldo.

4.1.2 Fabric Gateway

Fabric Gateway membantu menyederhanakan interaksi *backend* dengan jaringan Fabric. Dengan Fabric Gateway, *backend* hanya perlu terintegrasi dengan API yang disediakan oleh Fabric Gateway alih-alih melakukan konfigurasi melalui CLI untuk berinteraksi dengan setiap *node* di jaringan Fabric. Selain itu, Fabric Gateway dapat mengeksekusi *chaincode* di jaringan Fabric sehingga *backend* tidak perlu berkomunikasi langsung dengan *peer node* untuk melakukan eksekusi transaksi. Fabric Gateway juga berperan untuk melakukan validasi identitas pengguna sebelum melakukan transaksi ke jaringan Fabric. Setiap transaksi yang dikirim melalui Fabric Gateway harus disertai dengan sertifikat digital yang dikeluarkan CA di jaringan Fabric. Sertifikat digital tersebut digunakan untuk mengidentifikasi dan mengautentikasi pengguna yang melakukan transaksi serta memastikan bahwa hanya pengguna yang memiliki sertifikat yang sah yang dapat mengakses jaringan Fabric.

4.1.3 Backend

Backend merupakan aplikasi yang berinteraksi dengan pengguna untuk mengirimkan transaksi ke jaringan Fabric. Selain itu, *backend* juga bertanggung jawab terhadap manajemen data pengguna dan proses bisnis non-transaksional. *Backend* memiliki basis data MySQL tersendiri yang bertujuan untuk menyimpan data pengguna yang tidak perlu disimpan di dalam buku besar. Pada *backend* sistem *blockchain* MeetCoin, data yang disimpan di basis data meliputi *username*, *password*, nama, *email*, nomor telepon, dan *account ID*. *Account ID* tersebut terhubung dengan *account ID* pengguna yang disimpan dalam buku besar di dalam jaringan Fabric. Dengan adanya pembagian data yang disimpan di basis data *backend* dan buku besar jaringan Fabric, sistem memiliki tingkat keamanan data dan kinerja yang lebih baik dibandingkan dengan sistem *blockchain* MeetCoin sebelumnya. Sistem *blockchain* MeetCoin sebelumnya menyimpan semua data pengguna di buku besar yang menyebabkan risiko keamanan karena informasi sensitif pengguna seperti *username* dan *password* dapat diakses oleh pihak yang tidak berwenang dan berisiko terjadinya penyalahgunaan data. Dari sisi kinerja, penyimpanan data yang berukuran besar di buku besar memiliki dampak negatif pada kinerja jaringan Fabric, seperti memperlambat proses transaksi dan meningkatkan waktu respon jaringan.

Proses bisnis non-transaksional yang terdapat di *backend* meliputi pendaftaran akun, autentikasi akun, melihat profil akun, dan melihat riwayat transaksi. Proses bisnis tersebut hanya akan berjalan di *backend* sehingga tidak memengaruhi kinerja jaringan Fabric. Autentikasi akun pada *backend* mengimplementasikan JSON Web Token (JWT). Dengan menggunakan JWT, setiap kali pengguna berhasil masuk, *backend* akan *generate token* yang berisi informasi pengguna yang telah terverifikasi. *Token* tersebut digunakan untuk mengakses proses bisnis sistem *blockchain* MeetCoin tanpa perlu melakukan autentikasi ulang sehingga mengurangi beban pada *server*. Selain itu, JWT juga meminimalisir risiko penyalahgunaan data karena *token* dienkripsi dan ditandatangani untuk memastikan bahwa data *token* tidak dapat dimanipulasi oleh pihak yang tidak berwenang. *Backend* juga bertugas mencatat salinan transaksi yang terjadi di dalam jaringan Fabric. Ketika transaksi berhasil diproses oleh jaringan Fabric, *backend* akan membuat data transaksi di basis data untuk memastikan adanya salinan transaksi di luar jaringan untuk meningkatkan ketersediaan data.

4.2 Hasil Evaluasi Kinerja Sistem *Blockchain* MeetCoin

4.2.1 Pengujian Pengaruh Jumlah *Worker* terhadap Kinerja Sistem

Pada pengujian ini, variabel jumlah *worker* ditetapkan ke tiga nilai yang berbeda untuk melihat dampaknya terhadap kinerja sistem. Berikut adalah pembahasan mengenai hasil pengujian pengaruh jumlah *worker* pada setiap jenis transaksi.

1. Pengisian Saldo

Setiap konfigurasi jumlah *worker* memiliki tingkat keberhasilan sebesar 100% yang menandakan sistem mampu menangani beban transaksi yang diberikan tanpa ada kendala. *Send rate* mengalami peningkatan dari 50,3 TPS ke 50,7 TPS dan *throughput* juga meningkat dari 50,2 TPS ke 50,7 TPS seiring dengan bertambahnya jumlah *worker* yang menandakan sistem dapat memproses transaksi dengan kecepatan yang cukup stabil. *Latency* menunjukkan konfigurasi dengan 10 *worker* memiliki *latency* terendah dengan waktu 0,05 detik dibandingkan dengan 5 dan 15 *worker* yang menunjukkan konfigurasi 10 *worker* memiliki *response time* paling cepat. Penggunaan CPU mengalami peningkatan dari 1,28% ke 1,39% seiring dengan bertambahnya jumlah *worker*. Penggunaan memori juga mengalami peningkatan dari 129 MB ke 133 MB. Tabel 4.2 menunjukkan hasil pengujian pada pengisian saldo.

2. Pengecekan Saldo

Sama seperti pada pengisian saldo, tingkat keberhasilan mencapai 100% di setiap konfigurasi jumlah *worker*. *Send rate* dan *throughput* memiliki laju yang cenderung seimbang seiring dengan bertambahnya jumlah *worker*. *Latency* yang dihasilkan sangat rendah dan stabil di 0,01 detik di semua konfigurasi jumlah *worker*

yang menunjukkan bahwa proses pengecekan saldo sangat ringan karena merupakan operasi *read*. Penggunaan CPU juga relatif rendah di rentang 0,51% hingga 0,60% yang menunjukkan bahwa pengecekan saldo tidak membutuhkan sumber daya CPU yang besar. Penggunaan memori mengalami peningkatan seiring dengan bertambahnya jumlah *worker* dari 153 MB ke 163 MB. Tabel 4.3 menunjukkan hasil pengujian pada pengecekan saldo.

3. Transfer Saldo

Sama seperti dua jenis transaksi sebelumnya, tingkat keberhasilan transfer saldo mencapai 100% di setiap konfigurasi jumlah *worker*. *Send rate* mengalami sedikit peningkatan dari 50,3 TPS ke 50,7 TPS dan *throughput* juga mengikuti pola yang sama dari 50,1 TPS ke 50,6 TPS. *Latency* dengan 10 *worker* memiliki *latency* yang paling rendah di 0,05 detik, sedangkan 5 dan 15 *worker* memiliki *latency* yang sedikit lebih tinggi di 0,10 dan 0,11 detik. Seiring dengan bertambahnya jumlah *worker*, penggunaan CPU meningkat dari 1,59% hingga 1,91% dan penggunaan memori juga sedikit meningkat dari 147 MB ke 150 MB. Tabel 4.4 menunjukkan hasil pengujian pada transfer saldo.

Tabel 4.2. Hasil Pengujian Pengaruh Jumlah *Worker* pada Pengisian Saldo

<i>Worker</i>	Transaksi	TPS	<i>Success Rate</i>	<i>Send Rate</i>	<i>Latency</i> (detik)	<i>Throughput</i>	CPU (%)	Memory (MB)
5	1000	50	100%	50,3	0,08	50,2	1,28	129
10	1000	50	100%	50,5	0,05	50,4	1,50	129
15	1000	50	100%	50,7	0,11	50,7	1,39	133

Tabel 4.3. Hasil Pengujian Pengaruh Jumlah *Worker* pada Pengecekan Saldo

<i>Worker</i>	Transaksi	TPS	<i>Success Rate</i>	<i>Send Rate</i>	<i>Latency</i> (detik)	<i>Throughput</i>	CPU (%)	Memory (MB)
5	1000	50	100%	50,3	0,01	50,2	0,51	153
10	1000	50	100%	50,5	0,01	50,5	0,60	160
15	1000	50	100%	50,7	0,01	50,7	0,60	163

Tabel 4.4. Hasil Pengujian Pengaruh Jumlah *Worker* pada Transfer Saldo

<i>Worker</i>	Transaksi	TPS	<i>Success Rate</i>	<i>Send Rate</i>	<i>Latency</i> (detik)	<i>Throughput</i>	CPU (%)	Memory (MB)
---------------	-----------	-----	---------------------	------------------	------------------------	-------------------	---------	-------------

5	1000	50	100%	50,3	0,10	50,1	1,59	147
10	1000	50	100%	50,5	0,05	50,4	1,61	148
15	1000	50	100%	50,7	0,11	50,6	1,91	150

4.2.2 Pengujian Pengaruh Jumlah Transaksi terhadap Kinerja Sistem

Pada pengujian ini, variabel jumlah transaksi ditetapkan ke tiga nilai yang berbeda untuk melihat dampaknya terhadap kinerja sistem. Berikut adalah pembahasan mengenai hasil pengujian pengaruh jumlah transaksi pada setiap jenis transaksi.

1. Pengisian Saldo

Pengujian pada pengisian saldo memiliki tingkat keberhasilan sebesar 100% yang menandakan bahwa sistem dapat menangani beban kerja yang diberikan tanpa adanya kendala. *Send rate* dan *throughput* memiliki laju yang stabil di 50 TPS. *Latency* juga stabil di 0,08 detik dan mengalami peningkatan yang sedikit ke 0,09 detik ketika memproses 3000 transaksi. Hal tersebut menunjukkan bahwa sistem memiliki kinerja yang stabil meskipun mengalami peningkatan beban kerja. Penggunaan CPU mengalami peningkatan dari 1,28% ke 1,57% seiring dengan bertambahnya jumlah transaksi. Penggunaan memori juga mengalami peningkatan dari 129 MB pada saat jumlah transaksi sebanyak 1000 ke 174 MB pada saat jumlah transaksi sebanyak 3000, Tabel 4.5 menunjukkan hasil pengujian pada pengisian saldo.

2. Pengecekan Saldo

Pengujian pada pengecekan saldo juga memiliki tingkat keberhasilan yang sama seperti pada pengisian saldo, yaitu 100%, di setiap konfigurasi jumlah transaksi. *Send rate* dan *throughput* memiliki laju yang cenderung seimbang seiring dengan bertambahnya jumlah transaksi dan stabil di 50 TPS. Karena merupakan fungsi *read*, *latency* yang dihasilkan sangat rendah dan stabil di 0,01 detik. Penggunaan CPU relatif rendah di rentang 0,51% hingga 0,63% yang menunjukkan bahwa pengecekan saldo tidak membutuhkan sumber daya CPU yang besar. Penggunaan memori mengalami peningkatan dari 153 MB ke 177 MB seiring dengan bertambahnya jumlah transaksi. Tabel 4.6 menunjukkan hasil pengujian pada pengecekan saldo.

3. Transfer Saldo

Tingkat keberhasilan pengujian pada transfer saldo juga mencapai 100% sama seperti pada pengisian saldo dan pengecekan saldo. Sama seperti dua pengujian sebelumnya, *send rate* dan *throughput* juga memiliki laju yang cenderung seimbang seiring dengan bertambahnya jumlah transaksi dan stabil di 50 TPS. *Latency* ketika jumlah transaksi 2000 dan 3000 mengalami sedikit penurunan dari 0,10 detik ke

0,08 yang menunjukkan bahwa sistem dapat memproses transaksi dengan lebih cepat ketika beban kerja meningkat. Penggunaan CPU mengalami peningkatan dari 1,59% ke 1,72% ketika jumlah transaksi menjadi 2000 dan mengalami penurunan dari 1,72% ke 1,55% ketika jumlah transaksi menjadi 3000, Penggunaan memori juga meningkat dari 147 MB ke 193 MB seiring dengan bertambahnya jumlah transaksi. Tabel 4.7 menunjukkan hasil pengujian pada pengecekan saldo.

Tabel 4.5. Hasil Pengujian Pengaruh Jumlah Transaksi pada Pengisian Saldo

<i>Worker</i>	Transaksi	TPS	<i>Success Rate</i>	<i>Send Rate</i>	<i>Latency</i> (detik)	<i>Throughput</i>	CPU (%)	Memory (MB)
5	1000	50	100%	50,3	0,08	50,2	1,28	129
5	2000	50	100%	50,1	0,08	50,1	1,41	168
5	3000	50	100%	50,1	0,09	50,1	1,57	174

Tabel 4.6. Hasil Pengujian Pengaruh Jumlah Transaksi pada Pengecekan Saldo

<i>Worker</i>	Transaksi	TPS	<i>Success Rate</i>	<i>Send Rate</i>	<i>Latency</i> (detik)	<i>Throughput</i>	CPU (%)	Memory (MB)
5	1000	50	100%	50,3	0,01	50,2	0,51	153
5	2000	50	100%	50,1	0,01	50,1	0,63	177
5	3000	50	100%	50,1	0,01	50,1	0,62	177

Tabel 4.7. Hasil Pengujian Pengaruh Jumlah Transaksi pada Transfer Saldo

<i>Worker</i>	Transaksi	TPS	<i>Success Rate</i>	<i>Send Rate</i>	<i>Latency</i> (detik)	<i>Throughput</i>	CPU (%)	Memory (MB)
5	1000	50	100%	50,3	0,10	50,1	1,59	147
5	2000	50	100%	50,1	0,08	50,1	1,72	158
5	3000	50	100%	50,1	0,08	50,1	1,55	193

4.2.3 Pengujian Pengaruh Laju Transaksi terhadap Kinerja Sistem

Pada pengujian ini, variabel laju transaksi per detik ditetapkan ke tiga nilai yang berbeda untuk melihat dampaknya terhadap kinerja sistem. Berikut adalah pembahasan mengenai hasil pengujian pengaruh laju transaksi pada setiap jenis transaksi.

1. Pengisian Saldo

Pada pengujian pengisian saldo, tingkat keberhasilan dari setiap konfigurasi laju transaksi mencapai 100% yang menandakan sistem mampu menangani beban kerja yang diberikan tanpa mengalami kegagalan. *Send rate* dan *throughput* meningkat seiring dengan peningkatan laju transaksi dari 50 TPS hingga 150 TPS dan cukup seimbang yang menunjukkan bahwa sistem mampu menangani lebih banyak transaksi per detik dan tidak mengalami penurunan kinerja. *Latency* menurun dari 0,08 detik pada 50 TPS ke 0,05 detik pada 100 TPS dan tetap stabil hingga 150 TPS. Penggunaan CPU menunjukkan peningkatan yang signifikan dari 1,28% hingga 2,57% ketika laju transaksi meningkat. Penggunaan memori juga mengalami peningkatan dari 129 MB hingga 183 MB seiring meningkatnya laju transaksi dari 50 TPS ke 150 TPS. Tabel 4.8 menunjukkan hasil pengujian pada pengecekan saldo.

2. Pengecekan Saldo

Tingkat keberhasilan pada pengecekan saldo juga mencapai 100% sama seperti pada pengisian saldo. *Send rate* dan *throughput* pada 50 TPS hingga 150 TPS meningkat secara cukup seimbang. *Latency* tetap rendah di 0,01 detik pada 50 TPS hingga 150 TPS karena pengecekan saldo merupakan fungsi *read* yang sangat ringan. Penggunaan CPU pada setiap peningkatan laju transaksi relatif rendah dalam rentang 0,51% hingga 0,95% yang menandakan fungsi *read* pengecekan saldo tidak memerlukan sumber daya CPU yang besar. Namun, penggunaan memori mengalami peningkatan dari 153 MB hingga 180 MB seiring meningkatnya laju transaksi. Tabel 4.9 menunjukkan hasil pengujian pada pengecekan saldo.

3. Transfer Saldo

Sama seperti dua jenis transaksi sebelumnya, tingkat keberhasilan transfer saldo mencapai 100% di setiap konfigurasi laju transaksi. *Send rate* dan *throughput* meningkat dengan cukup seimbang seiring dengan peningkatan laju transaksi dari 50 TPS hingga 150 TPS. *Latency* menurun dari 0,10 detik pada 50 TPS ke 0,06 detik pada 100 TPS dan kemudian meningkat ke 0,05 detik pada 150 TPS. Penurunan *latency* menunjukkan bahwa peningkatan laju transaksi dapat dimanfaatkan dengan efisiensi yang lebih baik sehingga sistem dapat memproses transaksi dengan lebih cepat. Penggunaan CPU meningkat dari 1,59% ke 2,46% ketika laju transaksi juga meningkat yang menunjukkan bahwa laju transaksi yang meningkat membutuhkan sumber daya yang lebih besar. Penggunaan memori juga sedikit mengalami peningkatan dari 147 MB hingga 159 MB seiring meningkatnya laju transaksi. Tabel 4.10 menunjukkan hasil pengujian pada pengecekan saldo.

Tabel 4.8. Hasil Pengujian Pengaruh Laju Transaksi pada Pengisian Saldo

<i>Worker</i>	Transaksi	TPS	<i>Success Rate</i>	<i>Send Rate</i>	<i>Latency</i> (detik)	<i>Throughput</i>	CPU (%)	<i>Memory</i> (MB)
5	1000	50	100%	50,3	0,08	50,2	1,28	129
5	1000	100	100%	100,6	0,05	100,3	1,67	182
5	1000	150	100%	150,9	0,05	150,2	2,57	183

Tabel 4.9. Hasil Pengujian Pengaruh Laju Transaksi pada Pengecekan Saldo

<i>Worker</i>	Transaksi	TPS	<i>Success Rate</i>	<i>Send Rate</i>	<i>Latency</i> (detik)	<i>Throughput</i>	CPU (%)	<i>Memory</i> (MB)
5	1000	50	100%	50,3	0,01	50,2	0,51	153
5	1000	100	100%	100,6	0,01	100,5	0,71	180
5	1000	150	100%	150,9	0,01	150,7	0,95	179

Tabel 4.10. Hasil Pengujian Pengaruh Laju Transaksi pada Transfer Saldo

<i>Worker</i>	Transaksi	TPS	<i>Success Rate</i>	<i>Send Rate</i>	<i>Latency</i> (detik)	<i>Throughput</i>	CPU (%)	<i>Memory</i> (MB)
5	1000	50	100%	50,3	0,10	50,1	1,59	147
5	1000	100	100%	100,5	0,06	100,2	1,83	160
5	1000	150	100%	150,8	0,05	150,2	2,46	159

4.3 Perbandingan Hasil Penelitian dengan Hasil Terdahulu

Sistem *blockchain* MeetCoin yang dikembangkan pada tugas akhir ini memiliki beberapa perbedaan dengan sistem *blockchain* MeetCoin terdahulu. Perbedaan tersebut meliputi teknologi yang digunakan, arsitektur, konfigurasi, bahasa yang digunakan pada *chaincode*, mekanisme penyimpanan data aset, manajemen identitas, dan keamanan sistem *blockchain*. Penjelasan mengenai perbedaan sistem *blockchain* MeetCoin yang dikembangkan pada tugas akhir ini dengan sistem *blockchain* MeetCoin terdahulu dijelaskan pada Tabel 4.11.

Tabel 4.11. Perbandingan Sistem *Blockchain* MeetCoin yang Dikembangkan dengan Terdahulu

Perbedaan	Sistem <i>Blockchain</i> MeetCoin yang Dikembangkan	Sistem <i>Blockchain</i> MeetCoin Terdahulu
Teknologi yang Digunakan	Hyperledger Fabric versi 2.5.9 LTS	Hyperledger Fabric versi 1.2.1 dan Hyperledger Composer
Arsitektur	Jaringan Fabric, Fabric Gateway, dan <i>backend</i>	Jaringan Fabric dan Composer
Konfigurasi Jaringan	• Terdiri dari dua <i>peer node</i> • Mekanisme konsensus Raft • <i>State database</i> LevelDB	• Terdiri dari satu <i>peer node</i> • Mekanisme konsensus tidak disebutkan • <i>State database</i> CouchDB
Bahasa <i>Chain-code</i>	Go	Javascript (Node.js)
Data Aset	Data aset yang disimpan di dalam buku besar berupa <i>account ID</i> dan <i>value</i> saldo pengguna, sedangkan data pengguna disimpan di basis data <i>backend</i>	Data aset yang disimpan di dalam buku besar berupa data pengguna yang meliputi <i>username</i> , <i>password</i> , nama, <i>email</i> , nomor telepon, dan saldo pengguna
Manajemen Identitas	Menggunakan Certificate Authority (CA) untuk identitas pengguna di dalam jaringan Fabric dan basis data <i>backend</i> untuk manajemen data pengguna	Berbasis <i>username</i> dan <i>password</i> yang disimpan sebagai data aset di buku besar
Keamanan Sistem	CA untuk izin akses ke jaringan Fabric berbasis sertifikat digital dan autentikasi berbasis JWT untuk izin akses proses bisnis di <i>backend</i>	Autentikasi berbasis <i>username</i> dan <i>password</i>

Berdasarkan penjelasan pada Tabel 4.11, sistem *blockchain* MeetCoin yang dikembangkan pada tugas akhir ini memiliki kelebihan dibandingkan dengan sistem *blockchain* MeetCoin terdahulu dalam hal keandalan, keamanan, dan kinerja. Sistem menjadi

lebih andal karena memiliki konfigurasi dua *peer node* yang akan mencegah terjadinya *single point of failure* dan menjaga konsistensi serta integritas data karena buku besar didistribusikan ke setiap *peer node*. Dari sisi keamanan, implementasi CA untuk izin akses ke jaringan dan JWT untuk izin akses proses bisnis sistem membuat tahapan validasi pengguna menjadi lebih ketat sehingga meminimalisir risiko terjadinya akses serta pengiriman transaksi yang tidak sah. Mekanisme penyimpanan data yang terpisah antara aset yang disimpan di buku besar dan pengguna di basis data *backend* membuat beban kerja jaringan menjadi lebih ringan sehingga dapat meningkatkan kinerja dari sistem dalam memproses transaksi. Namun, sistem *blockchain* MeetCoin yang dikembangkan memiliki kekurangan dalam hal penggunaan sumber daya secara keseluruhan. Sistem *blockchain* MeetCoin terdiri tiga komponen yang berjalan bersamaan yang membuat sistem membutuhkan sumber daya yang besar untuk dapat berjalan.

BAB V

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Tugas akhir ini mengembangkan sistem *blockchain* MeetCoin menggunakan kerangka kerja Hyperledger Fabric dengan mempertimbangkan beberapa aspek yang meliputi keandalan, keamanan, dan kinerja. Tugas akhir ini juga mengevaluasi kinerja dari sistem yang dikembangkan untuk mengidentifikasi kemampuan sistem serta dapat dijadikan sebagai evaluasi untuk pengembangan sistem di masa depan. Sistem yang dikembangkan terdiri dari tiga komponen utama yang meliputi jaringan Fabric, Fabric Gateway, dan *backend*. Evaluasi kinerja sistem dilakukan melalui pengujian kinerja menggunakan alat Hyperledger Caliper dengan beberapa skenario.

Sistem yang dikembangkan memiliki konfigurasi dua *peer node* untuk mencegah sistem mengalami *single point of failure* sehingga sistem memiliki keandalan yang baik ketika salah satu *node* mengalami kegagalan. Selain itu, konfigurasi dua *peer node* membuat buku besar yang berisi data aset dapat terdistribusikan ke semua *peer node* untuk menjaga konsistensi dan integritas data. Implementasi Certificate Authority (CA) yang mengelola identitas pengguna dalam jaringan Fabric dan implementasi JWT di *backend* bertujuan untuk meningkatkan keamanan sistem dari risiko adanya pengiriman transaksi atau akses yang tidak sah ke dalam sistem. Transaksi yang dikirim juga perlu disahkan oleh kedua *peer node* untuk mengurangi risiko manipulasi data transaksi.

Pada evaluasi kinerja sistem, terdapat tiga skenario yang disusun dan diuji pada proses pengisian saldo, pengecekan saldo, dan transfer saldo. Skenario pertama bertujuan untuk melihat pengaruh dari perubahan jumlah *worker* yang mengirimkan transaksi terhadap indikator kinerja sistem yang meliputi *success rate*, *throughput*, *latency*, dan *resource consumption*. Skenario kedua bertujuan untuk melihat pengaruh dari perubahan jumlah transaksi dan skenario ketiga bertujuan untuk melihat pengaruh dari laju transaksi terhadap kinerja sistem. Hasil evaluasi kinerja yang dilakukan dapat disimpulkan sebagai berikut.

1. Tingkat keberhasilan sistem dalam menangani transaksi pada semua skenario dan jenis transaksi mencapai 100% yang menunjukkan bahwa sistem dapat menangani beban kerja yang diberikan tanpa mengalami kendala.
2. Penambahan jumlah *worker* tidak memengaruhi *throughput* secara signifikan, tetapi cenderung meningkatkan *latency* dan penggunaan CPU serta sedikit meningkatkan penggunaan memori.
3. Penambahan jumlah transaksi tidak memengaruhi *throughput*, tetapi cenderung

meningkatkan penggunaan CPU dan memori dengan *latency* yang cukup stabil.

4. Peningkatan laju transaksi secara signifikan meningkatkan *throughput* dan penggunaan CPU, sementara penggunaan memori mengalami sedikit peningkatan dan *latency* sedikit menurun.

Sistem *blockchain* MeetCoin yang dikembangkan dan dievaluasi pada tugas akhir ini masih memiliki kekurangan. Sistem yang terdiri dari tiga komponen yang berjalan bersama membutuhkan sumber daya yang besar untuk dapat berjalan. Dalam lingkungan *production*, hal tersebut dapat membuat biaya operasional dari sistem menjadi tinggi.

5.2 Saran

Berdasarkan kesimpulan dari tugas akhir ini, terdapat beberapa saran yang dapat dijadikan bahan pertimbangan untuk dapat dikembangkan dan ditambahkan pada penelitian atau tugas akhir selanjutnya. Saran tersebut di antaranya adalah sebagai berikut.

1. Mengoptimalkan arsitektur sistem agar penggunaan sumber daya untuk menjalankan sistem secara keseluruhan menjadi lebih ringan.
2. Mengembangkan evaluasi kinerja sistem dengan skenario yang lebih kompleks untuk mendapatkan hasil evaluasi yang lebih beragam dan lebih mencerminkan kondisi pada dunia nyata.

DAFTAR PUSTAKA

- [1] J. Zhang, “Ethereum vs fabric vs corda: Blockchain protocols compared,” [Accessed: 06-Aug-2024]. [Online]. Available: <https://www.kaleido.io/blockchain-blog/enterprise-blockchain-protocols-a-technical-analysis-of-ethereum-vs-fabric-vs-corda>
- [2] A. A. Monrat, O. Schelén, and K. Andersson, “Performance evaluation of permissioned blockchain platforms,” in *2020 IEEE Asia-Pacific Conference on Computer Science and Data Engineering (CSDE)*, 2020, pp. 1–8.
- [3] Besu, “Welcome | besu documentation,” [Accessed: 06-Aug-2024]. [Online]. Available: <https://besu.hyperledger.org/>
- [4] A. Abrol, “Hyperledger vs corda vs ethereum: A detailed comparison,” Mar 2022. [Online]. Available: <https://www.blockchain-council.org/blockchain/hyperledger-vs-corda-vs-ethereum/>
- [5] Fabric, “The ordering service - hyperledger fabric docs main documentation,” [Accessed: 03-Aug-2024]. [Online]. Available: https://hyperledger-fabric.readthedocs.io/en/latest/orderer/ordering_service.html
- [6] G. Yang, K. Lee, K. Lee, Y. Yoo, H. Lee, and C. Yoo, “Resource analysis of blockchain consensus algorithms in hyperledger fabric,” *IEEE Access*, vol. 10, pp. 74 902–74 920, 2022.
- [7] T. Han and K. Gao, “Review of blockchain consensus algorithms,” *Scientific Journal of Intelligent Systems Research*, vol. 2, no. 12, p. 138–144, 2020.
- [8] Fabric, “Couchdb as the state database - hyperledger fabric docs main documentation,” [Accessed: 03-Aug-2024]. [Online]. Available: https://hyperledger-fabric.readthedocs.io/en/latest/couchdb_as_state_database.html
- [9] Apache, “1.1. technical overview - apache couchdb® 3.3 documentation,” [Accessed: 04-Aug-2024]. [Online]. Available: <https://docs.couchdb.org/en/stable/intro/overview.html>
- [10] Google, “google/leveldb.” [Online]. Available: <https://github.com/google/leveldb>
- [11] Caliper, “Hyperledger caliper,” [Accessed: 07-Aug-2024]. [Online]. Available: <https://hyperledger.github.io/caliper/>
- [12] T. T. A. Dinh, J. Wang, G. Chen, R. Liu, B. C. Ooi, and K.-L. Tan, “Blockbench: A framework for analyzing private blockchains,” in *Proceedings of the 2017 ACM International Conference on Management of Data*, ser. SIGMOD ’17. New York, NY, USA: Association for Computing Machinery, 2017, p. 1085–1100. [Online]. Available: <https://doi.org/10.1145/3035918.3064033>
- [13] Fabric, “How fabric networks are structured - hyperledger fabric docs main documentation,” [Accessed: 28-Jul-2024]. [Online]. Available: <https://hyperledger-fabric.readthedocs.io/en/latest/network/network.html>

- [14] —, “Ledger - hyperledger fabric docs main documentation,” [Accessed: 28-Jul-2024]. [Online]. Available: <https://hyperledger-fabric.readthedocs.io/en/latest/ledger/ledger.html>
- [15] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, and et al., “Hyperledger fabric: A distributed operating system for permissioned blockchains,” *Proceedings of the Thirteenth EuroSys Conference*, Apr 2018.
- [16] Kitakabee, “11 agile testing challenges and its solutions,” Jun 2022. [Online]. Available: <https://www.browserstack.com/guide/agile-testing-challenges>
- [17] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008. [Online]. Available: <https://bitcoin.org/bitcoin.pdf>
- [18] J. L. Zhao, S. Fan, and J. Yan, “Overview of business innovations and research opportunities in blockchain and introduction to the special issue,” *Financial Innovation*, vol. 2, no. 1, Dec 2016.
- [19] S. Ghesmati, W. Fdhila, and E. Weippl, “Sok: How private is bitcoin? classification and evaluation of bitcoin privacy techniques,” in *Proceedings of the 17th International Conference on Availability, Reliability and Security*, ser. ARES '22. New York, NY, USA: Association for Computing Machinery, 2022. [Online]. Available: <https://doi.org/10.1145/3538969.3538971>
- [20] A. Altaf, F. Iqbal, R. Latif, B. M. Yakubu, S. Latif, and H. Samiullah, “A survey of blockchain technology: Architecture, applied domains, platforms, and security threats,” *Social Science Computer Review*, vol. 41, no. 5, pp. 1941–1962, 2023. [Online]. Available: <https://doi.org/10.1177/08944393221110148>
- [21] M. J. Tuyisenge, “Blockchain technology security concerns: Literature review,” Ph.D. dissertation, Uppsala University, Disciplinary Domain of Humanities and Social Sciences, Faculty of Social Sciences, Department of Informatics and Media, Information Systems, 2021.
- [22] Hyperledger, “Hyperledger - the open global ecosystem for enterprise blockchain,” [Accessed: 12-Jun-2024]. [Online]. Available: <https://www.hyperledger.org/>
- [23] A. Lawson and C. Page, “Bofa patents increase nearly 70% in 5 years,” Mar 2024, [Accessed 12-Jun-2024]. [Online]. Available: <https://newsroom.bankofamerica.com/content/newsroom/press-releases/2024/03/bofa-patents-increase-nearly-70--in-5-years.html>
- [24] V. Chang, P. Baudier, H. Zhang, Q. Xu, J. Zhang, and M. Arami, “How blockchain can impact financial services – the overview, challenges and recommendations from expert interviewees,” *Technological Forecasting and Social Change*, vol. 158, pp. 120–166, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0040162520309926>
- [25] M. R. Prastyo, “Pengembangan backend aplikasi dompet digital meetcoin berbasis blockchain,” Bachelor’s thesis, Gadjah Mada University, Faculty of Engineering, Department of Electrical dan Information Engineering, Information Engineering, 2022.

- [26] Composer, “Hyperledger composer - create business networks and blockchain applications quickly for hyperledger | hyperledger composer,” [Accessed: 20-Jul-2024]. [Online]. Available: <https://hyperledger.github.io/composer/latest/>
- [27] Hyperledger, “hyperledger/fabric.” [Online]. Available: <https://github.com/hyperledger/fabric>
- [28] P. Pourkhomami, “The risks of using outdated software systems and hardware,” Oct 2023, [Accessed: 06-Aug-2024]. [Online]. Available: <https://www.osibeyond.com/blog/dangers-of-outdated-software-hardware/>
- [29] R. K. Kausal, N. Kumar, S. Makka, and K. Saluja, “Demystifying hyperledger fabric framework for distributed ledgers and approach to evaluate its performance,” *2023 International Conference on Sustainable Emerging Innovations in Engineering and Technology (ICSEIET)*, Sep 2023.
- [30] M. Uddin, K. Salah, R. Jayaraman, S. Pesic, and S. Ellahham, “Blockchain for drug traceability: Architectures and open challenges,” *Health Informatics Journal*, vol. 27, no. 2, pp. 1–15, Apr 2021.
- [31] A. Srivastava and Y. Desai, “Performance analysis of hyperledger fabric based blockchain for traceability in food supply chain,” in *2021 IEEE 2nd International Conference on Technology, Engineering, Management for Societal impact using Marketing, Entrepreneurship and Talent (TEMSMET)*, 2021, pp. 1–5.
- [32] S. Vidwans, A. Deshpande, P. Thakur, A. Verma, and S. Palwe, “Permissioned blockchain voting system using hyperledger fabric,” in *2022 International Conference on IoT and Blockchain Technology (ICIBT)*, 2022, pp. 1–6.
- [33] G. Al-Sumaidae, R. Alkhudary, Z. Zilic, and A. Swidan, “Performance analysis of a private blockchain network built on hyperledger fabric for healthcare,” *Information Processing Management*, vol. 60, no. 2, p. 103160, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306457322002618>
- [34] G. Tripathi, M. A. Ahad, and G. Casalino, “A comprehensive review of blockchain technology: Underlying principles and historical background with future challenges,” *Decision Analytics Journal*, vol. 9, p. 100344, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2772662223001844>
- [35] R. Weerawarna, S. J. Miah, and X. Shao, “Emerging advances of blockchain technology in finance: A content analysis,” *Personal and Ubiquitous Computing*, vol. 27, no. 4, p. 1495–1508, Feb 2023.
- [36] W.-M. Lee, *Beginning Ethereum Smart Contracts Programming: With examples in python, solidity, and JavaScript*. Apress, 2023.
- [37] J. Zarrin, H. Wen Phang, L. Babu Saheer, and B. Zarrin, “Blockchain for decentralization of internet: Prospects, trends, and challenges,” *Cluster Computing*, vol. 24, no. 4, p. 2841–2866, May 2021.
- [38] Z. Zheng, S. Xie, H. Dai, X. Chen, and H. Wang, “An overview of blockchain technology: Architecture, consensus, and future trends,” in *2017 IEEE International Congress on Big Data (BigData Congress)*, 2017, pp. 557–564.

- [39] M. Chand, “Practical byzantine fault tolerance (pbft): Building trust in distributed systems,” Apr 2024, [Accessed: 23-Jul-2024]. [Online]. Available: <https://medium.com/@mehar.chand.cloud/practical-byzantine-fault-tolerance-pbft-building-trust-in-distributed-systems-41183c668b93>
- [40] C. Campbell, “What are the 4 different types of blockchain technology?: Techtarget,” Jun 2024, [Accessed: 23-Jul-2024]. [Online]. Available: <https://www.techtarget.com/searchcio/feature/What-are-the-4-different-types-of-blockchain-technology>
- [41] M. G. Tiwari and K. A. Kumar, “Decentralization system using smart blockchain with secure hash,” *SN Computer Science*, vol. 4, no. 5, Aug 2023.
- [42] S. Perera, S. Nanayakkara, M. Rodrigo, S. Senaratne, and R. Weinand, “Blockchain technology: Is it hype or real in the construction industry?” *Journal of Industrial Information Integration*, vol. 17, pp. 100–125, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2452414X20300017>
- [43] Fabric, “Introduction - hyperledger fabric docs main documentation,” [Accessed: 28-Jul-2024]. [Online]. Available: <https://hyperledger-fabric.readthedocs.io/en/latest/whatis.html>
- [44] R. Pujadas, E. Valderrama, and W. Venters, “The value and structuring role of web apis in digital innovation ecosystems: The case of the online travel ecosystem,” *Research Policy*, vol. 53, no. 2, p. 104931, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0048733323002159>
- [45] Go, “The go programming language,” [Accessed: 24-Jul-2024]. [Online]. Available: <https://go.dev/>
- [46] R3, “Corda enterprise vs corda open source - enterprise 4.12,” Jul 2024, [Accessed: 06-Aug-2024]. [Online]. Available: <https://docs.r3.com/en/platform/corda/4.12/enterprise/about-corda/enterprise-vs-community.html>
- [47] Fabric, “Performance considerations - hyperledger fabric docs main documentation.” [Online]. Available: <https://hyperledger-fabric.readthedocs.io/en/latest/performance.html>

LAMPIRAN

L.1 *Source Code* Konfigurasi *Node Jaringan Fabric*

```
1 # Copyright IBM Corp. All Rights Reserved.
2 #
3 # SPDX-License-Identifier: Apache-2.0
4 #
5
6 version: '3.7'
7
8 volumes:
9   orderer.example.com:
10    peer0.org1.example.com:
11    peer0.org2.example.com:
12
13 networks:
14   test:
15     name: fabric_test
16
17 services:
18
19   orderer.example.com:
20     container_name: orderer.example.com
21     image: hyperledger/fabric-orderer:2.5.9
22     labels:
23       service: hyperledger-fabric
24     environment:
25       - FABRIC_LOGGING_SPEC=INFO
26       - ORDERER_GENERAL_LISTENADDRESS=0.0.0.0
27       - ORDERER_GENERAL_LISTENPORT=7050
28       - ORDERER_GENERAL_LOCALMSPID=OrdererMSP
29       - ORDERER_GENERAL_LOCALMSPDIR=/var/hyperledger/orderer/msp
30       # enabled TLS
31       - ORDERER_GENERAL_TLS_ENABLED=true
32       - ORDERER_GENERAL_TLS_PRIVATEKEY=/var/hyperledger/orderer/
33         tls/server.key
34       - ORDERER_GENERAL_TLS_CERTIFICATE=/var/hyperledger/orderer/
35         tls/server.crt
36       - ORDERER_GENERAL_TLS_ROOTCAS=[/var/hyperledger/orderer/
37         tls/ca.crt]
38       - ORDERER_GENERAL_CLUSTER_CLIENTCERTIFICATE=/var/
```

```

hyperledger/orderer/tls/server.crt
36     - ORDERER_GENERAL_CLUSTER_CLIENTPRIVATEKEY=/var/
hyperledger/orderer/tls/server.key
37     - ORDERER_GENERAL_CLUSTER_ROOTCAS=[/var/hyperledger/
orderer/tls/ca.crt]
38     - ORDERER_GENERAL_BOOTSTRAPMETHOD=none
39     - ORDERER_CHANNELPARTICIPATION_ENABLED=true
40     - ORDERER_ADMIN_TLS_ENABLED=true
41     - ORDERER_ADMIN_TLS_CERTIFICATE=/var/hyperledger/orderer/
tls/server.crt
42     - ORDERER_ADMIN_TLS_PRIVATEKEY=/var/hyperledger/orderer/
tls/server.key
43     - ORDERER_ADMIN_TLS_ROOTCAS=[/var/hyperledger/orderer/tls/
ca.crt]
44     - ORDERER_ADMIN_TLS_CLIENTROOTCAS=[/var/hyperledger/
orderer/tls/ca.crt]
45     - ORDERER_ADMIN_LISTENADDRESS=0.0.0.0:7053
46     - ORDERER_OPERATIONS_LISTENADDRESS=orderer.example.com
:9443
47     - ORDERER_METRICS_PROVIDER=prometheus
48     working_dir: /root
49     command: orderer
50     volumes:
51     - ../organizations/ordererOrganizations/example.com/
orderers/orderer.example.com/msp:/var/hyperledger/orderer/msp
52     - ../organizations/ordererOrganizations/example.com/
orderers/orderer.example.com/tls:/var/hyperledger/orderer/
tls
53     - orderer.example.com:/var/hyperledger/production/
orderer
54     ports:
55     - 7050:7050
56     - 7053:7053
57     - 9443:9443
58     networks:
59     - test
60
61 peer0.org1.example.com:
62     container_name: peer0.org1.example.com
63     image: hyperledger/fabric-peer:2.5.9
64     labels:

```

```

65     service: hyperledger-fabric
66     environment:
67         - FABRIC_CFG_PATH=/etc/hyperledger/peerconfig
68         - FABRIC_LOGGING_SPEC=INFO
69         #- FABRIC_LOGGING_SPEC=DEBUG
70         - CORE_PEER_TLS_ENABLED=true
71         - CORE_PEER_PROFILE_ENABLED=false
72         - CORE_PEER_TLS_CERT_FILE=/etc/hyperledger/fabric/tls/
server.crt
73         - CORE_PEER_TLS_KEY_FILE=/etc/hyperledger/fabric/tls/
server.key
74         - CORE_PEER_TLS_ROOTCERT_FILE=/etc/hyperledger/fabric/tls/
ca.crt
75         # Peer specific variables
76         - CORE_PEER_ID=peer0.org1.example.com
77         - CORE_PEER_ADDRESS=peer0.org1.example.com:7051
78         - CORE_PEER_LISTENADDRESS=0.0.0.0:7051
79         - CORE_PEER_CHAINCODEADDRESS=peer0.org1.example.com:7052
80         - CORE_PEER_CHAINCODELISTENADDRESS=0.0.0.0:7052
81         - CORE_PEER_GOSSIP_BOOTSTRAP=peer0.org1.example.com:7051
82         - CORE_PEER_GOSSIP_EXTERNALENDPOINT=peer0.org1.example.com
:7051
83         - CORE_PEER_LOCALMSPID=Org1MSP
84         - CORE_PEER_MSPCONFIGPATH=/etc/hyperledger/fabric/msp
85         - CORE_OPERATIONS_LISTENADDRESS=peer0.org1.example.com
:9444
86         - CORE_METRICS_PROVIDER=prometheus
87         - CHAINCODE_AS_A_SERVICE_BUILDER_CONFIG={"peername ":"
peer0org1"}
88         - CORE_CHAINCODE_EXECUTETIMEOUT=300s
89     volumes:
90         - ../organizations/peerOrganizations/org1.example.com/
peers/peer0.org1.example.com:/etc/hyperledger/fabric
91         - peer0.org1.example.com:/var/hyperledger/production
92     working_dir: /root
93     command: peer node start
94     ports:
95         - 7051:7051
96         - 9444:9444
97     networks:
98         - test

```

```

99
100 peer0.org2.example.com:
101     container_name: peer0.org2.example.com
102     image: hyperledger/fabric-peer:2.5.9
103     labels:
104         service: hyperledger-fabric
105     environment:
106         - FABRIC_CFG_PATH=/etc/hyperledger/peerconfig
107         - FABRIC_LOGGING_SPEC=INFO
108         #- FABRIC_LOGGING_SPEC=DEBUG
109         - CORE_PEER_TLS_ENABLED=true
110         - CORE_PEER_PROFILE_ENABLED=false
111         - CORE_PEER_TLS_CERT_FILE=/etc/hyperledger/fabric/tls/
server.crt
112         - CORE_PEER_TLS_KEY_FILE=/etc/hyperledger/fabric/tls/
server.key
113         - CORE_PEER_TLS_ROOTCERT_FILE=/etc/hyperledger/fabric/tls/
ca.crt
114         # Peer specific variables
115         - CORE_PEER_ID=peer0.org2.example.com
116         - CORE_PEER_ADDRESS=peer0.org2.example.com:9051
117         - CORE_PEER_LISTENADDRESS=0.0.0.0:9051
118         - CORE_PEER_CHAINCODEADDRESS=peer0.org2.example.com:9052
119         - CORE_PEER_CHAINCODELISTENADDRESS=0.0.0.0:9052
120         - CORE_PEER_GOSSIP_EXTERNALENDPOINT=peer0.org2.example.com
:9051
121         - CORE_PEER_GOSSIP_BOOTSTRAP=peer0.org2.example.com:9051
122         - CORE_PEER_LOCALMSPID=Org2MSP
123         - CORE_PEER_MSPCONFIGPATH=/etc/hyperledger/fabric/msp
124         - CORE_OPERATIONS_LISTENADDRESS=peer0.org2.example.com
:9445
125         - CORE_METRICS_PROVIDER=prometheus
126         - CHAINCODE_AS_A_SERVICE_BUILDER_CONFIG={"peername":"
peer0org2"}
127         - CORE_CHAINCODE_EXECUTETIMEOUT=300s
128     volumes:
129         - ../organizations/peerOrganizations/org2.example.com/
peers/peer0.org2.example.com:/etc/hyperledger/fabric
130         - peer0.org2.example.com:/var/hyperledger/production
131     working_dir: /root
132     command: peer node start

```

```
133     ports :
134         - 9051:9051
135         - 9445:9445
136     networks :
137         - test
```

L.2 *Source Code* Konfigurasi Certificate Authority (CA) Jaringan Fabric

```
1 # Copyright IBM Corp. All Rights Reserved.
2 #
3 # SPDX-License-Identifier: Apache-2.0
4 #
5
6 version: '3.7'
7
8 networks:
9   test:
10     name: fabric_test
11
12 services:
13
14   ca_org1:
15     image: hyperledger/fabric-ca:latest
16     labels:
17       service: hyperledger-fabric
18     environment:
19       - FABRIC_CA_HOME=/etc/hyperledger/fabric-ca-server
20       - FABRIC_CA_SERVER_CA_NAME=ca-org1
21       - FABRIC_CA_SERVER_TLS_ENABLED=true
22       - FABRIC_CA_SERVER_PORT=7054
23       - FABRIC_CA_SERVER_OPERATIONS_LISTENADDRESS=0.0.0.0:17054
24     ports:
25       - "7054:7054"
26       - "17054:17054"
27     command: sh -c 'fabric-ca-server start -b admin:adminpw -d'
28     volumes:
29       - ../organizations/fabric-ca/org1:/etc/hyperledger/fabric-ca-server
30     container_name: ca_org1
31     networks:
32       - test
33
34   ca_org2:
35     image: hyperledger/fabric-ca:latest
36     labels:
37       service: hyperledger-fabric
38     environment:
39       - FABRIC_CA_HOME=/etc/hyperledger/fabric-ca-server
```

```

40     - FABRIC_CA_SERVER_CA_NAME=ca-org2
41     - FABRIC_CA_SERVER_TLS_ENABLED=true
42     - FABRIC_CA_SERVER_PORT=8054
43     - FABRIC_CA_SERVER_OPERATIONS_LISTENADDRESS=0.0.0.0:18054
44 ports:
45     - "8054:8054"
46     - "18054:18054"
47 command: sh -c 'fabric-ca-server start -b admin:adminpw -d'
48 volumes:
49     - ../organizations/fabric-ca/org2:/etc/hyperledger/fabric-
ca-server
50 container_name: ca_org2
51 networks:
52     - test
53
54 ca_orderer:
55     image: hyperledger/fabric-ca:latest
56     labels:
57         service: hyperledger-fabric
58     environment:
59         - FABRIC_CA_HOME=/etc/hyperledger/fabric-ca-server
60         - FABRIC_CA_SERVER_CA_NAME=ca-orderer
61         - FABRIC_CA_SERVER_TLS_ENABLED=true
62         - FABRIC_CA_SERVER_PORT=9054
63         - FABRIC_CA_SERVER_OPERATIONS_LISTENADDRESS=0.0.0.0:19054
64     ports:
65         - "9054:9054"
66         - "19054:19054"
67     command: sh -c 'fabric-ca-server start -b admin:adminpw -d'
68     volumes:
69         - ../organizations/fabric-ca/ordererOrg:/etc/hyperledger/
fabric-ca-server
70     container_name: ca_orderer
71     networks:
72         - test

```

L.3 *Source Code* Konfigurasi *Channel* dan Mekanisme Konsensus Jaringan Fabric

```
1 # Copyright IBM Corp. All Rights Reserved.
2 #
3 # SPDX-License-Identifier: Apache-2.0
4 #
5
6 Organizations:
7   - &OrdererOrg
8     Name: OrdererOrg
9     ID: OrdererMSP
10    MSPDir: ../organizations/ordererOrganizations/example.com/
11    msp
12    Policies:
13      Readers:
14        Type: Signature
15        Rule: "OR('OrdererMSP.member')"
16      Writers:
17        Type: Signature
18        Rule: "OR('OrdererMSP.member')"
19      Admins:
20        Type: Signature
21        Rule: "OR('OrdererMSP.admin')"
22    OrdererEndpoints:
23      - orderer.example.com:7050
24  - &Org1
25    Name: Org1MSP
26    ID: Org1MSP
27    MSPDir: ../organizations/peerOrganizations/org1.example.com/
28    msp
29    Policies:
30      Readers:
31        Type: Signature
32        Rule: "OR('Org1MSP.admin', 'Org1MSP.peer', 'Org1MSP.
33        client')"
34      Writers:
35        Type: Signature
36        Rule: "OR('Org1MSP.admin', 'Org1MSP.client')"
37      Admins:
38        Type: Signature
39        Rule: "OR('Org1MSP.admin')"
```

```

37     Endorsement:
38         Type: Signature
39         Rule: "OR( 'Org1MSP.peer' )"
40 - &Org2
41     Name: Org2MSP
42     ID: Org2MSP
43     MSPDir: ../organizations/peerOrganizations/org2.example.com/
msp
44     Policies:
45         Readers:
46             Type: Signature
47             Rule: "OR( 'Org2MSP.admin', 'Org2MSP.peer', 'Org2MSP.
client' )"
48         Writers:
49             Type: Signature
50             Rule: "OR( 'Org2MSP.admin', 'Org2MSP.client' )"
51         Admins:
52             Type: Signature
53             Rule: "OR( 'Org2MSP.admin' )"
54     Endorsement:
55         Type: Signature
56         Rule: "OR( 'Org2MSP.peer' )"
57 Capabilities:
58     Channel: &ChannelCapabilities
59     V2_0: true
60     Orderer: &OrdererCapabilities
61     V2_0: true
62     Application: &ApplicationCapabilities
63     V2_5: true
64 Application: &ApplicationDefaults
65     Organizations:
66     Policies:
67         Readers:
68             Type: ImplicitMeta
69             Rule: "ANY Readers"
70         Writers:
71             Type: ImplicitMeta
72             Rule: "ANY Writers"
73         Admins:
74             Type: ImplicitMeta
75             Rule: "MAJORITY Admins"

```

```

76     LifecycleEndorsement :
77         Type: ImplicitMeta
78         Rule: "MAJORITY Endorsement "
79     Endorsement :
80         Type: ImplicitMeta
81         Rule: "MAJORITY Endorsement "
82     Capabilities :
83         <<: *ApplicationCapabilities
84 Orderer: &OrdererDefaults
85     Addresses :
86         - orderer.example.com:7050
87     BatchTimeout: 2s
88     BatchSize :
89         MaxMessageCount: 10
90         AbsoluteMaxBytes: 99 MB
91         PreferredMaxBytes: 512 KB
92     Organizations :
93     Policies :
94         Readers :
95             Type: ImplicitMeta
96             Rule: "ANY Readers "
97         Writers :
98             Type: ImplicitMeta
99             Rule: "ANY Writers "
100    Admins :
101        Type: ImplicitMeta
102        Rule: "MAJORITY Admins "
103    BlockValidation :
104        Type: ImplicitMeta
105        Rule: "ANY Writers "
106 Channel: &ChannelDefaults
107     Policies :
108         Readers :
109             Type: ImplicitMeta
110             Rule: "ANY Readers "
111         Writers :
112             Type: ImplicitMeta
113             Rule: "ANY Writers "
114         Admins :
115             Type: ImplicitMeta
116             Rule: "MAJORITY Admins "

```

```

117 Capabilities :
118     <<: *ChannelCapabilities
119 Profiles :
120 ChannelUsingRaft :
121     <<: *ChannelDefaults
122     Orderer :
123         <<: *OrdererDefaults
124         OrdererType: etcdraft
125         EtcdRaft :
126             Consenters :
127                 - Host: orderer.example.com
128                   Port: 7050
129                   ClientTLSCert: ../organizations/ordererOrganizations
130                               /example.com/orderers/orderer.example.com/tls/server.crt
131                   ServerTLSCert: ../organizations/ordererOrganizations
132                               /example.com/orderers/orderer.example.com/tls/server.crt
133             Organizations :
134                 - *OrdererOrg
135             Capabilities: *OrdererCapabilities
136 Application :
137     <<: *ApplicationDefaults
138     Organizations :
139         - *Org1
140         - *Org2
141     Capabilities: *ApplicationCapabilities

```

L.4 Source Code Fungsi Mint pada Chaincode

```
1 func (s *SmartContract) Mint(ctx contractapi.  
    TransactionContextInterface, amount int) error {  
2  
3     // Check if contract has been initialized first  
4     initialized, err := checkInitialized(ctx)  
5     if err != nil {  
6         return fmt.Errorf("failed to check if contract is already  
            initialized: %v", err)  
7     }  
8     if !initialized {  
9         return fmt.Errorf("Contract options need to be set before  
            calling any function, call Initialize() to initialize  
            contract")  
10    }  
11  
12    // Check minter authorization – this sample assumes Org1 is  
        the central banker with privilege to mint new tokens  
13    clientMSPID, err := ctx.GetClientIdentity().GetMSPID()  
14    if err != nil {  
15        return fmt.Errorf("failed to get MSPID: %v", err)  
16    }  
17    if clientMSPID != "Org1MSP" {  
18        return fmt.Errorf("client is not authorized to mint new  
            tokens")  
19    }  
20  
21    // Get ID of submitting client identity  
22    minter, err := ctx.GetClientIdentity().GetID()  
23    if err != nil {  
24        return fmt.Errorf("failed to get client id: %v", err)  
25    }  
26  
27    if amount <= 0 {  
28        return fmt.Errorf("mint amount must be a positive integer")  
29    }  
30  
31    currentBalanceBytes, err := ctx.GetStub().GetState(minter)  
32    if err != nil {  
33        return fmt.Errorf("failed to read minter account %s from  
            world state: %v", minter, err)
```

```

34     }
35
36     var currentBalance int
37
38     // If minter current balance doesn't yet exist, we'll create
39     // it with a current balance of 0
40     if currentBalanceBytes == nil {
41         currentBalance = 0
42     } else {
43         currentBalance, _ = strconv.Atoi(string(currentBalanceBytes))
44         // Error handling not needed since Itoa() was used when
45         // setting the account balance, guaranteeing it was an integer.
46     }
47
48     updatedBalance, err := add(currentBalance, amount)
49     if err != nil {
50         return err
51     }
52
53     err = ctx.GetStub().PutState(minter, []byte(strconv.Itoa(
54         updatedBalance)))
55     if err != nil {
56         return err
57     }
58
59     // Update the totalSupply
60     totalSupplyBytes, err := ctx.GetStub().GetState(totalSupplyKey)
61     if err != nil {
62         return fmt.Errorf("failed to retrieve total token supply: %v", err)
63     }
64
65     var totalSupply int
66
67     // If no tokens have been minted, initialize the totalSupply
68     if totalSupplyBytes == nil {
69         totalSupply = 0
70     } else {
71         totalSupply, _ = strconv.Atoi(string(totalSupplyBytes)) //
72         // Error handling not needed since Itoa() was used when setting

```

```

    the totalSupply , guaranteeing it was an integer.
68 }
69
70 // Add the mint amount to the total supply and update the
    state
71 totalSupply , err = add(totalSupply , amount)
72 if err != nil {
73     return err
74 }
75
76 err = ctx .GetStub().PutState(totalSupplyKey , []byte(strconv .
    Itoa(totalSupply)))
77 if err != nil {
78     return err
79 }
80
81 // Emit the Transfer event
82 transferEvent := event{"0x0" , minter , amount}
83 transferEventJSON , err := json.Marshal(transferEvent)
84 if err != nil {
85     return fmt.Errorf("failed to obtain JSON encoding: %v" , err)
86 }
87 err = ctx .GetStub().SetEvent("Transfer" , transferEventJSON)
88 if err != nil {
89     return fmt.Errorf("failed to set event: %v" , err)
90 }
91
92 log.Printf("minter account %s balance updated from %d to %d" ,
    minter , currentBalance , updatedBalance)
93
94 return nil
95 }

```

L.5 Source Code Fungsi Burn pada Chaincode

```
1 func (s *SmartContract) Burn(ctx contractapi.  
    TransactionContextInterface, amount int) error {  
2  
3     // Check if contract has been initialized first  
4     initialized, err := checkInitialized(ctx)  
5     if err != nil {  
6         return fmt.Errorf("failed to check if contract is already  
            initialized: %v", err)  
7     }  
8     if !initialized {  
9         return fmt.Errorf("Contract options need to be set before  
            calling any function, call Initialize() to initialize  
            contract")  
10    }  
11    // Check minter authorization – this sample assumes Org1 is  
        the central banker with privilege to burn new tokens  
12    clientMSPID, err := ctx.GetClientIdentity().GetMSPID()  
13    if err != nil {  
14        return fmt.Errorf("failed to get MSPID: %v", err)  
15    }  
16    if clientMSPID != "Org1MSP" {  
17        return fmt.Errorf("client is not authorized to mint new  
            tokens")  
18    }  
19  
20    // Get ID of submitting client identity  
21    minter, err := ctx.GetClientIdentity().GetID()  
22    if err != nil {  
23        return fmt.Errorf("failed to get client id: %v", err)  
24    }  
25  
26    if amount <= 0 {  
27        return errors.New("burn amount must be a positive integer")  
28    }  
29  
30    currentBalanceBytes, err := ctx.GetStub().GetState(minter)  
31    if err != nil {  
32        return fmt.Errorf("failed to read minter account %s from  
            world state: %v", minter, err)  
33    }  
}
```

```

34
35     var currentBalance int
36
37     // Check if minter current balance exists
38     if currentBalanceBytes == nil {
39         return errors.New("The balance does not exist")
40     }
41
42     currentBalance, _ = strconv.Atoi(string(currentBalanceBytes))
43     // Error handling not needed since Itoa() was used when
44     // setting the account balance, guaranteeing it was an integer.
45
46     updatedBalance, err := sub(currentBalance, amount)
47     if err != nil {
48         return err
49     }
50
51     err = ctx.GetStub().PutState(minter, []byte(strconv.Itoa(
52         updatedBalance)))
53     if err != nil {
54         return err
55     }
56
57     // Update the totalSupply
58     totalSupplyBytes, err := ctx.GetStub().GetState(totalSupplyKey
59         )
60     if err != nil {
61         return fmt.Errorf("failed to retrieve total token supply: %v", err)
62     }
63
64     // If no tokens have been minted, throw error
65     if totalSupplyBytes == nil {
66         return errors.New("totalSupply does not exist")
67     }
68
69     totalSupply, _ := strconv.Atoi(string(totalSupplyBytes)) //
70     // Error handling not needed since Itoa() was used when setting
71     // the totalSupply, guaranteeing it was an integer.
72
73     // Subtract the burn amount to the total supply and update the

```

```

    state
68     totalSupply , err = sub(totalSupply , amount)
69     if err != nil {
70         return err
71     }
72
73     err = ctx.GetStub().PutState(totalSupplyKey , []byte(strconv.
        Itoa(totalSupply)))
74     if err != nil {
75         return err
76     }
77
78     // Emit the Transfer event
79     transferEvent := event{minter , "0x0" , amount}
80     transferEventJSON , err := json.Marshal(transferEvent)
81     if err != nil {
82         return fmt.Errorf("failed to obtain JSON encoding: %v" , err)
83     }
84     err = ctx.GetStub().SetEvent("Transfer" , transferEventJSON)
85     if err != nil {
86         return fmt.Errorf("failed to set event: %v" , err)
87     }
88
89     log.Printf("minter account %s balance updated from %d to %d" ,
        minter , currentBalance , updatedBalance)
90
91     return nil
92 }

```

L.6 Source Code Fungsi Transfer pada Chaincode

```
1 func (s *SmartContract) Transfer(ctx contractapi.  
    TransactionContextInterface, recipient string, amount int)  
    error {  
2  
3    // Check if contract has been intilized first  
4    initialized, err := checkInitialized(ctx)  
5    if err != nil {  
6        return fmt.Errorf("failed to check if contract is already  
            initialized: %v", err)  
7    }  
8    if !initialized {  
9        return fmt.Errorf("Contract options need to be set before  
            calling any function, call Initialize() to initialize  
            contract")  
10    }  
11  
12    // Get ID of submitting client identity  
13    clientID, err := ctx.GetClientIdentity().GetID()  
14    if err != nil {  
15        return fmt.Errorf("failed to get client id: %v", err)  
16    }  
17  
18    err = transferHelper(ctx, clientID, recipient, amount)  
19    if err != nil {  
20        return fmt.Errorf("failed to transfer: %v", err)  
21    }  
22  
23    // Emit the Transfer event  
24    transferEvent := event{clientID, recipient, amount}  
25    transferEventJSON, err := json.Marshal(transferEvent)  
26    if err != nil {  
27        return fmt.Errorf("failed to obtain JSON encoding: %v", err)  
28    }  
29    err = ctx.GetStub().SetEvent("Transfer", transferEventJSON)  
30    if err != nil {  
31        return fmt.Errorf("failed to set event: %v", err)  
32    }  
33  
34    return nil  
35 }
```

```

36
37 func transferHelper(ctx contractapi.TransactionContextInterface ,
    from string , to string , value int) error {
38
39     if from == to {
40         return fmt.Errorf("cannot transfer to and from same client
            account")
41     }
42
43     if value < 0 { // transfer of 0 is allowed in ERC-20, so just
        validate against negative amounts
44         return fmt.Errorf("transfer amount cannot be negative")
45     }
46
47     fromCurrentBalanceBytes , err := ctx.GetStub().GetState(from)
48     if err != nil {
49         return fmt.Errorf("failed to read client account %s from
            world state: %v", from , err)
50     }
51
52     if fromCurrentBalanceBytes == nil {
53         return fmt.Errorf("client account %s has no balance", from)
54     }
55
56     fromCurrentBalance , _ := strconv.Atoi(string(
        fromCurrentBalanceBytes)) // Error handling not needed since
        Itoa() was used when setting the account balance ,
        guaranteeing it was an integer.
57
58     if fromCurrentBalance < value {
59         return fmt.Errorf("client account %s has insufficient funds"
            , from)
60     }
61
62     toCurrentBalanceBytes , err := ctx.GetStub().GetState(to)
63     if err != nil {
64         return fmt.Errorf("failed to read recipient account %s from
            world state: %v", to , err)
65     }
66
67     var toCurrentBalance int

```

```

68 // If recipient current balance doesn't yet exist, we'll
   create it with a current balance of 0
69 if toCurrentBalanceBytes == nil {
70     toCurrentBalance = 0
71 } else {
72     toCurrentBalance, _ = strconv.Atoi(string(
toCurrentBalanceBytes)) // Error handling not needed since
Itoa() was used when setting the account balance,
guaranteeing it was an integer.
73 }
74
75 fromUpdatedBalance, err := sub(fromCurrentBalance, value)
76 if err != nil {
77     return err
78 }
79
80 toUpdatedBalance, err := add(toCurrentBalance, value)
81 if err != nil {
82     return err
83 }
84
85 err = ctx.GetStub().PutState(from, []byte(strconv.Itoa(
fromUpdatedBalance)))
86 if err != nil {
87     return err
88 }
89
90 err = ctx.GetStub().PutState(to, []byte(strconv.Itoa(
toUpdatedBalance)))
91 if err != nil {
92     return err
93 }
94
95 log.Printf("client %s balance updated from %d to %d", from,
fromCurrentBalance, fromUpdatedBalance)
96 log.Printf("recipient %s balance updated from %d to %d", to,
toCurrentBalance, toUpdatedBalance)
97
98 return nil
99 }

```

L.7 *Source Code Konfigurasi Network pada Caliper*

```
1 name: Caliper test
2 version: "2.0.0"
3
4 caliper:
5   blockchain: fabric
6
7 channels:
8   - channelName: mychannel
9     contracts:
10    - id: basic
11
12 organizations:
13   - mspid: Org1MSP
14     identities:
15       certificates:
16         - name: 'User1'
17           clientPrivateKey:
18             path: '../hlf-mtcnv2/test-network/organizations/
19               peerOrganizations/org1.example.com/users/User1@org1.example.
20               com/msp/keystore/priv_sk'
21           clientSignedCert:
22             path: '../hlf-mtcnv2/test-network/organizations/
23               peerOrganizations/org1.example.com/users/User1@org1.example.
24               com/msp/signcerts/cert.pem'
25       connectionProfile:
26         path: '../hlf-mtcnv2/test-network/organizations/
27               peerOrganizations/org1.example.com/connection-org1.yaml'
```

L.8 *Source Code Konfigurasi Benchmark* pada Caliper

```
1 test:
2   name: basic-contract-benchmark
3   description: test benchmark
4   workers:
5     number: 5
6   rounds:
7     - label: Mint
8       description: Mint some coins
9       txNumber: 1000
10      rateControl:
11        type: fixed-rate
12        opts:
13          tps: 50
14      workload:
15        module: workload/mint.js
16        arguments:
17          contractId: basic
18    - label: Balance
19      description: Get balance
20      txNumber: 1000
21      rateControl:
22        type: fixed-rate
23        opts:
24          tps: 150
25      workload:
26        module: workload/balance.js
27        arguments:
28          contractId: basic
29    - label: Transfer
30      description: Transfer some coins
31      txNumber: 1000
32      rateControl:
33        type: fixed-rate
34        opts:
35          tps: 150
36      workload:
37        module: workload/transfer.js
38        arguments:
39          contractId: basic
40
```

```
41 monitors:
42     resource:
43         - module: docker
44         options:
45             interval: 1
46             cpuUsageNormalization: true
47             containers:
48                 - /peer0.org1.example.com
49                 - /orderer.example.com
50                 - /peer0.org2.example.com
```

L.9 Source Code Modul Workload Mint pada Caliper

```
1  'use strict';
2
3  const { WorkloadModuleBase } = require('@hyperledger/caliper-
    core');
4
5  class MintWorkload extends WorkloadModuleBase {
6      constructor() {
7          super();
8      }
9
10     async initializeWorkloadModule(workerIndex, totalWorkers,
roundIndex, roundArguments, sutAdapter, sutContext) {
11         await super.initializeWorkloadModule(workerIndex,
totalWorkers, roundIndex, roundArguments, sutAdapter,
sutContext);
12         console.log('Worker ${this.workerIndex}: Initializing
Mint Test ');
13     }
14
15     async submitTransaction() {
16         const amount = '1000';
17         const args = {
18             contractId: this.roundArguments.contractId,
19             contractFunction: 'Mint',
20             invokerIdentity: 'User1',
21             contractArguments: [amount],
22             readOnly: false
23         };
24
25         await this.sutAdapter.sendRequests(args);
26     }
27 }
28
29 function createWorkloadModule() {
30     return new MintWorkload();
31 }
32
33 module.exports.createWorkloadModule = createWorkloadModule;
```

L.10 *Source Code Modul Workload Balance pada Caliper*

```
1  'use strict';
2
3  const { WorkloadModuleBase } = require('@hyperledger/caliper-
    core');
4
5  class BalanceWorkload extends WorkloadModuleBase {
6      constructor() {
7          super();
8      }
9
10     // Initialize the workload with the parameters needed
11     async initializeWorkloadModule(workerIndex, totalWorkers,
        roundIndex, roundArguments, sutAdapter, sutContext) {
12         await super.initializeWorkloadModule(workerIndex,
            totalWorkers, roundIndex, roundArguments, sutAdapter,
            sutContext);
13
14         console.log('Worker ${this.workerIndex}: Initializing
            Balance Test');
15     }
16
17     async submitTransaction() {
18         // Construct the transaction request for the Balance
        function
19         const args = {
20             contractId: this.roundArguments.contractId,
21             contractFunction: 'ClientAccountBalance',
22             invokerIdentity: 'User1',
23             readOnly: true
24         };
25
26         await this.sutAdapter.sendRequests(args);
27     }
28 }
29
30 function createWorkloadModule() {
31     return new BalanceWorkload();
32 }
33
34 module.exports.createWorkloadModule = createWorkloadModule;
```

L.11 *Source Code Modul Workload Transfer pada Caliper*

```
1  'use strict';
2
3  const { WorkloadModuleBase } = require('@hyperledger/caliper-
    core');
4
5  class TransferWorkload extends WorkloadModuleBase {
6      constructor() {
7          super();
8      }
9
10     // Initialize the workload with the parameters needed
11     async initializeWorkloadModule(workerIndex, totalWorkers,
        roundIndex, roundArguments, sutAdapter, sutContext) {
12         await super.initializeWorkloadModule(workerIndex,
            totalWorkers, roundIndex, roundArguments, sutAdapter,
            sutContext);
13
14         console.log('Worker ${this.workerIndex}: Initializing
            Transfer Test');
15
16         const args = {
17             contractId: this.roundArguments.contractId,
18             contractFunction: 'MintTest',
19             invokerIdentity: 'User1',
20             contractArguments: ['100000000'],
21             readOnly: false
22         };
23
24         await this.sutAdapter.sendRequests(args);
25     }
26
27     async submitTransaction() {
28
29         const receiver = "
        eDUwOT06Q049VXNlcjIsT1U9Y2xpZW50LE89SHlwZXJsZWRnZXIsU1Q9Tm9ydGggQ2Fyb2xp
        "; // User2 account ID
30
31         // Construct the transaction request for the Transfer
        function
32         const args = {
```

```

33         contractId: this.roundArguments.contractId ,
34         contractFunction: 'Transfer ',
35         invokerIdentity: 'User1 ',
36         contractArguments: [receiver , '1000'],
37         readOnly: false
38     };
39
40     await this.sutAdapter.sendRequests( args );
41 }
42
43 async cleanupWorkloadModule() {
44     console.log( 'Worker ${this.workerIndex}: Transfer test
completed ' );
45 }
46 }
47
48 function createWorkloadModule() {
49     return new TransferWorkload();
50 }
51
52 module.exports.createWorkloadModule = createWorkloadModule;

```